



MASTER THESIS

Accelerating Unreal Engine Compilation by Utilizing Developer Workstations

Author:

Hidde Derks

Supervisor:

Thomas Buijtenweg MSc

Jeremiah van Oosten MSc

*This thesis is submitted in fulfilment of the requirements
for the degree of Master of Science in Game Technology*

In the programme

Professional Master Game Technology

Academy for AI, Games and Media

Breda University of Applied Sciences

Date

Declaration of Authorship

I, Hidde Derks, declare that this thesis titled, “Accelerating Unreal Engine Compilation by Utilizing Developer Workstations” and the work presented are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:



Date: 7th of June, 2026

“This study taught me how to view things through an even more analytical lens. I’ve always strived to understand the underlying why, but it was refreshing to have to write that down instead of just fix it.”

Hidde Derks

Abstract

Academy for AI, Games and Media

Master of Game Technology

Accelerating Unreal Engine Compilation by Utilizing Developer Workstations

By Hidde Derks

Compilation in Unreal Engine is time-intensive and prone to disrupting developer workflows as they go off-task, waiting for builds to complete. This can be alleviated by accelerating the compilation through distributing work across dedicated compile agents. Deploying dedicated hardware is not always feasible though, particularly in educational or small studio environments, where acquisition costs of such hardware can be hard to justify. Existing research primarily focuses on software-level optimization, which is why this project investigates the potential of using existing developer workstations as an alternative to dedicated compute. Technical Action Research was used to evaluate different configurations of using Unreal Horde and the Unreal Build Accelerator across three teams. Complementary scaling tests across 216 configurations provided an estimate of speedup potential. Scaling tests showed that speedup follows a predictable reciprocal curve, with parallelizability ranging from 53.2% to 92.5%, depending on task size. This translates to theoretical speedups ranging from 2.1x to 13.4x faster than unaccelerated builds. It was also found that the RAM-per-core ratio was a strong predictor of negative impacts on workstation responsiveness when contributing to compilation. The current state of the system can already meaningfully improve compile times without incurring additional hardware costs, and future development will likely only improve this further. The scheduling overhead of using multiple smaller agents means that for pure performance, it is still outperformed by stronger, dedicated compile agents. This trade-off, along with other practical considerations, is addressed through the deployment guide and configuration recommendations produced by this research.

Acknowledgements

AI Usage

Before thanking people, I want to add an acknowledgement of AI usage. Over the course of this thesis, Anthropic's Claude LLM (under a BUAs provided GDPR-compliant data-processing agreement) has been used to provide feedback on the thesis as a supplement to supervisor feedback. All feedback given by Claude has been critically reviewed to gauge validity. None of the writing was done by Claude, occasionally taking inspiration from suggestions, but never copying suggested paragraphs word-for-word.

This thesis would not have reached its current state without the support of several people, whom I would like to thank here.

First, I would like to thank both of my supervisors. Thomas Buijtenweg for the support in learning academic writing and research methodologies, and Jeremiah van Oosten for the technical feedback and critique.

Second, I would also like to thank my participants for their openness to testing the Unreal Build Accelerator in their daily workflows, which offered valuable insight into the practical deployment.

Finally, I would like to thank my peers in the MGT programme, for the good vibes in the classroom, and the regular sparring sessions on the approach and the contents of our research projects.

Contents

Declaration of Authorship.....	II
Abstract.....	IV
Acknowledgements.....	V
AI Usage	V
List of Figures	IX
List of Tables.....	X
List of Abbreviations	XI
Chapter 1: Introduction (Introduction to the thesis topic)	0
1.1 Compilation in Game Development	0
1.2 Optimizing Multiple Projects with Ease	2
Chapter 2: Literature Review (Background information and theory).....	3
2.1 Accelerating the Compilation Process Through Distributed Compilation.....	3
2.1.1 Contributor to Unreal Engine 5.4 & 5.5 Source	8
2.1.2 More Powerful Hardware.....	8
2.2 Skipping the Compilation Process Through Distributed Caching	8
2.3 Continuous Integration & Continuous Deployment Systems.....	9
2.4 Integration with Developers.....	10
2.5 Educational context.....	12
2.6 Research Gap	14
2.7 Research Questions	15
Chapter 3: Methodology.....	17
3.1 Prerequisite services and impacted development aspects	18
3.2 Testing methodology for answering Sub-Question 2 (Scaling Tests).....	22
3.2.1 Switch from Portainer to Kubernetes	25
3.2.2 Wine	25
3.2.3 Compute stability limitation	26
3.3 The TAR cycle.....	26
3.3.1 Preliminary survey test runs	29
3.4 Participants and Contexts	29
3.5 Data analysis strategy.....	30
3.6 Ethics	33
3.7 Hypotheses.....	34
3.7.1 Sub-Question 1: What services need to be set up for teams to benefit from distributed caching and compilation?	34

3.7.2	<i>Sub-Question 2: What build time reductions can be expected from implementing distributed caching and compilation in specific configuration contexts?</i>	34
3.7.3	<i>Sub-Question 3: How should distributed caching and compilation be configured for developers to obtain ideal workflows?</i>	35
3.7.4	<i>Sub-Question 4: What configuration trade-offs exist between cluster hardware variance, compile performance, and developer workflows?</i>	35
3.7.5	<i>Sub-Question 5: What practical trade-offs exist when increasing hardware variance of the cluster?</i>	36
3.8	Limitations	36
	Chapter 4: Experiment Results	39
4.0.1	<i>Participant numbering</i>	39
4.1	First Iteration	39
4.2	Second Iteration	40
4.3	Third Iteration	43
4.4	TAR Overview	44
4.5	Core Scaling Tests	44
4.5.1	<i>Aitken Extrapolation (Delta Squared)</i>	49
4.5.2	<i>Lost Trace Files</i>	49
4.6	Agent Scaling Tests	50
4.7	RAM Scaling Tests	51
4.8	VPN Scaling Tests	53
	Chapter 5: Discussion of Data and Results	55
5.1	Sub-Question 1: What services need to be set up for teams to benefit from distributed caching and compilation?	55
5.2	Sub-Question 2: What build time reductions can be expected from implementing distributed caching and compilation in specific configuration contexts?	56
5.2.1	<i>NetBird Relay Mode</i>	58
5.2.2	<i>Aitken's Extrapolation</i>	58
5.3	Sub-Question 3: How should distributed caching and compilation be configured for developers to obtain ideal workflows?	59
5.4	Sub-Question 4: What configuration trade-offs exist between cluster hardware variance, compile performance, and developer workflows?	60
5.5	Sub-Question 5: What practical trade-offs exist when increasing hardware variance of the cluster?	61
5.6	Additional Finding 1	63
5.7	Additional Finding 2	64
5.8	Additional Finding 3	64

5.9	Main Research Question: How can short term Unreal Engine projects improve their development processes by creating and distributing compiled data by utilizing existing infrastructure in a more optimized way?	65
	Chapter 6: Conclusion and Future Directions.....	69
	Appendix A: Horde installation guides	71
A.1	Unreal Horde Server Deployment (Windows).....	71
A.2	Unreal Horde Server Deployment (Docker).....	71
A.3	Unreal Horde Agent Deployment (Windows)	71
A.4	Unreal Horde Agent Deployment (Docker)	71
	Appendix B: Scaling Test agents' Kubernetes Manifests	72
B.1	Main manifest.....	72
B.2	agent.json.....	73
B.3	entrypoint.sh	74
	Appendix C: TAR Survey Data	76
C.1	Iteration 1	76
	C.1.1 Additional Notes	77
C.2	Iteration 2	78
	C.2.1 Additional Notes	80
C.3	Iteration 3.....	81
	C.3.1 Additional Notes	83
	Appendix D: Core / Agent Scaling Test Data	85
	Appendix E: RAM Scaling Test Data.....	89
	Appendix F: Evolutionary Python Script.....	91
	Bibliography.....	92

List of Figures

Figure 1. Developers making changes to code	1
Figure 2. Developer (re)starting their project to have the most up-to-date version	1
Figure 3. Unreal Build Accelerator Visualizer. Screenshot from distributed compile test that shows the compile broken up into which module was compiled by which agent. Along with stats, failures, and the failure reasons (such as the researcher's ATLAS-WS agent disconnecting and reconnecting, shown by the gap at the bottom)	7
Figure 4. Unreal Horde analytics dashboard. Screenshot from Epic Games documentation, via Unreal Engine Documentation. (https://dev.epicgames.com/documentation/en-us/unreal-engine/horde-analytics-tutorial-for-unreal-engine).....	11
Figure 5. Elements that should be removed from ignore files.....	19
Figure 6. Example Perforce typemap changes for keeping .dll, .exe, .pdb files writable in client workspaces, and keeping no more than 2 revisions of .dll and .exe (in case of needing a rollback), and no more than 1 revision for .pdb files (largest files; purely for debugging, not needed for functionality of rollback).....	19
Figure 7. Developer making an iterative change to code (daily if not hourly process).....	19
Figure 8. Developer making a large change to code (daily to weekly process).....	20
Figure 9. Merging conflicting changes (weekly process).....	20
Figure 10. Game build (weekly to monthly process).....	20
Figure 11. Disruptions such as new engine releases, plugin additions, or other changes (occasionally, but very high impact)	20
Figure 12. Developer (re)starting their project to have the most up-to-date version (daily if not hourly process).....	21
Figure 13. NetBird Horde access policy	21
Figure 14. Docker compose agent specification for 8 (physical) cores (16 threads) and 4GB of RAM	25
Figure 15. System Usability Scale score calculation formula	30
Figure 16. Compile time scaling of a single helper agent with varying amounts of CPU cores for the UE5 target.	45
Figure 17. Compile time scaling of a single helper agent with varying amounts of CPU cores for the UnrealInsights & UnrealPackageTool targets	46
Figure 18. Speedup scaling of a single helper agent with varying amounts of CPU cores.	46
Figure 19. Multiplicative inverse function (reciprocal) adjusted by constants A, B, and C to match the data points gathered. x is the variable representing the amount of remote cores. The outcome y represents the predicted time of compilation.	47
Figure 20. Multiplicative inverse function with constants for UE5 target.....	47
Figure 21. Multiplicative inverse constants for UnrealInsights target.....	48
Figure 22. Multiplicative inverse constants for UnrealPackageTool target.....	48
Figure 23. Multiplicative inverse function plotted on graph, predicting scaling up to 96 cores for the UE5 target. Error bars represent minimum and maximum observed values.	48
Figure 24. Multiplicative inverse functions plotted on graph, predicting scaling up to 96 cores for the UnrealInsights and UnrealPackageTool targets. Error bars represent minimum and maximum observed values.....	49
Figure 25. Formula to get the percentage of parallel work (p).	49
Figure 26. Compile time scaling of 32 cores distributed across a varying amount of agents for the UE5 target.	50

Figure 27. Compile time scaling of 32 cores distributed across a varying amount of agents for the UnrealInsights and UnrealPackageTool targets.....	50
Figure 28. Speedup scaling of varying amount of agents with 8 cores assigned to them.	51
Figure 29. Multiplicative inverse constants for UE5 target with 4 agents. Since x is measured as cores/agent, it is multiplied by 4 to get the total cores assisting in the compile.....	51
Figure 30. Multiplicative inverse constants for UE5 target with 8 agents.....	51
Figure 31. UE5 Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.....	52
Figure 32. UnrealInsights Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.....	52
Figure 33. UnrealPackageTool Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.....	53
Figure 34. Cropped UbaVisualizer trace showing inactivity on the initiator (empty space between top green row) due to agents (blue rows on bottom) picking up tasks and failing to finish them in a timely manner. Only after the agents disconnect (end of blue bar) can the initiator pick up those final few tasks to complete the session. Full trace file can be found as: Group-1/RAM-Test-1GB-UnrealInsights-Target-1.uba.....	53
Figure 35. Flowchart steps to deploy Horde for distributed compilation	66

List of Tables

Table 1. Potential implications of different hardware configurations when connected to the cluster.....	6
Table 2. Educational characteristics not covered in existing research	12
Table 3. Educational ethics framework presented by Pereira et al. [36], adjusted for higher education.....	13
Table 4. Variations of tests that were run to evaluate scalability	24
Table 5. Survey questions, SUS augmented with system-specific questions.....	28
Table 6. Hypothesis of required components to benefit from distributed caching and compilation, along with their purpose in the process.....	34
Table 7. Expected compile speed multiplier for small, incremental compile tasks, along with the resulting time it would take	34
Table 8. Expected compile speed multiplier for large, parallelizable compile tasks, along with the resulting time it would take	35
Table 9. CPU and RAM configurations of participant's machines. Only devices that appeared in compile session trace files have been included. Multiple participants appearing in the same row does not indicate an identical laptop, merely similar counts in logical threads and RAM capacity.	41

List of Abbreviations

API	Application Programming Interface
BUas	Breda University of Applied Sciences
CD	Continuous Deployment
CDE	Continuous Delivery
CI	Continuous Integration
CMGT	Creative Media and Game Technologies
CPU	Central Processing Unit
DDC	Derived Data Cache
DNS	Domain Name System
DRAM	Dynamic Random-Access Memory
GB	Gigabyte
GiB	Gibibyte
GRG	Generalized Reduced Gradient
RPC	Remote Procedure Call
HEDT	High-End Desktop
HTTP(S)	Hypertext Transfer Protocol (Secure)
ICMP	Internet Control Message Protocol
IP	Internet Protocol
KPI	Key Performance Indicator
LAN	Local Area Network
MGT	Master Game Technology
MSRP	manufacturer's suggested retail price
NAT	Network Address Translation
NUMA	Non-Uniform Memory Access
OIDC	Open ID Connect
OS	Operating System
P2P	Peer-to-Peer
PCH	Precompiled Header
PAR	Participatory Action Research
QUIC	Quick UDP Internet Connections
RAM	Random Access Memory
RMSE	Root Mean Square Error
SCM	Source Control Management

STUN	Session Traversal Utilities for NAT
SUS	System Usability Scale
TAR	Technical Action Research
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TURN	Traversal Using Relays around NAT
UBA	Unreal Build Accelerator
UBT	Unreal Build Tool
UDP	User Datagram Protocol
UE	Unreal Engine
UHT	Unreal Header Tool
VLAN	Virtual Local Area Network
VPN	Virtual Private Network
XML	Extensible Markup Language

For/Dedicated to/To my...

Chapter 1: Introduction (Introduction to the thesis topic)

In game development, there is the process of compiling code within projects. Compilation is the process of taking human-readable source code into code that a computer can understand and run. Usually, this process of compilation is a time-intensive process [22]. The Unreal Engine for example, can take half an hour or more to compile from source [9]. This often requires developers to wait for the process to complete before they can continue working on that same task. The longer this process takes, the higher the chances are that the developer goes “off-task” making it more cognitively demanding to get back to the task at hand once the build has finished [27].

1.1 Compilation in Game Development

Compilation exists in all software development, but game development introduces some unique challenges compared to conventional software development. Especially the game engines upon which games are built provide these large differences:

- Very large C++ codebases (>60k C++ files in Unreal’s case)
- Real-time graphics requiring high-performing code
- Real-time graphics requiring thousands of shader variations
- Target platform cross-compilation (different operating system, game consoles, mobile)

For these reasons, compile durations are especially noticeable within game development. Therefore, game development has the potential to benefit to a great extent from these pipeline improvements. Because these improvements are reliant on the size of the codebase, a larger codebase will also display more meaningful time improvements that have a lower chance of falling within the margin of error. Longer compilation times are less susceptible to performance variations from secondary system tasks, making measurements more reliable across different acceleration configurations.

Different optimization approaches exist to reduce compilation time, either through language optimizations, alterations in code complexity, or differences in the size of the codebase; but there are also methods of accelerating these processes. The methods that are interesting for this paper are distributed caching and distributed compilation. Both of these methods focus on sharing resources between a cluster of machines that are connected over the network.

Distributed caching is a type of incremental compilation, it caches compile results. Performing (re)compiles of only the files that have changed since the last time. If the changes are minimal enough, there is barely any new code to compile, and therefore can greatly improve the time it takes to compile a codebase. This is usually already implemented, but only for the local machine. This paper will explore if distributed caching across networked machines is a greater benefit than local caching.

36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59

The second method is simply adding more processing power to the task for it to complete faster, which in the case of distributed compilation is done by combining multiple PCs into a collective build farm that can distribute the compile load across a cluster of machines instead of a singular machine. C++ code, including game engines, parallelizes effectively due to its modular structure with thousands of independent modules. While dependencies exist between modules, many can be compiled in parallel [40]. This should be especially easy to implement with recent developments in Unreal Engine’s own Continuous Integration (CI) tools.

There are several stages within development that touch upon compilation and where this thesis aims to accelerate development. To clarify which stages, a simplified breakdown has been made of different points of compilation, and how this research might impact those workflows (more detailed breakdown will follow in Chapter 3):

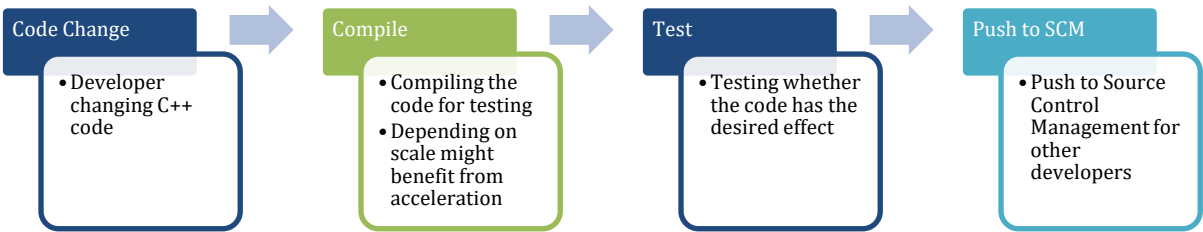


Figure 1. Developers making changes to code

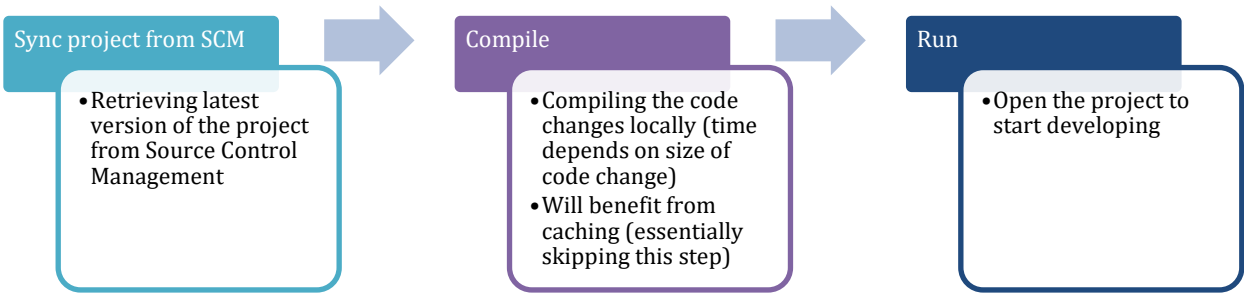


Figure 2. Developer (re)starting their project to have the most up-to-date version

In these diagrams, the sections highlighted are the ones this research will focus on or otherwise impact. Green shows segments in the workflow where distributed compilation can have an impact; Purple shows segments impacted by distributed caching, which should allow these sections to be skipped (bar some potential edge cases); Cyan shows segments that will receive small tweaks to the files pushed to Source Control Management (SCM); And lastly dark blue shows segments that will remain unchanged.

1.2 Optimizing Multiple Projects with Ease

Optimization of Building and Compilation is something that is not new, but these systems are set up on a specialized per-project basis. According to several conversations with lecturers and other experts in the educational field, this is lacking and under-explored within game development education, whilst this is a prime example of many short-term projects, often running simultaneously, all sharing the same server infrastructure and compute resources. Therefore, education could serve as a great and varied sample-set to see how a generalized setup of systems could integrate with all these varying projects. Along with offering the benefits of these improvements to students in their academic projects.

This leads into how projects are set up by default; they start from a blank slate, each time having to integrate with CI and acceleration services separately. Templates or guides could be considered to offer easy integration with these systems. To find out how to best do so, this paper will attempt to answer the question:

How can short term Unreal Engine projects improve their development processes by creating and distributing compiled data by utilizing existing infrastructure in a more optimized way?

This question, and several sub questions emerge from the research gap in existing research on build systems, CI, and developer productivity explored and detailed in Chapter 2.

Chapter 2: Literature Review (Background information and theory)

This chapter will be a review of existing academic and industry literature on build system optimization. With a focus on distributed compilation and caching, and the effects on developer productivity.

Sources for this literature review have been explored using a combination of Google Scholar, the Breda University of Applied Sciences (BUas) library, and regular google searches. Specifically searching with a concept-centric approach [47]. Supplemented by Anthropic's AI Claude for additional sources and keywords. All sources were manually evaluated before inclusion to negate any potential for hallucination. Due to some areas lacking peer-reviewed academic sources, industry documentation and sources have been used to augment the academic literature.

2.1 Accelerating the Compilation Process Through Distributed Compilation

The process of compiling source code is a time-consuming process. This is due to the computer having to process and optimize the source code into executable binaries through several stages of computation:

- Preprocessing: Resolving header files
- Compilation: Translating source code into optimized machine code
- Linking: Combining object files and connecting function calls

These stages result in a self-contained binary executable that can run on its own [22], ready for shipping or for further testing. The time it takes to compile scales with the size of the source code that is being compiled. This is especially apparent in the context of Unreal Engine, which also has its own Unreal Header Tool (UHT) to parse any Unreal specific macros (UCLASS, UPROPERTY, UFUNCTION) and generate additional header files containing reflection bindings for the preprocessor to resolve. This is done to enable other Unreal systems such as Blueprint scripting, garbage collection, and other engine systems. Collectively this can take between 10-40 minutes to compile from source as mentioned in the documentation [9].

Existing literature focused on accelerating the process of compiling source code is heavily focused on the code that performs the compilation, as can be seen from a recent systematic literature review that analyzed 97 papers [1], but lacked focus on the distributed infrastructure or hardware as means of accelerating builds. Other papers explore scheduling improvements through dependency tracking [40], scalable build systems capable of handling 15 million daily builds [46], and preventing out of memory

errors down to only 0.08% through batching [45]. There was also a paper regarding caching improvements, which showed that ~83-97% of the recompilations were unnecessary [50]. There is however also literature that is more focused on the underlying architecture on which these compilation systems run. A 2008 paper explored declarative operating systems for defining build farms, which could vastly improve maintenance and reproducibility [8]. Another paper explored the possibility of utilizing idle computer resources for distributed compilation systems across the network to better utilize already existing resources [30]. Another paper also explored using non-dedicated hardware (such as idle developer PCs) to form a distributed compilation cluster, though they did find a potential scaling issue with the amount of compute that the developer's workstation had compared to the resources available on a distributed cluster [20]. A highly relevant paper from the high-energy physics community addresses similar large C++ codebases that require compilation. In this paper they used C++ to benchmark and prove the viability of several different compilation accelerations (amongst which distributed compilation using distcc). Matev reports this was 'well received by developers' [31]. However, specific details or results of implementation are not shown. The paper is peer-reviewed and published by CERN software engineer Rosen Matev, leading to little doubt about the trustworthiness.

Finally, there is also a less academic, but contextually relevant source. Epic Games has released a tool called the Unreal Build Accelerator (UBA) along with Unreal Engine 5.4 [10, 11]. This tool allows a compile task to be distributed across multiple compile agents across the network. UBA is part of a larger system called Unreal Horde, which has been successfully implemented by Epic Games themselves internally [3, 42], there is some doubt whether this is due to this being a very purpose-built tool, but from preliminary tests, it has shown reliable acceleration in personal contexts as well. Their documentation also guides developers on how to set it up, though does occasionally miss out on prerequisites or complexities, making implementation more challenging. An example of this would be the guide on hosting the horde server in Docker [12]; this used to point to a Docker image for the horde server that works, but did not include any of the tool downloads (such as the UBA agent installer) which are essential for developers to connect to Horde and start using the UBA. These Docker images have since been updated to include the tool downloads, but would require the image to be created locally from the full engine's source code. Despite the documentation missing some details, it is the official source and accurate in the details that it does cover.

Despite substantial academic research on compilation software optimizations, this thesis will focus on hardware infrastructure for distributed compilation. Leveraging the new UBA software provided by Epic Games, which natively integrates with Unreal Engine. As in the context of Unreal Engine, compilation optimization through software has already been widely implemented, even having configuration options for different needs based by means of build configurations [13]. Without changes

to the source code, infrastructure changes such as the ones Matev [31] explored can provide potential for accelerations to complement the software optimizations.

Besides UBA there are also a few other tools that offer distributed compilation. Of these Incredibuild [24] represents a well-established alternative, especially as it has native Unreal integration [25]. However, Incredibuild's free tier used to be limited to 2 developers with 16 remote cores [23], but appears to have removed mentions of a free plan from its site [24]. FASTBuild [19] and BuildXL [34] are other alternatives. The difficulty with both of these would be that they require an additional network storage for each agent to access and do not natively integrate with Unreal's build system. Finally there is distcc, but distcc is not made for a Windows environment. UBA's native integration with Unreal Engine, lack of additional shared network storage requirements, and it is freely included with Unreal; form the reasons for which UBA was selected over alternatives.

Some things that should be taken into consideration are the requirements of network bandwidth and processing power of the machine (the initiator) that is distributing the compile task across the agents that form the compile cluster. This is because the initiator has to run preprocessing steps before it can send a compile chunk to the cluster. If the initiator cannot preprocess chunks fast enough for all compile agents in the cluster, there comes a point of diminishing returns, as shown by Gattermayer and Tvrdik [20]. In their 2012 study, they used 24 computers running Intel® Pentium® G6950s (2 cores, 2 threads, 2.80 GHz), with 4GB of DDR2. These 24 machines were connected in a star topology using 100Mbit Ethernet; and showed scaling issues around 5-6 nodes when using plain distcc and Clondike. Modern hardware has evolved significantly: gigabit ethernet is mainstream, CPUs are commonly 6-8 cores with 12-16 threads (54% of users), with 16-32 GB of RAM (80% of users) at their disposal according to the Steam® hardware survey [44]; Even though Steam can give a decent indication, it is more focused towards players of games rather than creators of games. Yet despite hardware improvements, Matev's [31] study indicated that similar preprocessing limitations were observed with more modern hardware; two servers with dual Intel® Xeon® E5-2630 v4 (10 cores, 20 threads each, 2.20 GHz). Showing that the limitations mentioned by Gattermayer and Tvrdik are still relevant. Alternatively, Matev [31] also explored a "pump" method, to allow distributing the preprocessing step partially across the compile cluster as well, which showed significant reductions in that bottleneck. Another way to improve this might be if multiple machines are initiating compile tasks across the cluster simultaneously, where cluster utilization will be shared between initiators. These requirements and limitations will be determined through different scale tests to evaluate the degree of scalability.

The existing research however does not explore the effects of hardware asymmetry when clustering machines. They had clusters of identical hardware setups, thus had linearly scaling hardware

resources. If in a real case the cluster would consist of multiple different configurations, the hardware resources would not scale linearly. This would be more realistic if actual developer’s machines would be used, as different disciplines might have different requirements or preferences for their machine’s hardware.

As the preprocessing was shown to be a potential limiting factor, different clients within the cluster might benefit to different degrees. A lower end CPU will reach its preprocessing limit sooner than a high end CPU. This leads to an asymmetry in how much a client can utilize or benefit from being part of a cluster:

Low end Client	Lower preprocessing capabilities might not be able to utilize the cluster fully. They will likely outsource a larger portion of the actual compilation to the cluster; and will contribute less total resources to the cluster. Because of this, the benefit of being in a cluster will be high.
High end Client	Higher preprocessing capabilities will be able to utilize the cluster further. They might also perform some of the actual compilation locally; and will contribute more total resources to the cluster. Because of this, the benefit of being in a cluster will be lower.

Table 1. Potential implications of different hardware configurations when connected to the cluster

This could lead to high end clients contributing more power and utilizing less of it, whereas low end clients contribute less power and utilize more. For now the extent of this is unknown, but could be an important consideration when deploying clustering of developer machines into production.

Besides the pure Low/High end Clients, there is also a certain level of unpredictability. Clients being occupied with other tasks from the user using their desktop/laptop, thermal throttling causing slowdowns, or laptops being on their battery saving mode.

Besides unpredictability, compatibility can be considered, though should not give problems. The research is aimed at accelerating Unreal Engine. If a laptop can run Unreal Engine, it is more than capable enough to run the UBA. This is because both are compiled for the same architecture, whether that be Linux or Windows.

This variability of exact compute resources available, combined with the inherent uncertainty of the exact C++ code being compiled, leads to time measurements of compile jobs being an unreliable data source. However, traces of these compile sessions can be performed to analyze the acceleration / parallelization of these compile tasks, as it shows the exact structure of the cluster at every point in the compile, example of which is shown in Figure 3. By analyzing these traces rather than the pure compile

time, a much more detailed analysis can be made of how the compile was performed and accelerated by the UBA.

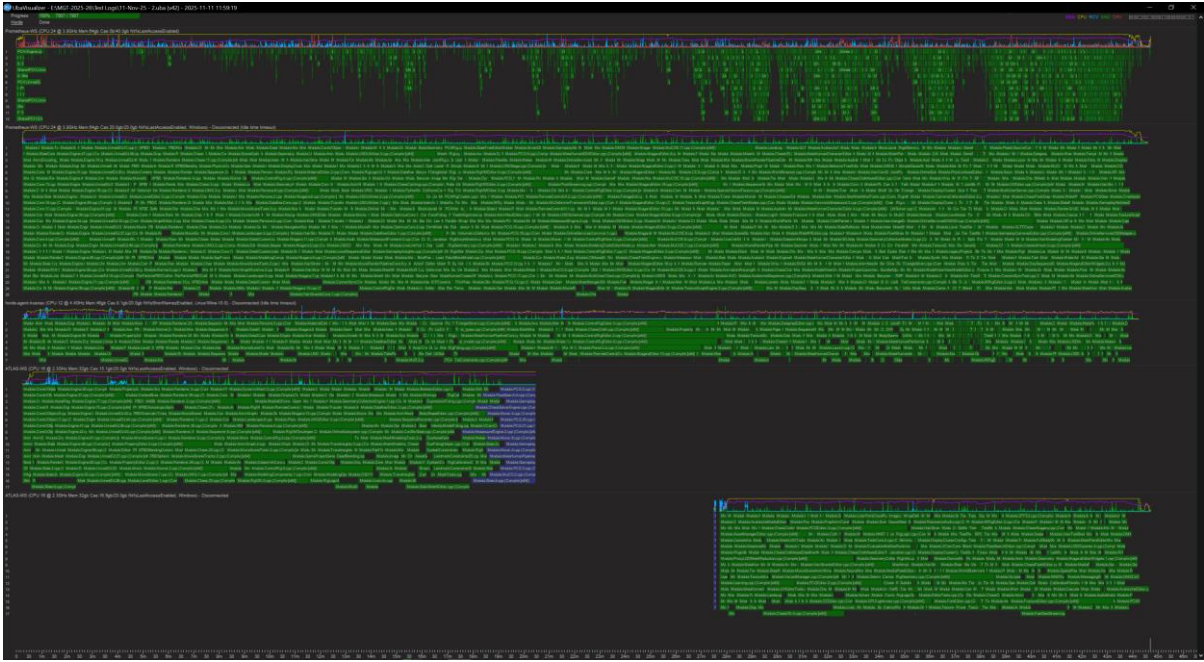


Figure 3. Unreal Build Accelerator Visualizer. Screenshot from distributed compile test that shows the compile broken up into which module was compiled by which agent. Along with stats, failures, and the failure reasons (such as the researcher's ATLAS-WS agent disconnecting and reconnecting, shown by the gap at the bottom)

Another factor that could impact the benefits that a developer might gain from being part of the cluster is where they might be working. If compile accelerations are set up on an office network, developers working from home might not be able to connect to the cluster due to corporate firewall configurations, losing the benefit of acceleration. Likewise the cluster would also lose that developer's machine to parallelize across. This could be alleviated by using a Virtual Private Network (VPN) for the developer's machine to appear as part of the office network, but latency and bandwidth might introduce other bottlenecks.

The Horde system also has potential for configuring different pools of resources. One Horde server can coordinate any amount of pools, it just needs to facilitate the connection between agents. These resource pools can be used to also limit the amount of compute someone can access from the cluster, which can be very helpful in educational contexts; In the case that student teams would like to limit their tasks to their own (team's) hardware. Or potentially to limit the influence on the system as a whole.

Lastly, there is the carbon footprint that could be affected. Theoretically the only difference is where things are compiled, leading to higher compute utilization across shorter time. But the overhead for coordinating and transferring compile jobs does exist. The exact impact of this is outside the scope of

this research, but can become important in actual deployments. At the same time, this could be countered greatly by not having to compile as regularly in the first place through distributed caching, which will be discussed in the following chapter.

2.1.1 Contributor to Unreal Engine 5.4 & 5.5 Source

The Unreal Engine 5.4 release notes also list the author of this paper as a contributor under the online alias ‘Sigma-Erebus’. As do the Unreal Engine 5.5 release notes [14]. In both instances, the code contributions were related to the Unreal Automation Tool crashing when running BuildGraph scripts (XML based language to define build processes with dependency mapping). The author also has several years of professional Unreal experience besides engine source contributions, creating some potential preference bias towards the software. While this background is helpful in providing expertise with the software and tools, objectivity will be retained through reliance on participant feedback and performance measurements.

2.1.2 More Powerful Hardware

Alternatively, compilation could be accelerated by acquiring more powerful hardware. But this solution has upper bounds, as there is a limit to how powerful that a single developer’s machine can get. Getting into such levels of performance often utilizes very expensive high-end desktop (HEDT) level hardware. HEDT uses CPUs such as Intel® Xeon® W processors, or AMD Ryzen™ Threadripper™ series, where the top models have a manufacturer’s suggested retail price (MSRP) in the thousands of US dollars [26]. Especially when these sorts of investments would have to be made for multiple developers, budget requirements will grow drastically compared to the cost of leveraging shared equipment.

2.2 Skipping the Compilation Process Through Distributed Caching

The most effective way to speed up compilation is to not have to do it in the first place; this is what caching can achieve. Caching works on the concept that if the source code and compile flags remain unchanged, the output should remain unchanged as well and could therefore be re-used, skipping the compilation of sections that have already been compiled previously. This means that the first run takes a similar amount of time compared to a clean build without caching.

An important consideration with caching is also the cache hit rate. This is the amount of times that an artifact exists in the cache versus the amount of times it does not. A 0% cache rate means nothing can be re-used, and a 100% cache rate means every cache request is found. The reality often falls between these, with only a part of cache requests hitting. This leads to varying degrees of improvements, based on how much of the cache has been invalidated by code changes.

While local caching saves storage and bandwidth, scaling caching across networked teams can reduce total computational overhead and developer wait times. This can be done by using either a dedicated

270 caching setup or configuring source control management (SCM) to include those files upon syncing
271 with the repository.

272
273 While extensive research exists on compile caching strategies, the aforementioned peer-reviewed
274 paper by the high-energy physics researcher Rosen Matev [31], is one of the few that also goes in depth
275 about distributed compilation as an acceleration strategy. Matev's study evaluated several different
276 compilation optimizations, including distributed caching with tools such as ccache [7], sccache [35],
277 and Memcached [33], which showed from 3.5x (sccache) to 16x (ccache) improvement in time it takes
278 when caching is used compared to uncached builds (at 100% cache hit rate).

279 Most innovation in this area is also done by industry for direct application, rather than by academic
280 research. Another good example of a software which implements this, is Google's Bazel CI system,
281 which aims to be deterministic. It does this by relying on an assumption that the builds it runs are
282 hermetic [21]. Their reasoning is that if the input to a build step remains unchanged, the output should
283 be functionally unchanged as well, albeit not necessarily bitwise identical. Therefore, the result can be
284 substituted with a cached version.

285
286 On the topic of bitwise equivalence. One of the existing risks with caching binaries is the reproducibility
287 of binaries. The results of compilation in Unreal Engine are not deterministic. Even though the same
288 source code yields the same functioning executables, they will not be binary-identical, and will result in
289 different outputs if you were to compare their hashes. There have been developments towards fully
290 deterministic compile processes [37], but most of these developments focus on security and trust
291 verification. Where through compiling the source code yourself, you can ensure that the machine code
292 being executed has not been tampered with. But since security and trust is not the primary topic this
293 thesis aims to explore; The goal will be to create and cache functionally identical binaries, without
294 focusing on pure bitwise equivalence.

296 2.3 Continuous Integration & Continuous Deployment Systems

297 Continuous Integration (CI) and Continuous Deployment (CD) are concepts that aim to provide
298 automated code compilation, testing, and deployment. These often appear as one concept called CI/CD,
299 which indicates the newly integrated code is automatically deployed after testing. Occasionally, CD is
300 also replaced with Continuous Delivery (abbreviated as CDE to distinguish it from CD) which indicates
301 that there is a manual step before the code is deployed into production. For that reason CDE is often
302 preferred for user-facing deployments as a final checkpoint for the development team.

303
304 Academic sources on CI/CD come to the unanimous conclusion that implementing CI/CD practices is
305 beneficial to streamline workflows, improve code quality, and accelerate the delivery of software

products [17, 38, 49]. Some papers expand on just the CI/CD tools to also include the role Source Control Management (SCM) systems such as Git play in ensuring smooth integration between developers [43]. As well as papers focused on specific software packages, namely Jenkins & Teamcity for educational use [16], Bazel for scalable build systems and smart scheduling [46], Jenkins & Gitlab cloud platform integration [39].

This thesis leverages documented CI/CD benefits by applying similar automation to development workflows. Tools such as the UBA mentioned in 2.1 will perform code integration and code deployment, but in contrast to the CI/CD systems mentioned here, will not act on the end-product that will be released. Instead, these systems will focus on integrating and deploying the code that developers need to develop the end-product. Therefore, since CI/CD for end-products is so well received, this thesis aims to test whether similar improvements can be achieved when applying similar automation principles to development workflows using UBA.

2.4 Integration with Developers

According to a paper from Google [27], it is almost as important – if not more important – to clearly communicate such improvements to development teams. As accurate estimations allow them to time their return to the task at hand with a ready compile. For this, the Unreal Horde suite (which UBA is part of) also offers analytics through a dashboard [15]. This would aim to provide developers with accurate reports and visualizations of the metrics. So that they can not only receive the aforementioned improvements, but truly benefit from them.

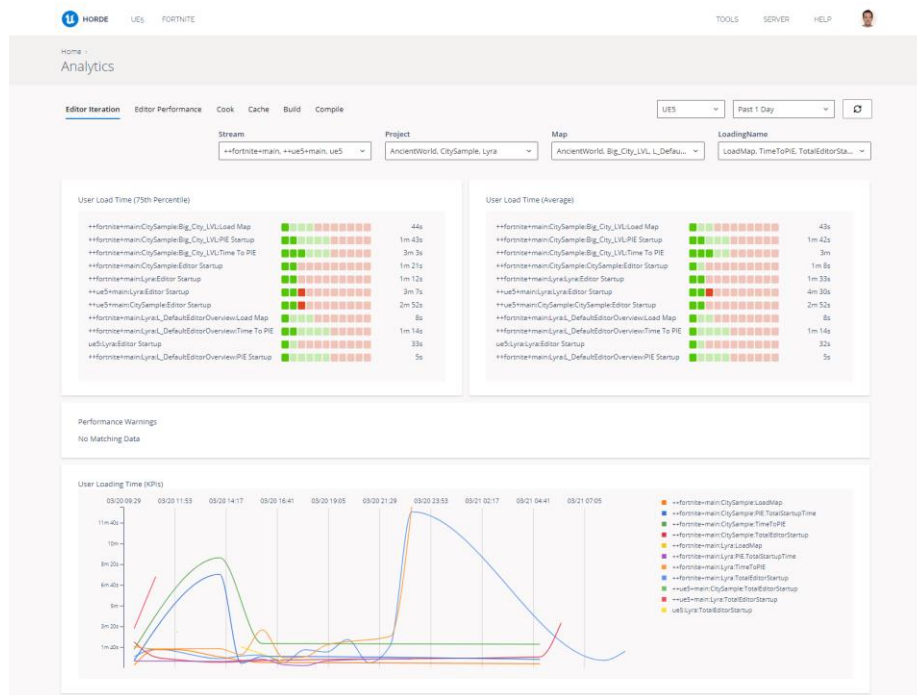


Figure 4. Unreal Horde analytics dashboard. Screenshot from Epic Games documentation, via Unreal Engine Documentation. (<https://dev.epicgames.com/documentation/en-us/unreal-engine/horde-analytics-tutorial-for-unreal-engine>).

To effectively measure the pipeline improvements, the SPACE framework for measuring developer productivity has detailed metrics with which a systematic breakdown of productivity can be made [18]. Within this framework there are 5 criteria:

- Satisfaction & Well-Being
- Performance
- Activity
- Communication & Collaboration
- Efficiency & Flow

Within this framework, this project will be aiming for the metrics of performance and efficiency & flow. Aiming to improve the performance metric by having a system that detects faults in reliability before they impact a developers workflow. Along with the metric of flow, to reduce the amount and length of interruptions during development. Finally, the satisfaction of the work pipeline could also be seen as a potential improvement within this framework.

Metrics collection creates privacy and personal identifiability risks. Analytics systems must either avoid collecting identifying data or actively anonymize it before collection. This will need to be a large consideration during the setup of the metrics and analytics dashboards within Unreal Horde. Despite build time potentially looking like a key performance indicator (KPI), it has no relation to code quality or development efficiency.

Besides communicating the speedup that the system provides, another aspect that will need clear communication is how the system works and what it does; or potentially more importantly, what it does not do. For developers to clearly understand the underlying system, where issues could appear, how to mitigate those potential issues, and to clear up potential misconceptions before they happen.

2.5 Educational context

Educational game development environments present distinct characteristics compared to industry contexts that existing build system research has not covered:

Short term project cycles	Projects run for shorter terms, usually fitting within the span of a block or semester of 8-16 weeks.
Simultaneous projects	a multitude of different projects on shared infrastructure.
Project diversity	projects differ greatly in team size (2-40 students), technical complexity, automation pipeline requirements or target platforms.
Shared infrastructure	Due to projects sharing the same infrastructure, creating challenges of how to allocate these resources to the different projects appropriately.
Learning objectives vs. productivity	The goal is not to deliver an end product, but instead to learn from developing said product

Table 2. Educational characteristics not covered in existing research

Due to these differences, educational contexts provide a very good variety of projects to test the implementation of these optimizations on. Which can provide benefits to educational programs that are set up to teach through practical projects. Given the lack of literature on educational build systems, preliminary consultations with 7 game development education experts confirmed that accelerations in this domain are an unexplored opportunity and a welcome improvement to the pipeline.

Since this research operates in educational contexts, there are some additional considerations to be aware of. As within education, there is a concern for power imbalance between the students and the staff. A good framework was presented by Pereira et al. [36]. It focuses on children in Brazilian education. The core principles that they list have been slightly adjusted for higher education, Since students are adults, parental control provisions are not applicable.

Data Privacy & Control	Empowering students with robust controls over their personal data.
Data Roles Identification	Identify and inform data controllers and processors
Data Retention Policies	Implementing stringent policies to prevent indefinite storage of student's educational data, retaining it only as needed.
Data Integrity & Confidentiality	Ensuring data accuracy and completeness, and restricting access through appropriate security measures.

Data Minimization	Collecting only personal data that is strictly necessary for the specified and legitimate educational purpose
Data Use Limitations	Preventing data use beyond its initially specified educational purpose without obtaining new, explicit consent
Purpose Definition (Educational)	Ensuring data is collected and used solely for educational purposes, strictly prohibiting commercial exploitation
Public Purpose Data Sharing	Managing the sharing of sensitive data obtained from public sector bodies responsibly and with clear public interest
Consent Transparency	Ensuring transparent processes for obtaining informed consent from students for data collection, use, and sharing
Records of Processing Activities	Maintaining comprehensive and accessible records of all data processing operations
Data Proportionality	Ensuring access to essential functionalities is not conditioned on the collection of excessive personal data

Table 3. Educational ethics framework presented by Pereira et al. [36], adjusted for higher education

Within these data protection guidelines, care needs to be taken in how these systems are implemented. Especially the data use limitations, to make sure these systems will not and cannot be used for assessment. As the metrics of how quickly a build gets created might seem like a potential grading criteria, but serve operational purposes such as informing task-switching decisions. This is because they do not reflect the quality of the code or what a student has learnt from it, making it inappropriate for academic assessment.

Another consideration is that there might be a shift in topics for students to focus on. If templates automate the process of setting up these kinds of acceleration services, students that want to explore it for themselves might have less of an opportunity to do so. As templates and implementation have already been provided, there is less benefit in setting it up again; Simultaneously, students working within this framework will need to make sure that their contributions are compatible with such systems. For this, it would also be important for students to know at least the concept of how this system operates and distributes tasks. Both to understand what it is doing, and in what ways it could break. Likely to varying degrees of detail for different disciplines, but it is important to have a base understanding for everyone.

There is also the aforementioned asymmetry in hardware that will likely show up. Especially when students bring their own laptops to work on, there will be a wide variety of hardware arrangements. With a potential for some students to benefit more than others. Though in the field of education, this

might be a positive thing rather than a negative, as it levels the field to give students more equal chances, regardless of background.

Besides the practical achievability of this acceleration, there are also some more psychological aspects to evaluate for effective deployment of these acceleration systems. This is because students might not be as stringent in following rules as a studio and its employees might be. In case students would not want to use an acceleration system such as UBA, especially in times such as deadline weeks; how do you make sure that the benefit is not fully lost on them, or that they take advantage of it in unintended ways. For this reason, there are considerations in how much control clients are given over the cluster. Potentially sectioning off teams using the resource pools mentioned in Section 2.1 such that each team can parallelize across their own hardware. This way they can coordinate amongst themselves much more directly rather than having to coordinate across an entire student population; Whilst adding some isolation and fairness to the system as well, when for example a team contributes to the compute of the cluster, but rarely has to compile something for themselves. These kinds of practical implications and considerations will also be a large part of the recommendations that this thesis aims to provide at the end of the research.

2.6 Research Gap

Existing academic literature details CI/CD optimization at software and hardware levels. However current academic work is very limited in the following areas:

- Impacts on compilation systems based on the infrastructure and hardware setup
- Hardware asymmetry across a distributed compile cluster
- CI/CD systems for short-term projects where set up time may exceed benefits
- Effectiveness within practically organized education with multiple concurrent projects
- Productivity impacts in game development contexts

These aspects form the core gap in which this thesis will make its contribution.

While this research focuses specifically on game development education, the methodologies and implementations could be relevant to other fields. Especially fields where Unreal Engine might already be in use, such as virtual production or serious games. While this research focuses on Unreal Engine, the methodologies and implementation approaches remain relevant to other software development contexts through different tools such as the ones mentioned in Matev's [31] paper. This paper itself also forms a great example for the wide potential of applicability, as this paper originated from the high energy physics field.

2.7 Research Questions

This thesis will not try to address the entire research gap, as that will go far beyond a viable scope. Instead, the scope will be more focused on small-scale educational projects. As the research setting has good opportunities for test cases. Therefore, the following research questions have been devised to fill a small part of the identified research gap:

- Main Research Question:

How can short term Unreal Engine projects improve their development processes by creating and distributing compiled data by utilizing existing infrastructure in a more optimized way?

- Sub-Question 1:

What services need to be set up for teams to benefit from distributed caching and compilation?

- Sub-Question 2:

What build time reductions can be expected from implementing distributed caching and compilation in specific configuration contexts?

- Sub-Question 3:

How should distributed caching and compilation be configured for developers to obtain ideal workflows?

- Sub-Question 4:

What configuration trade-offs exist between cluster hardware variance, compile performance, and developer workflows?

- Sub-Question 5:

What practical trade-offs exist when increasing hardware variance of the cluster?

For these questions, short term can apply to different contexts, but this paper will mostly focus on 8-16 week projects to fit with academic blocks and semesters. Optimization will entail an acceleration in time it takes to compile compared to the default setup without distributed compilation; and a cache hit rate to make retrieving a cached version faster than (distributed) compilation. Further outputs will take the form of documented setup requirements and steps for teams to incorporate these infrastructures into their own pipelines. Ideal workflows in this context will aim to have a minimal amount of intrusion on the regular pipeline, or adjust the pipeline to be an improvement over the regular.

In these questions, hardware variance refers to the changing state of the cluster (worker agents disconnecting and reconnecting with the cluster). Either due to network connectivity or intentional user actions interfering with proper functionality of the cluster. This is why different clustering scenarios of dedicated or variable hardware will be evaluated, along with the trade-offs that these

461 scenarios might introduce. The methodology for investigating these questions and scenarios is what
462 will be detailed in the upcoming Chapter 3.
463

Chapter 3: Methodology

Following the research question – How can short term Unreal Engine projects improve their development processes by creating and distributing compiled data by utilizing existing infrastructure in a more optimized way? – mentioned in the last chapter, there are several sub-questions that contribute to answering the main research question. This has been done through evaluating different configuration contexts, each with a greater unpredictability of hardware and user interactions. These different contexts have been described as:

- Dedicated compile acceleration hardware
- Variable compile acceleration hardware within a small studio
- Variable compile acceleration hardware within an educational setting

The split between a small studio and an educational setting has been made due to the scale of multiple projects and multiple teams working on a single server in the educational context. Versus the scale of a few projects with a single or few teams working on a single server in the studio context. Besides the two variable hardware setups forming the main source of data for sub-questions 1, 3, 4, and 5. The dedicated hardware setup has been used for providing a hypothesis for Sub-Question 1, and evaluating Sub-Question 2, since they provided a much more stable environment for direct comparison, without the uncertainties that the variable scenarios introduced.

For the main source of data collection, several types of research were evaluated. Action research [32] was one that was recommended by peers, especially since it is close to the regular development cycle of a game project. Within action research there are Participatory Action Research (PAR) [28]– sometimes also known as collaborative action research – and Technical Action Research (TAR) [5, 48]. Due to the technical nature of this research, and the more objective goals of accelerating the compilation stage of development, TAR appeared to be the most appropriate of the action research methods. Especially as Horde is a relatively new system that participants are less likely to be informed about, making the collaborative aspect of PAR difficult to justify. Though the iterative nature of action research is ideal for this research, by having developer feedback inform iterative changes to the configuration of the system.

Besides purely evaluating action research, tool development & evaluation research was also investigated. With methods such as the System Usability Scale (SUS) survey for comparing tools [6, 29]. No tool is being developed in this thesis, but for implementation, evaluation using the SUS survey can still yield valuable results for comparing between TAR iterations, providing a quantifiable frame of reference. Besides the SUS survey, some more specific questions have been included to indicate problems or friction points.

To get started with the TAR cycle, preliminary tests have shown what services need to be ready to benefit from the acceleration system. This has been taken as the hypothesis for Sub-Question 1, and implemented into the participating teams as the initial state of the acceleration system. Through iterations, it was evaluated whether that hypothesis was complete or not. As preliminary testing was not done within the context of active development.

Sub-Question 2 is somewhat disconnected from the TAR cycle, and aims to answer the quantitative relationship between compute power and task acceleration. At the start of the data collection, the UBA and Horde have been deployed into the participating teams for the TAR cycle to start. After which the idealized test cases for Sub-Question 2 have run in parallel with the team's TAR cycle. This is because measuring development workflow improvements cannot be done as singular interventions but needs to be measured over an extended period of time. The data collection of these idealized test cases has been performed individually by the researcher on dedicated hardware, and therefore had no impact on the TAR cycle, nor did the TAR cycle impact the quantitative testing.

3.1 Prerequisite services and impacted development aspects

To be able to evaluate the UBA system's functionality, there were a few prerequisites that needed to be in place for the system to work. During preliminary testing the system had already been set up in a testing environment. From this environment a guide has been created for setting up both server and client for the base functionality, omitting any optional features that are supported by the software but not used by this research.

These guides have 2 variants for the server deployment, and 2 for the agent installation. This is because both can be deployed directly as a Windows application, or as a Docker container. These guides were written on the researcher's personal blog site for ease of access (blog.hidde-derks.com). The guides have also been attached as appendices A1-4. During the setup, the researcher has been present to assist with the setup of the services. These guides were tested thrice to explore issues an inexperienced user might run into. This test highlighted an oversight regarding file permissions and hidden files, since configuration for the Horde Agent is partially stored in ProgramData, a hidden Windows folder with a rather self-explanatory name. The test also showed that some users were prone to copy-pasting the configuration as shown in the guide, failing to modify it for their personal setup (e.g. using their own name instead of the one in the template).

Distributed caching had a similar approach, and was thought to be easier to set up than distributed compilation. When sharing projects across SCM providers such as Perforce, ignore files are often configured to ignore compiled binaries. So correctly tweaking these ignore files should be sufficient.

535 Though it is likely that there might be some tweaks to the way these files are stored. Since these files
536 are generated or compiled from source files, it is not as useful to store each iteration of these files.
537 Instead it is much more valuable to use that data storage to keep revisions of the source code, but only
538 keep the latest versions of the compiled binaries. For this, a breakdown of what files need to be
539 removed from ignore files has been made. Along with configuration options to reduce the amount of
540 revisions kept through p4 typemap definitions.

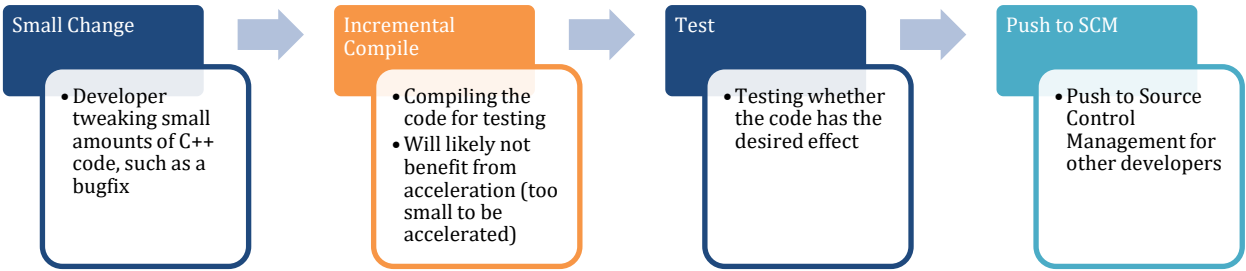
```
Binaries/  
*.dll  
*.pdb  
*.lib  
*.exe  
*.target  
*.modules  
*.exp
```

541 *Figure 5. Elements that should be removed from ignore files*

```
binary+wS2 //....dll  
binary+wS2 //....exe  
binary+wS1 //....pdb
```

542 *Figure 6. Example Performe typemap changes for keeping .dll, .exe, .pdb files writable in client workspaces, and keeping no more*
543 *than 2 revisions of .dll and .exe (in case of needing a rollback), and no more than 1 revision for .pdb files (largest files; purely for*
544 *debugging, not needed for functionality of rollback)*

545 To show the exact aspects of the development process, and how heavily they might be impacted, the
546 diagrams from the introduction have been expanded upon:



547 *Figure 7. Developer making an iterative change to code (daily if not hourly process)*

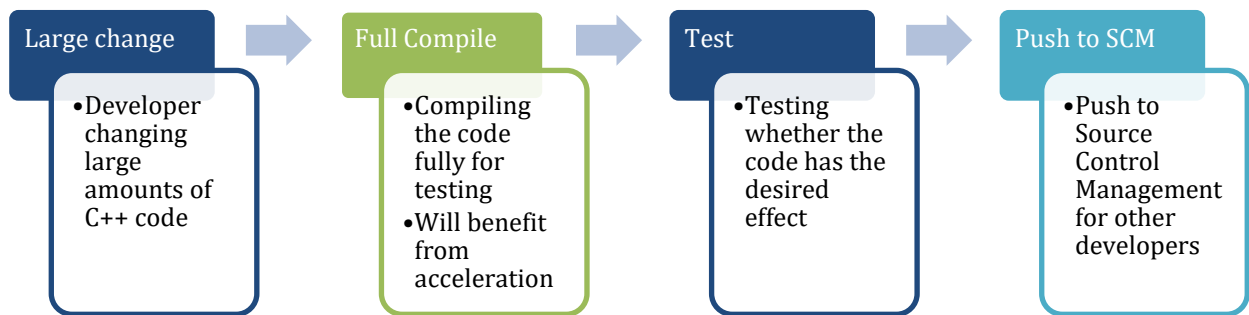


Figure 8. Developer making a large change to code (daily to weekly process)

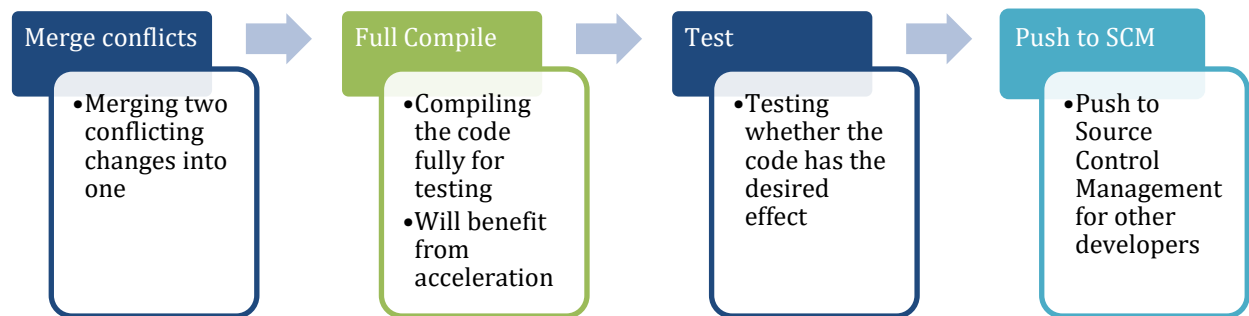


Figure 9. Merging conflicting changes (weekly process)

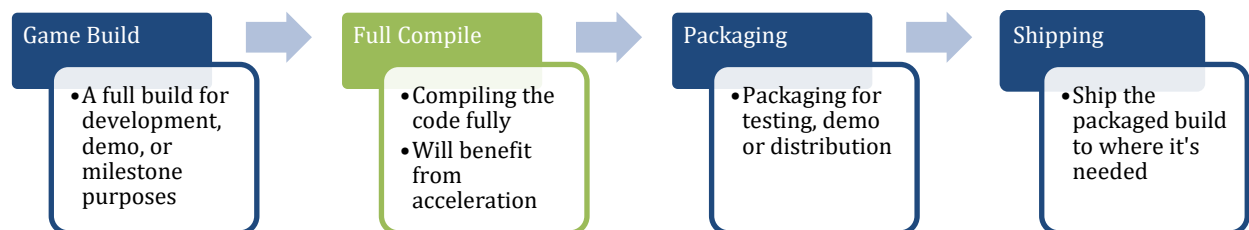


Figure 10. Game build (weekly to monthly process)

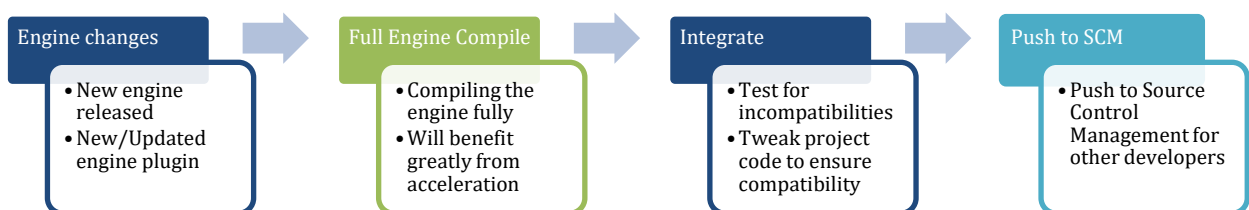


Figure 11. Disruptions such as new engine releases, plugin additions, or other changes (occasionally, but very high impact)

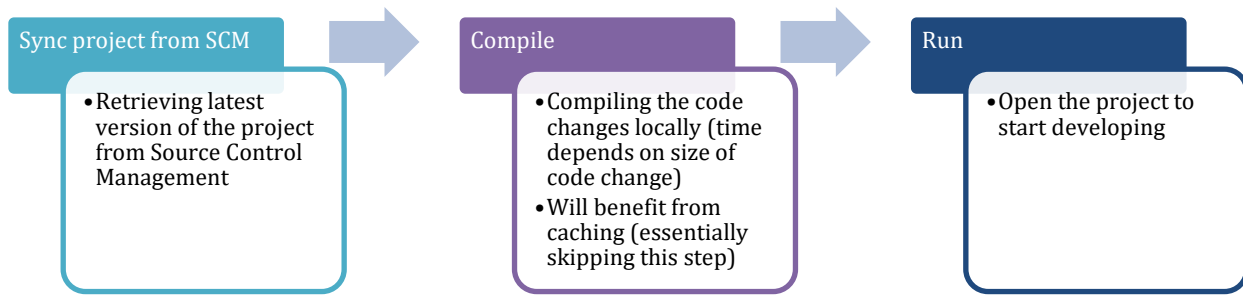


Figure 12. Developer (re)starting their project to have the most up-to-date version (daily if not hourly process)

In these diagrams, the sections highlighted are the ones this research has focused on or otherwise impacted. This legend has been slightly altered from the one used in the introduction:

- Green: impacted by distributed compilation and can have meaningful benefits
- Orange: impacted by distributed compilation, but might not have great benefit from it
- Purple: impacted by distributed caching, which should allow these sections to be skipped (bar some potential edge cases)
- Cyan: impacted by distributed caching, by reducing the ignored files when pushing to SCM
- Dark blue: no impact

Whilst this research does appear to have quite an impact on different aspects of the workflow, each of these systems should largely work in the background, automatically kicking in when needed. Leading to little action required by the developers themselves. Furthermore, these also represent a subset of existing workflows, and were (with exception of Figure 12) mostly applicable to programming workflows. Figure 12 is applicable to all disciplines, and is therefore also the one that aimed to skip part of the workflow, reducing difficulties that might block developers from working on the project.

In a normal environment, this would be enough for Horde agents to connect to each other, but in cases of networks with very restrictive firewalls, a VPN overlay network can provide a solution. This allows any Horde specific connections to go across the VPN's network, creating an environment where agents believe they are on the same network without firewalls blocking their connections. Not all cases needed this, but since especially BUAs' campus WiFi restricted devices from directly talking to one another, NetBird was used to circumvent this restriction. As an added bonus, it meant that students were not limited to being on-premises, as the VPN could also connect when working from home. For security purposes, it was also scoped such that only the horde agent ports are opened through the following access policy:

NAME	ACTIVE	SOURCES	DIRECTION	DESTINATIONS	PROTOCOL	PORTS	POSTURE CHECKS
Horde Interconnect	☒	Horde Cluster		Horde Cluster	TCP	7000-7010	1 Posture Check(s)

Figure 13. NetBird Horde access policy

This access policy was set to only allow group members of the Horde Cluster to connect to other members of the same group across TCP ports 7000-7010. It also had a Posture Check, making sure that the OS and NetBird client are not too outdated for security reasons. For transparency, each participant was also given an Auditor role in NetBird, which allowed them to have full visibility of the exact permissions and restrictions of the VPN connection at any time.

For this research, NetBird was chosen since it had already been set up, and offered the following beneficial features:

- Ability to contact each client using a unique IP address.
Often referred to as Peer-to-Peer (P2P), this is why many commercial VPN providers would not work. This is because they focus on connecting to the internet through the VPN, not connecting to other users through the VPN.
- Encrypted traffic.
- NAT traversal capabilities.
For traversing complex Network Address Translation (NAT) setups, NetBird employs Session Traversal Utilities for NAT (STUN), Traversal Using Relays around NAT (TURN), and WebSocket/QUIC Relays. This is a fallback from the P2P tunnels, but is required to traverse complex NAT or firewall setups.
- Authorization / Authentication.
NetBird is connected to an Open ID Connect (OIDC) provider for authentication. Other methods can also be used, so long as the network's security is maintained.
- Transparency.
Each participant was given an auditor role in NetBird, this gave them read-only access to all VPN settings, providing transparency in knowing exactly what the VPN can (and especially cannot) access.
- Self-hosted.
This deployment of NetBird was run on a private server, maintaining full control over any user/participant data.

3.2 Testing methodology for answering Sub-Question 2 (Scaling Tests)

To get a baseline of improvement, performance benchmarking has been done. This was done to get a rough idea of the improvement that can be expected in practical deployments, and whether things such as network latency have a large impact on the functionality of the Unreal Build Accelerator. These performance benchmarks have been used to see how well the system scales across devices, as preliminary tests have shown a hyperbolic relationship between the amount of compute and the time it

takes to compile. In theory this makes perfect sense, as it follows Amdahl's law [2], showing that infinite parallelization has diminishing returns. However, whether this theory holds true for this software across larger scales was evaluated through the performance benchmarks.

To control for inconsistencies, a fully dedicated setup has been used for testing this. The setup consisted of dual AMD EPYC 9554 CPUs with 1.5TiB of RAM total, and two i9-10980HK laptops with 32GiB of RAM as the initiators. This allowed consistent increments of allocated resources. Each time performing a recompile/rebuild of Unreal Engine's source code UE5 target (version 5.6.1-release), as it consists of 7914 actions to profile the acceleration of large tasks. For medium tasks, the UnrealInsights program target was taken, consisting of 501 actions. And finally, for small tasks, the UnrealPackageTool (65 actions) has been used. These tests have been run 4 times each, after which the average (arithmetic mean) and maximum observed deviations from that average were taken. To gauge the length of the compile process in these controlled scenarios, time measurements have been taken from both the UBA's compile trace and the time reported by Microsoft Visual Studio. This way the compile session can be isolated from preprocessing. The increments of resources were done in the following configurations:

0 cluster agents	Baseline local compile
1 cluster agent with 4 cores and 8 GB of RAM	Small low-end cluster
1 cluster agent with 8 cores and 16 GB of RAM	Small mid-end cluster
1 cluster agent with 16 cores and 32 GB of RAM	Small high-end cluster
1 cluster agent with 32 cores and 64 GB of RAM	Small very high-end cluster
1 cluster agent with 64 cores and 128 GB of RAM	Small server-grade cluster
4 cluster agents with 4 cores and 8 GB of RAM	Medium low-end cluster
4 cluster agents with 8 cores and 1 GB of RAM	Medium mid-end cluster (extremely low RAM)
4 cluster agents with 8 cores and 2 GB of RAM	Medium mid-end cluster (very low RAM)
4 cluster agents with 8 cores and 4 GB of RAM	Medium mid-end cluster (low RAM)
4 cluster agents with 8 cores and 8 GB of RAM	Medium mid-end cluster (medium RAM)
4 cluster agents with 8 cores and 16 GB of RAM	Medium mid-end cluster (high RAM)
4 cluster agents with 8 cores and 24 GB of RAM	Medium mid-end cluster (very high RAM)
4 cluster agents with 16 cores and 32 GB of RAM	Medium high-end cluster
4 cluster agents with 24 cores and 48 GB of RAM	Medium very high-end cluster
8 cluster agents with 4 cores and 8 GB of RAM	Large low-end cluster (without VPN)
8 cluster agents with 4 cores and 8 GB of RAM	Large low-end cluster (with VPN)
8 cluster agents with 8 cores and 16 GB of RAM	Large mid-end cluster
8 cluster agents with 12 cores and 24 GB of RAM	Large high-end cluster (without VPN)
8 cluster agents with 12 cores and 24 GB of RAM	Large high-end cluster (with VPN)

Table 4. Variations of tests that were run to evaluate scalability

These variations were used to evaluate the effects of different tiers of hardware being clustered in different amounts. And on the Medium cluster sizes, tweaks to the mid-end RAM allocation have been made to see the effects of RAM availability when compiling. Since this is running in Linux through Wine (a compatibility layer allowing Windows software to run on Linux), the overhead was not as big as a Windows laptop; Therefore more extreme variations have been added to simulate high amount of RAM utilization by other tasks on the computer. These situations can happen when a user is already using their laptop rather heavily, leaving limited amount of RAM for other applications such as UBA.

The basis of RAM allocation is grounded in Unreal Engine's base configuration. By default it limits the amount of parallel compilation actions to the available ram divided by 1.5 GiB (discarding any decimal values). Meaning that if a system has 8 physical cores and 11 GiB of RAM available, it will be limited to 7 parallel compile actions ($11 / 1.5 = 7\frac{1}{3}$). Using the Horde Agent this parallel action limiting is not performed, but provides a good baseline for RAM requirements. Accounting for OS overhead is why the 2 GB per core baseline was decided, but due to the uncertainty of the effects, environments as low as 1 GB for 8 cores were included.

On top of the RAM test, a test has also been added for evaluating the effect of a VPN on the cluster, to account for any inconsistencies that that might cause in datasets. All preliminary tests were run across the NetBird VPN, but whether this VPN overhead has a meaningful effect on compilation performance is unknown. For realistic deployment recommendations, this test case has been added to evaluate how much a VPN can impact performance. The VPN test cases have been applied on the large cluster. In hopes that this would show the time impact the most, as each agent would have its own VPN overhead.

These test cases have been run in Kubernetes containers, the Kubernetes cluster has been configured to assign physical cores on the same CPU socket as much as possible to remove overhead of cross-socket communication. Within Kubernetes the CPU cores and RAM amounts have been configured as both the reservations and limits, meaning that was running as the 'Guaranteed' Quality of Service (QoS) class. This meant that it had those resources at its full disposal, not having to compete with other services for said compute. The Kubernetes manifests used for the tests have been based on the Docker Compose file below (Figure 14); and are included in the repository mentioned in Section 4.5.

```

services:
  horde-agent:
    image: horde-agent
    environment:
      HORDE_SERVER_URL: https://horde.example.com/
      HORDE_DIRECTORY: /home/horde-docker/Horde/Agent
    volumes:
      - ./agent.json:/home/horde-docker/Horde/Agent/Data/agent.json
      - ./appsettings.Production.json:/home/horde-
docker/Horde/Agent/appsettings.Production.json
      - ./appsettings.json:/home/horde-docker/Horde/Agent/appsettings.json
    tty: true
    stdin_open: true
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 4GB
        reservations:
          memory: 4GB
    memswap_limit: 4GB
    cpuset: 0-15

```

Figure 14. Docker compose agent specification for 8 (physical) cores (16 threads) and 4GB of RAM

The evaluation of this sub-question is detached from the TAR methodology, but can be used to inform the findings of the TAR cycle. And could help in making decisions on whether there might be a set amount of compute statically available to the cluster to stabilize the variability that comes from developer's computers, and offer a base level of acceleration.

3.2.1 Switch from Portainer to Kubernetes

Initially, Portainer was used to run the agent's containers, but it showed intermittent TLS timeouts in the dashboard, and consistent DNS failures in containers. Due to these issues, the move was made to Kubernetes, that system had reliably been used in other projects. Both Kubernetes and Portainer rely on similar containerization backends. Suggesting that these inconsistencies are likely due Portainer's configuration, rather than Portainer as a software itself. For replication of this study, Kubernetes, Docker compose, Portainer or other containerization softwares can be chosen based on practicality, so long as the same constraints can be applied to these environments (CPU pinning & RAM limits).

3.2.2 Wine

For these tests, running agents in Docker containers with Wine was chosen. Windows containers are available, but the hardware on which these tests were run were Linux machines, and did not support Windows or Windows containers without a full OS reinstall and reconfiguration. The Unreal Horde agent also has native support for Wine, which only furthers the reasoning for choosing this approach.

3.2.3 *Compute stability limitation*

Due to linearly scaling the workload on the same machine, there was a slight concern regarding stability of the hardware. When more cores are assigned to the workload, the CPU outputs more heat that has to be dissipated. For this reason, linearly scaling across multiple machines would be a better solution to account for temperature and potential thermal throttling that might appear. The main cases where this might impact the findings is the full UE5 target compile on the 8 cluster agents with 12 cores each; and on the 4 cluster agents with 24 cores each. Both these cases would saturate a large amount of the cores of the CPU, but all other configurations were running at only a fraction (50% or less) of the CPUs power. Due to this, along with the sufficient cooling capacity of the machine, the limitation has been acknowledged, but no further measures have been taken to mitigate it.

3.3 The TAR cycle

At the start of the research, a presentation has been held within the participating team to inform the (potential) participants what the research is about, a high-level overview of what UBA and Horde are, what they do, along with how the research would be structured. Leaving space for questions from students, so that participating in the study could be an informed decision.

After the presentation, an informed consent form has been handed out to participants, indicating that they agree to participation in the research. When participants signed this informed consent form, they were given login details to one of the pre-made participant accounts. For ease of integration, the Horde server and NetBird server have been connected to the same Open ID Connect (OIDC) provider. This means that they can use a single account to log into both services. This account also served as a participant ID for the researcher. Making the setup easier by not having to make personalized login credentials for each participant, and assisting in anonymizing the data for after the data collection has concluded.

After the participants signed the informed consent form and received their login details, the researcher guided the participants through the setup process of the individual components, since the agent setup with a VPN is a little more involved than the agent on its own (due to autodetection of the computer's IP done by the Horde Agent).

As is common within TAR, the researcher is active as both researcher and technical support of the tool. This dual role provides several advantages: direct knowledge of technical issues when they appear, ability of rapid iterations, along with a thorough understanding of the tool's capabilities and constraints. However – as mentioned in the literature review, the researcher needs to be aware of this to prevent a biased approach to the analysis.

At Bi-weekly intervals, a survey has been sent out to the participants, consisting of the SUS survey, augmented with some extra questions. These extra questions aim to understand more nuanced aspects of the workflow around the tool, also allowing these to be tied to the SPACE framework mentioned in Section 2.4. Similar to the original SUS survey, the extra questions keep alternating between strong agreement as a positive and negative result to combat acquiescence bias:

What is your given participant login name?	Username used to sign into Horde and NetBird
What is your discipline within the team?	Either Producer, Designer, Programmer, or Artist
What is your specialization within the team?	Open question
1. I think that I would like to use this system frequently	1 (Strongly disagree) – 5 (Strongly agree) Alternating between strong agreement and strong disagreement being positive outcomes to combat acquiescence bias. 1-10 = SUS survey 11-16 = extra questions
2. I found the system unnecessarily complex	
3. I thought the system was easy to use	
4. I think that I would need the support of a technical person to be able to use this system	
5. I found the various functions in this system were well integrated	
6. I thought there was too much inconsistency in this system	
7. I would imagine that most people would learn to use this system very quickly	
8. I found the system very cumbersome to use	
9. I felt very confident using the system	
10. I needed to learn a lot of things before I could get going with this system	
11. I think this system positively impacted my team's development workflow	
12. I noticed that the system affected my computer's responsiveness	
13. I think this system positively impacted my personal development workflow	
14. There were times when I found it necessary to disable the system (during work hours)	
15. I could easily tell whether the system was active (either ready to receive tasks or actively running a task)	

16. There were times where I expected the system to be running when it was not	
(If applicable) Upload the zip file of your UnrealBuildTool logs and traces here. Usually found in: PATH_TO_ENGINE_ROOT\Engine\Programs\UnrealBuildTool	.zip file containing the .uba traces and logs for insight into parallelization and potential errors
Additional notes e.g. - Problems encountered - Desired improvements - Questions - Other remarks	Ability for participants to add any additional information, such as issues they have encountered

Table 5. Survey questions, SUS augmented with system-specific questions.

This survey has been performed using Google Forms, with Google Drive integration for uploading the .zip file. Since the trace files are small (~50-500 kilobytes), and many in number (~10-100), it was requested that a .zip file be made for easier uploading and handling of the data. The eventual storage requirements for these files are estimated to not exceed 2 gigabytes. But to prevent storage limits a shared drive was prepared with 100 gigabytes of available space.

The final two questions in the survey have not been listed as mandatory. This is done because the trace files only exist on the machines of people that initiated compile tasks on the cluster. Take for example an artist, their laptop would be part of the cluster, assisting in compile tasks. But they likely did not start any C++ compiles themselves, therefore also not have any trace files. The additional notes are there for additional information, should the participant want to share extra information about an experience they have had with the system, akin to a bug-report that might warrant additional investigation by the researcher.

Based on the survey questions, the researcher occasionally asked participants for further elaboration on what parts of the system led to negative experiences. This was done to further inform the iterative changes for the next TAR cycle. In case of such questions for further elaboration, notes were taken by the researcher. The questions during these elaborative sessions are heavily dependent on the nature of the topic or issue being elaborated upon, making methodological preparations hard to define. For this reason, these interviews have been conducted as unstructured interviews. During these unstructured interviews, the researcher aimed to understand what happened to cause the results shown in the data. Semi-structured is not applicable here as things such as participant information are already known, especially as these interviews were initiated with a specific participant at the researcher's request.

After these evaluations, The results were analyzed to determine problems or shortcomings of the system. Based on those findings, a solution has been explored and designed by tweaking configuration settings, or gauging whether additional tools might need to be introduced. This solution was then implemented within the team and reviewed during the next evaluation cycle.

At any point during the research, participants could also approach the researcher with further questions or issues. To clarify their understanding of the system, or to indicate a problem they were facing.

The TAR cycles ended after 8 weeks due to time constraints. After cycles were completed, a summarizing analysis has been made on how effective the tool was. Along with documenting any questions that remain unanswered due to these constraints. Even when the configuration of the system does not change between cycles, another cycle would have been performed to assess potential issues in different stages of the project. For example, how participants handle this tool when a deadline is approaching.

3.3.1 Preliminary survey test runs

To gauge the initial state of the survey, several peers were asked to run through the survey for its clarity. Due to the requirement of actually using the tool, they were asked to pretend as though they had used the tool for a while. These peers were successful in filling out the survey, and did not get stuck on any of the questions due to unclarities. The actual survey responses have been deleted as to not get mixed into the actual data collection phase of this project.

3.4 Participants and Contexts

For participants, the researcher had some talks with studios and teams from the education that might be interested in helping out with the research. Especially since they would be able to benefit from the acceleration as well. This resulted in 1 studio and 2 educational game teams from the Creative Media and Game Technologies (CMGT) course at Breda University of Applied Sciences (BUas) being interested in helping the research.

In the educational game teams, one consists of 25 members, and the other 26, with various distributions of specializations. Certain members within these teams indicated to be willing to help the research. After signing the informed consent form, a pre-generated participant ID was assigned, which (combined with a password) also served as the login credentials to the Unreal Horde and NetBird dashboards.

At the start of data-collection, these teams were in pre-production stage, and moved into production stage halfway through this research. All running on laptop hardware. Due to them working from campus regularly, a VPN was required to bypass campus firewall restrictions that would otherwise interfere with the functioning of the UBA communication.

The studio consists of 7 team members, also with varying specializations. During preliminary talks they did mention wanting to maintain control over their own server infrastructure. For this reason, they were provided the guides to set up the Unreal Horde system on-premises. And due to more control over the network, did not have to use the NetBird VPN to bypass restrictive firewalls. This made setup and management much simpler. With more focus on the Horde system and no risks the VPN might introduce, this however also meant developers could not use the system from home. After signing the informed consent a pre-generated participant ID was assigned, but without password, as there were no services for them to sign in to.

The studio had a longer project compared to the educational teams and was getting close to release stage, with a focus on polishing. They did also indicate a desire to explore different target platforms such as consoles (which likely required some larger compilation tasks). The studio was also running on laptop hardware.

Each of these 3 participating teams represented separate contexts, that were compared to understand broader UBA deployment implications. The three teams provided varied contexts (professional vs. educational), team structure, and project phases. By evaluating these contexts separately, the research could identify which findings generalize across contexts versus which are context-specific.

3.5 Data analysis strategy

After getting the results of the survey, the first 10 questions were used to generate the standardized SUS score using the formula:

$$SUS = 2.5(20 + \sum (SUS01, SUS03, SUS05, SUS07, SUS09) - \sum (SUS02, SUS04, SUS06, SUS08, SUS10))$$

Figure 15. System Usability Scale score calculation formula

On top of the base SUS score, there are some further questions that aim to gauge the nuances of the UBA/Horde system, these were seen as individual metrics and evaluated as such. This is because they are not inherently proven to be linked in the same way that the SUS survey is. And whilst they were very helpful for comparing between TAR iterations, they were also primarily there to indicate predicted problems that might need to be investigated for solutions. Such as the potential for the

system to take up system resources in a noticeable way, the potential for the system to silently fail. And to also indicate whether participants see this system as an improvement to their workflow or more of a nuisance.

The additional questions were evaluated according to the following groupings:

- Questions 11 and 13 focusing on workflow impact; with 4 or higher indicating a positive impact on the workflow, and 2 or lower indicating a negative impact on the workflow.
- Question 12 focuses on computer responsiveness impact; with responses of 2-3 indicating small impact on the responsiveness on the machine, whilst 4-5 was seen as problematic overhead impacting a developer's usual workflow
- Question 14 gauged whether participants elected to disable the system at any point. Either because it was actively interfering with their work, or because they wanted to prevent it from interfering (e.g. when running benchmarks). Responses of 1-2 were seen as a positive response. And responses of 3 or higher were asked for elaboration for what their underlying reason for disabling the system was (if they have not mentioned in the additional notes already).
- Question 15 and 16 focus on the transparency of the system, whether it is clear when the system is active or inactive. A response of 2 or lower on question 15 would mean the system needs to be more transparent to participants. And a response of 3 or higher on question 16 would indicate potential restart issues, where the system seemed to function, but silently failed to connect or perform acceleration.

In combination with the survey results, the UBA trace results contained within the .zip files were analyzed using the UBA Visualizer (Figure 3). From these trace files, the main metrics that were extracted from these trace files were:

- The number of agents that assisted with the compile
- How many of the compile actions were run by agents compared to how many were run by the initiator of the compile (effectively how much of the compile was distributed across the cluster agents)
- Which agents helped out to see whether the resource pooling works correctly
- Whether any compile actions ran into errors
- Whether agents encountered frequent disconnections
- How different hardware setups assist in compile tasks

This was done to cross-reference with the responses given by participants in the survey. To validate whether the results line up, or whether they might indicate a disparity between the perception and performance of the system.

Lastly, the open field was used for recording any additional information participants want to share. Answers to this question might warrant additional investigation by the researcher. This might indicate problems the participant experienced, recommendations to improve the system, or any other regards they might have for the researcher. However, due to the incidental nature of these additional remarks, this is not an aspect that was analyzed between TAR iterations. Instead, it was taken as a useful metric to inform changes for the next iteration.

To inform further iterations, issues were roughly grouped into several categories to indicate priority:

- Priority 0 – Immediate fix (before evaluation of current TAR cycle)
 - Issue preventing the current configuration from being tested properly. This could include, but is not limited to: Service backends (NetBird or Horde) being offline; an error in configuration causing the system to fail completely (e.g. syntax error); a participant running configuration that is incorrect or outdated from what is being tested in the current TAR cycle.
- Priority 1 – Critical issues
 - Average (mean) SUS score below 50, informed by Bangor et al. [4]
 - Average (mean) of an individual metric is below 2 (when 5 is a positive result) or above 4 (when 1 is a positive result).
 - Trace files showing more than 30% of compile actions failing or not completing due to disconnections
 - Open response indicating a critical functionality issue not accounted for in other ways
- Priority 2 – Significant issues
 - Average (mean) SUS score between 50 and 70, informed by Bangor et al. [4]
 - Average (mean) of an individual metric is below 2.5 (when 5 is a positive result) or above 3.5 (when 1 is a positive result).
 - Trace files showing more than 5% of compile actions failing or not completing due to disconnections
 - Open response indicating a significant issue not accounted for in other ways
- Priority 3 – Optimizations
 - Average (mean) SUS score above 70 but specific features scoring low
 - Open response indicating an improvement that can be implemented

Due to different teams being seen as separate contexts, considerations were made whether a solution for one team should be applied to the other team as well. In most cases, improvements applied to one team were applied to the other team as well. But there might be some cases, especially between educational teams and the studio team, where that is not applicable. For example, specific

configurations needed for two separate teams to share the same Horde server effectively, had little use in the studio team which is hosting their own dedicated Horde server instance.

After the TAR cycle had concluded, evaluations were made as to the efficacy of configuration changes in each context. For example, if a configuration change had equally beneficial effects in the educational contexts as it had in the studio context. For this, a more extensive comparison was done. With specific focus on:

- Common patterns:
Solutions that showed the same response between cases
- Context-dependent patterns:
Solutions that showed varying responses between cases
- Failed transfers:
Solutions that worked in one case, but when applied to another had negative results

3.6 Ethics

To address and minimize the impact of the ethical principles mentioned in Section 2.5, some specific measures have been taken to address the informed consent, data control, data minimization, transparency, and anonymity.

Within each participating team, an introduction to the research and the tool connected with the research was given. During this introduction, the extent of the software was explained; Detailing what exactly is communicated between laptops when being part of the cluster, and perhaps more importantly, what would not be communicated. After this introduction, participants were asked to sign an informed consent form indicating they agree to help with this research. Participants also maintained the right to step out of the research at any point, or to ask for further clarification of the technical functionality of the tool in case of questions or concerns.

As mentioned in Section 3.1, the VPN was also kept transparent by giving each participant an auditor role in the NetBird dashboard. This did not allow participants to change any routing rules, as that would be quite the security risk. Though they were able to see the entire state of the NetBird VPN and its routing rules. Since it had explicit rules in place, blocking any traffic unless specifically whitelisted through a configuration policy.

To maintain anonymity of participants in data collection, answers and trace files were anonymized where applicable by omitting personally identifiable information, or by only including the analysis results rather than the raw data file that would contain user or computer names. During the TAR

iterations participant names were mapped to participant IDs, since this offers no value after the research, those mappings were held in a separate file to be removed after the data collection phase has concluded.

3.7 Hypotheses

Following are the hypotheses for each sub-question that, through the course of this research, were evaluated for completeness and accuracy.

3.7.1 Sub-Question 1: What services need to be set up for teams to benefit from distributed caching and compilation?

Horde Server	To coordinate acceleration / parallelization
Horde Agent	To execute parallelized compiles
SCM server	To store / share project files and binaries
SCM client	To sync project files and binaries
Unrestricted LAN or VPN	To connect agents / clients to their respective servers

Table 6. Hypothesis of required components to benefit from distributed caching and compilation, along with their purpose in the process

3.7.2 Sub-Question 2: What build time reductions can be expected from implementing distributed caching and compilation in specific configuration contexts?

Compile speeds will heavily depend based on the compile task being run, but it is expected that the speed multiplier roughly scales with the amount of agents being added (with the assumption that agents are equal in compute power). At the same time, for the time that the compile takes, a hyperbolic relationship is expected. This is because if extremes are evaluated, no compute would take infinitely long to compile something. And infinite agents would still take a moment, as there is a limit in tasks to parallelize, where the limiting factor becomes the speed at which a single core of an agent can perform an action.

For the time column in these tables, an arbitrary value has been taken, as the hypothesis is about the relationship between cases, rather than the absolute values.

0 agents	1x	~1 minute
1 agent	~1x	~1 minute
4 agents	~1x	~1 minute
8 agents	~1x	~1 minute

Table 7. Expected compile speed multiplier for small, incremental compile tasks, along with the resulting time it would take

0 agents	1x	~8 minutes
1 agent	$\lesssim 2x$	~4 minutes

4 agents	$\lesssim 4x$	~ 2 minutes
8 agents	$\lesssim 8x$	~ 1 minute

Table 8. Expected compile speed multiplier for large, parallelizable compile tasks, along with the resulting time it would take

Besides the core and agent amounts being tweaked, for RAM allocation, performance degradation is expected below 1 GB per core. However, the extent of this degradation is hard to quantify as a hypothesis.

As for the VPN, the overhead is expected to be minimal. The main problems with using a VPN would be the bandwidth of the VPN's relay server being a limitation, along with disconnections being more frequent than non-VPN connections. This could impact the stability of the system, but it is expected that performance stays the same.

3.7.3 Sub-Question 3: How should distributed caching and compilation be configured for developers to obtain ideal workflows?

Educational teams indicated before starting that they preferred resource pooling to keep their compute power for benefitting their project. This also eliminates the risk of a team disproportionately utilizing compute resources from another team. Studio teams did not encounter this issue. They instead utilized resource pooling for testing of different hardware setups.

Question 15 will likely also be low (< 3.5) across all contexts, indicating bad visibility of when the system is active. This is because the real-time activity indication is not directly part of the UBA, instead running as a separate process that can exit with the actual compile agent service of UBA continuing to run. On top of this, historical telemetry is a system that is not feature-complete as of the writing of this thesis. The trace files can be used to view the time previous compiles took, but it is not the most accessible way of doing so. Therefore question 15 is not expected to exceed an average score of 3.5.

The low scores on question 15 may partially explain the higher responses on question 16. As there is unclarity whether the system is off or disconnected instead.

3.7.4 Sub-Question 4: What configuration trade-offs exist between cluster hardware variance, compile performance, and developer workflows?

When working with developer workstations that also form a large part of the compute power, you want to make sure that the compute of their machine being used for compilation in the background does not impact their workflow. Default Horde Agent priority settings will show acceptable performance impact (Question 12 being lower than 2.5) on high-end laptops (16 logical cores or more), but problematic impact (Question 12 being higher than 3.5) on low-end laptops (12 logical cores or less), requiring either core limitations (max 66% of CPU being used by Horde), or parallel task limitations (max 4 parallel tasks if the device has 6 physical cores)

There is also a trade-off by using a VPN. Campus testing of the educational teams necessitates use of a VPN due to restrictive firewall configurations, which have implications compared to direct LAN communication. Mainly showing itself in reliability risks, as the VPN service becomes a single point of failure for the entire cluster. Agent disconnections shown by trace files are often caused by VPN disconnections. And Question 16 shows responses higher than 3.5 for the educational teams. This is much greater than the responses given by the studio team (lower than 2), who do not require a VPN due to less restrictive firewalls interfering with the system's acceleration. The studio team have however set up a VPN to benefit from acceleration outside of the office, still using LAN when in the office.

3.7.5 Sub-Question 5: What practical trade-offs exist when increasing hardware variance of the cluster?

When the cluster consists of many individual developer machines, a VPN service such as NetBird will need to maintain connections between each individual machine, instead of having a single VPN connection that gives access to a set of dedicated compile agents. This will show itself by individual agents regularly disconnecting in a trace file, instead of all agents disconnecting at once (the initiator disconnecting from the VPN). This is because a Peer-to-Peer VPN as NetBird has multiple smaller VPN tunnels between clients, instead of one main VPN tunnel towards the server.

Another big trade-off is shown by the participants' discipline. Programmers have higher engagement and benefit from the system than visual artists and designers. Shown by the results of Question 13 being higher scored by programmers (above 4.0) than other disciplines (below 3.5). This is because programmers experience direct compilation acceleration, whilst other disciplines only experience a team velocity improvement, but no direct personal benefits.

Due to the previous point, it is also expected that non-programmers will disable the system intentionally more frequently (Question 14 above 3.5), whilst programmers do not (Question 14 below 2.5). By extension, also meaning that programmers benefit and contribute most from the system.

3.8 Limitations

A large limitation in this study was the usage of the VPN. In an ideal scenario, you would want to test this system without needing a VPN to bypass firewall restrictions. Besides adding limitations in latency and bandwidth, it also forms a large dependency. Making it easy for the VPN to become the problem preventing UBA from working, instead of UBA itself being the problem. Whilst a VPN can be helpful for

1018 allowing people to use UBA when working from home, it would have been better if the VPN was an
1019 option rather than a requirement.

1020

1021 The pool of participants was also rather shallow. The problem is present in a larger area, but for this
1022 research, an amount of BUas students formed the participants. Whilst this provided meaningful
1023 implementation data, it made it more challenging to be widely applicable. And has a chance of missing
1024 common issues that the participants have not encountered. On top of this, students have received
1025 similar education, potentially biasing their view compared to students in a different school.

1026

1027 There was also a selection bias, in the sense that people choose to participate. This might indicate that
1028 participants are already more technically adept, or more interested in using these kinds of tools. In a
1029 similar vein as the shallow participant pool, this might evade issues that less technically adept
1030 participants might have experienced. Technical support was provided by the researcher throughout
1031 the research cycle where required. But this potential for bias is important to be aware of.

1032

1033 (Re)Configuration can be an important metric as well. If this system is to be deployed to many teams at
1034 once, the configuration of all the teams is not something that staff wants to do on a regular basis. The
1035 pool definitions are held within the globals.json configuration file; If this file needs to be tweaked every
1036 time the team compositions change, there is risk of syntax errors along with the time it takes (failed
1037 validation will refuse the new globals.json config). This can be automated through scripts in similar
1038 ways that SCM repositories get created. But the reconfiguration implementation was not covered by
1039 this research.

1040

1041 Unreal Horde is a system that is still being developed, certain features are still in an experimental stage
1042 and subject to change. Luckily the acceleration and configuration of it have already reached a
1043 production-ready state, but aspects of the configurations used for this paper might be tweaked in
1044 future. That being said, a lot of the findings are still applicable, but might need to be updated to the new
1045 configuration definitions.

1046

1047 Since UBA is specifically integrated with Unreal, anything that wants to benefit from it needs to be
1048 compiled using the Unreal Build Tool (UBT). This limits compilation tasks to originate from code in the
1049 context of Unreal Engine. This is not a problem for teams that are already working in Unreal Engine,
1050 but does mean that the acceleration is not applicable in other engines, or pure C++ projects.

1051

1052 The answer from Sub-Question 2 can also not be directly equated to the acceleration findings in the
1053 teams. It is a different hardware setup, and was run through Wine on Linux. Whilst the system scales

1054 similarly, the overhead of the OS is different. And as mentioned in Section 3.2, the cases utilizing almost
1055 all cores of the CPU might be subject to some degree of thermal throttling. This might mean that the
1056 acceleration curve bottoms out earlier than it would without thermal throttling, but as this is subject to
1057 hardware variance either way, its impact has not been considered further.

1058
1059 Already mentioned was the limitation that the researcher had a dual role, which is inherent to TAR, but
1060 remains a limitation.

1061

Chapter 4: Experiment Results

To start the TAR cycle, the UBA / Horde system has been deployed to the three teams. Initially, the aim was to have one studio team and two educational teams, but due to personal reasons, one of the educational teams decided to drop out of the research; they were replaced by another studio team. The educational team (Team-1) used the researcher's personal (Linux) server hosted in a Docker environment. The first studio team (Team-2) deployed the Windows Server on their local network. And the second studio team (Team-3) used the Linux server hosted as a Docker container in a Kubernetes environment. For the first iteration, only the first studio team had completed deployment. And for the second iteration, the first studio team and educational team were ready. This means from the third iteration onwards all 3 teams were deployed and providing results. Improvements found during the first two iterations were immediately implemented such that all configs were synchronous with each other. The raw results for the surveys can be found in appendix C. Additionally, there are some qualitative findings that came from in-person follow-up conversations with participants.

4.0.1 Participant numbering

As a quick note regarding which participant is part of which team. User-01 until User-20 were provisioned for Team-1, User-41 until User-60 were provisioned for Team-2, and User-21 until User-40 were provisioned for Team-3. User amounts were overprovisioned on purpose to avoid running out of participant numbers part-way through, and to keep team ranges intact.

4.1 First Iteration

During the first iteration, a member of Team-2 reached out about the Horde Agent on their machine using resources rather aggressively (User-45 in the survey). This meant that his engine and other software was severely slowed down by the agent executing other tasks. As a solution, he installed Unreal Toolbox. Unreal Toolbox is a separate software running on the user's machine that measures CPU utilization and overrides the Horde Agent's settings through a named pipe (a method of communication between different software processes). It provides a UI to configure the settings that would otherwise be stored in an agent.json file. Along with monitoring CPU utilization to check whether an agent is idle or busy, in order to refuse compute requests if there is no compute to be shared from that agent.

Part of these configurations are already configurable in the agent.json file. However, Unreal Toolbox appears to be too aggressive in its marking agents as idle. When Unreal Toolbox is configured to mark an agent as idle when at least 1% of the CPU is free, an action as simple as moving the cursor will pause the agent when total CPU utilization in task manager does not exceed 15%. This clearly meets the set

requirements for helping compute tasks, but still marks the agent as busy. Instead, 30 seconds of no mouse movement is another (unconfigurable) requirement.

Since Unreal Toolbox seems to mark an agent as busy even when meeting the idle requirements, configuration through the agent.json is preferred. The main change will be marking the agent as a Workstation agent instead of a Dedicated agent. This means that any compute tasks will be started as 'Below Normal' priority. Since user's processes will be 'Normal', they will be prioritized by the CPU scheduler. On top of this, the 'cpuMultiplier' will be set to 0.9, meaning that at least 10% of the CPU threads (logical cores) will be kept free for other (non-agent) processes.

The survey results from this iteration are consistent with these findings, the SUS score averaged to 68. And scores on extra questions 12 (computer responsiveness) and 15 (system activity) averaged to 3.5 and 1.83 respectively, where a higher score on Q12 indicates greater performance impact, and a lower score on Q15 indicates poorer system visibility, which might influence the reliability of question 16 (system unexpectedly disabled), which yielded an average of 2.0, indicating that the system was rarely disabled when it should have been active. Another interesting note is that artists scored the SUS substantially lower, especially question 4, stating that they would likely need support of a technical person to use this system. Although Q12's average of 2.5 sits at the priority 2 boundary, the strength of the responsiveness complaints in the comments, and User-45's report in particular, led to it being treated as a Priority 1 issue, becoming the main driver of the next iteration's changes.

4.2 Second Iteration

During the second iteration, User-45 mentioned that their performance had not improved much, but lowering the cpuMultiplier down to 0.8 seemed to have solved their issues. The cpuMultiplier acts upon logical cores rather than physical ones. A multiplier of 0.9 sometimes only disables a single logical core. This means that the physical core is still tasked with compilation by one of its logical cores. In those cases, lowering the value to 0.8 will mean that a full physical core can be left available for user applications. Another noteworthy observation is that User-45's laptop had the highest amount of cores, but a similar amount of RAM as others. Meaning that the ratio of RAM per core was lower than other participants. Below is a small table of the machine configs pulled from the UbaVisualizer trace files.

Participant IDs	CPU logical threads	RAM	Ratio
User-01 User-05 User-41	20	32 GB	1.6 GB per core
User-02	16	32 GB	2 GB per core

User-03	32	32 GB	1 GB per core
User-07	16	16 GB	1 GB per core
User-22	22	32	1.45 GB per core
User-42 User-06 User-43 (2 nd machine) User-44	12	16 GB	1.33 GB per core
User-43 User-46	12	32 GB	2.66 GB per core
User-45	24	32 GB	1.33 GB per core

Table 9. CPU and RAM configurations of participant's machines. Only devices that appeared in compile session trace files have been included. Multiple participants appearing in the same row does not indicate an identical laptop, merely similar counts in logical threads and RAM capacity.

During the scaling tests intermittent errors were observed. These intermittent errors occurred in long-running tests of the UE5 target in cases where the majority of compile tasks were performed by the initiator. The errors “C3859: Failed to create virtual memory for PCH” and “C1076: Compiler limit: internal heap limit reached” always appeared in tandem, and point at memory pressure being an issue.

For both these reasons, the following iteration will move the cpuMultiplier from 0.9 to 0.6. This is done to reduce the memory pressure on agents and initiators.

This optimization has been left out of the scaling tests, as changing this value could impact the validity of data already collected. As such, scaling tests were still run using a cpuMultiplier of 0.9.

Another change is the reintroduction of Unreal Toolbox. After examining the code that caused the agent to be disabled for 30 seconds after any user input (e.g. mouse movement), it was found that the initial conclusion of this being an (unconfigurable) requirement for an agent to be marked as idle is incorrect. If the ‘Minimum idle time’ is set to 0 seconds, it will no longer disable the agent on user input such as mouse movement. Instead, the 30 seconds came from another part of the system, which is implemented to ensure that the agent does not rapidly switch between idle and busy. The change from idle to busy is made after a minimum time of 2 seconds, but the change from busy back to idle has a minimum time of 30 seconds. After which it will stay idle until another condition such as the minimum available CPU triggers the agent to switch to its busy state.

For this iteration, the Unreal Toolbox is set to the following settings:

- Mode: Workstation
(same as agent.json, this ensures tasks will be run at ‘Below Normal’ priority)

- Minimum idle time: 0 seconds
(setting this to 1 or more seconds will mean mouse movement disables the agent)
- Minimum free virtual memory: 256 MB
(default)
- Minimum free CPU: 1%
(CPU utilization is mainly controlled by CPU Multiplier)
- CPU Count: leave at autodetected value
(default)
- CPU Multiplier: 0.6
(matching agent.json)

As might be noticed by the settings, they are set at a very low minimum, this is to keep the agent open to tasks unless there is an extremely heavy workload taking all of the computer's resources. The primary purpose for installing Unreal Toolbox is to increase the visibility of the agent in an attempt to increase the score for question 15. Unreal Toolbox has taken the role of a software called TrayApp or HordeTrayApp. This software would show an icon in the taskbar with the state of the HordeAgent service. With Unreal Toolbox running, users will see an unreal engine logo in their system tray. The colour of this logo will indicate the state of the agent:

- Black: Operating Normally
- Blue: Running a lease (compute task or workspace conform)
- Red: Error (usually disconnection from Horde server)
- Black with red square: Agent is marked as busy by Unreal Toolbox and is prevented from taking on new leases

In this iteration, the survey results remain mostly the same, yielding an average SUS of 66, but question 15 has gone from 1.83 to 2.33. Like last time, artists still reported they felt the need of a technical person to be able to use this system.

Measuring distributed caching is also proving to be rather challenging. Caching kicks in transparently in the background, making it hard to notice or measure in the first place. On top of this, Team-1 and Team-2 both do not have an SCM server that has branching functionality enabled. This leads to regular mismatches and overwrites on the submitting side, forming a race condition [41:Ch. 2.3.1]. For this reason, they have decided to disable the caching functionality and revert to every developer compiling locally. This still results in accelerated compiles, but no longer allows compiles to be skipped entirely by all but one team member.

4.3 Third Iteration

During the third iteration, the main pain point that was raised by participants were false failures of the cook stage due to login failures. Horde also connects into the cook stage of packaging a project, to assist in shader compilation, however, this connection does not always succeed. Connection to the shared DDC provider was observed to be inconsistent across sessions. Normally this would not be an issue, as no connection would lead to no changes from usual. No speedup, but also no failures.

The problem is that when packaging happens, a connection to Horde fails with the message “LogUbaHorde: Error: Failed to create HttpClient for UbaAgent”. It would be expected that this kind of failure should result in a warning and fallback onto the default method of cooking and packaging the game. Instead, this results in an error, failing the cook; and by extension, the packaging process.

In one of the studio teams (Team-3), it was also noted that the login prompt kept popping up. Sometimes up to 5 times a day, which greatly breaks the flow of development. This studio team also had a much lower SUS score than other teams, sitting at about 47, compared to the average of 75 that other teams reported. Besides the login prompt, they also mentioned that their performance gains were negligible, with builds often being too small to warrant distributing to other agents.

Besides the issues with login and the cook stage, there were also some participants that noticed issues with compilation sometimes failing. These compilation failures were not necessarily due to issues in the code being compiled, but an issue in UBA distributing a task incorrectly. In these cases, building the project again would only build the tasks that failed from the previous run. The process itself would still be faster, but the unclarity of where the errors originated from, along with the need to issue the compile command a second time to compile successfully was not beneficial to their workflow.

Both of these issues – the cook stage auth failures & compile failures – are issues within the core software itself. And as such, cannot be solved by iterating on the configuration of the software; instead, resolving them would require iteration on the software itself.

During this iteration, User-43 was very positive about the build time of a custom engine dropping from 1.5 hours to 30 minutes. They also identified two further issues:

- Cross-compiling for a Linux target on a windows machine would fail with an access violation. This was fixed by disabling the local executor (bAllowUBALocalExecutor=false).
- A mix between Windows 10 and 11 agents could result in a module failing to be found in the UbaDetours.dll. Ensuring that all agents were running on Windows 11 resolved this issue.

1223 4.4 TAR Overview

1224 When looking at the TAR cycle as a whole, some other interesting findings appear. One especially
1225 important one is the usage of the system as reported by the trace files. By this metric, Team-2 used the
1226 system the most. They have submitted multiple trace files with multiple agents, which were also large
1227 enough to benefit from acceleration, accompanied with User-43 mentioning that a 1.5-hour engine
1228 compile had been reduced to 30 minutes (for which the trace file has sadly been lost). Team-1 and
1229 Team-3 have also submitted trace files, but only a single trace from Team-1 showed a large compile
1230 task that benefitted from distributed compilation.

1231
1232 Another notable thing is that over the course of the iterations, comparable disciplines have given very
1233 similar SUS scores. Designers started out high in the 80-90 range, dropped down to the 70-80 range,
1234 after which they returned to the 80-90 range. One outlier started at 52.5, but in their comment they
1235 also mentioned “I don’t know if I’m running it correctly”, potentially contributing to the lower score.
1236 Artists gave very low scores in the 25-60 range. And Programmers started out in the 70-80 range, but
1237 consistently dropped in score, with two of them ending in the 60-70 range, and one dropping to 47.5.
1238 These programmers are also the ones utilizing it most, and reporting most of the experienced
1239 problems. In many cases, these problems also prompted them to disable the system to continue
1240 working. Other disciplines did not disable the system as much, treating it as a regular background
1241 service on their machine.

1242
1243 Not all participants replied to the survey for every iteration, leading to some gaps in the data. Similarly,
1244 not all participants included their compile session trace files. It is possible that other teams have had
1245 other accelerated compiles that benefitted from UBA and Horde. But the existence of these compiles is
1246 unknown.

1247 1248 4.5 Core Scaling Tests

1249 Besides the configuration iterations that the TAR cycle has gone through, analysis of the scaling test
1250 data resulted in several findings. A full copy of the raw data has been attached in appendices D and E,
1251 along with the templates that were used as the Kubernetes manifests (scaled to 0 replicas due to
1252 Guaranteed Quality of Service class). Both the UBA session time and the Visual Studio compile time
1253 were noted down. To isolate preparation of the compile session, the following analyses will use the
1254 values gained from the UBA session time. With Baseline and measurements of 0 cores or 0 agents
1255 referring to a “no-acceleration” case where zero remote cores were assigned, leaving only the initiator
1256 to participate in compilation. The hardware used to run these tests is:

- 1257 • Initiators: laptop with an Intel i9-10980HK CPU (8 cores, 16 threads) and 32GB of RAM

- Agents: server with dual AMD EPYC 9554 CPUs (2x 64 cores, 2x 128 threads) and 1.5TiB of RAM (running multiple virtualized instances).

The visualizations shown in this chapter are a subset of all visualizations of the data. Not all data points can fit within a single graph, for this reason, graphs have been chosen that best represent the finding. A full copy of raw data, graphical visualizations, and trace files will be hosted in a git repository at: <https://github.com/Sigma-Erebus/MGT-Thesis-Repository> (mirrored from <https://gitea.sigma-erebus.com/public-webmaster/MGT-Thesis-Repository>)

Data of the core scaling tests shows that scaling the amount of cores in a compile session yields diminishing returns, which was especially visible on the large compiles of the UE5 target (Figure 17). Smaller tests reached diminishing returns at a faster rate (Figure 17), and did not reach as high of a total acceleration (Figure 18).

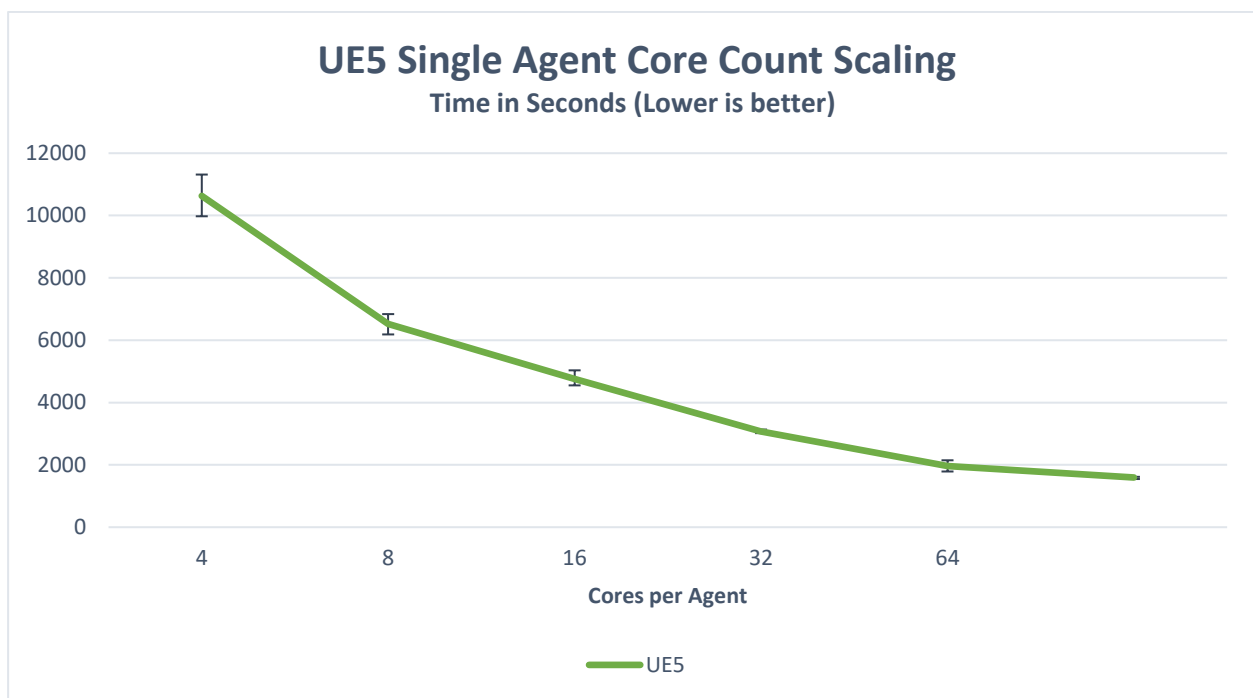


Figure 16. Compile time scaling of a single helper agent with varying amounts of CPU cores for the UE5 target.

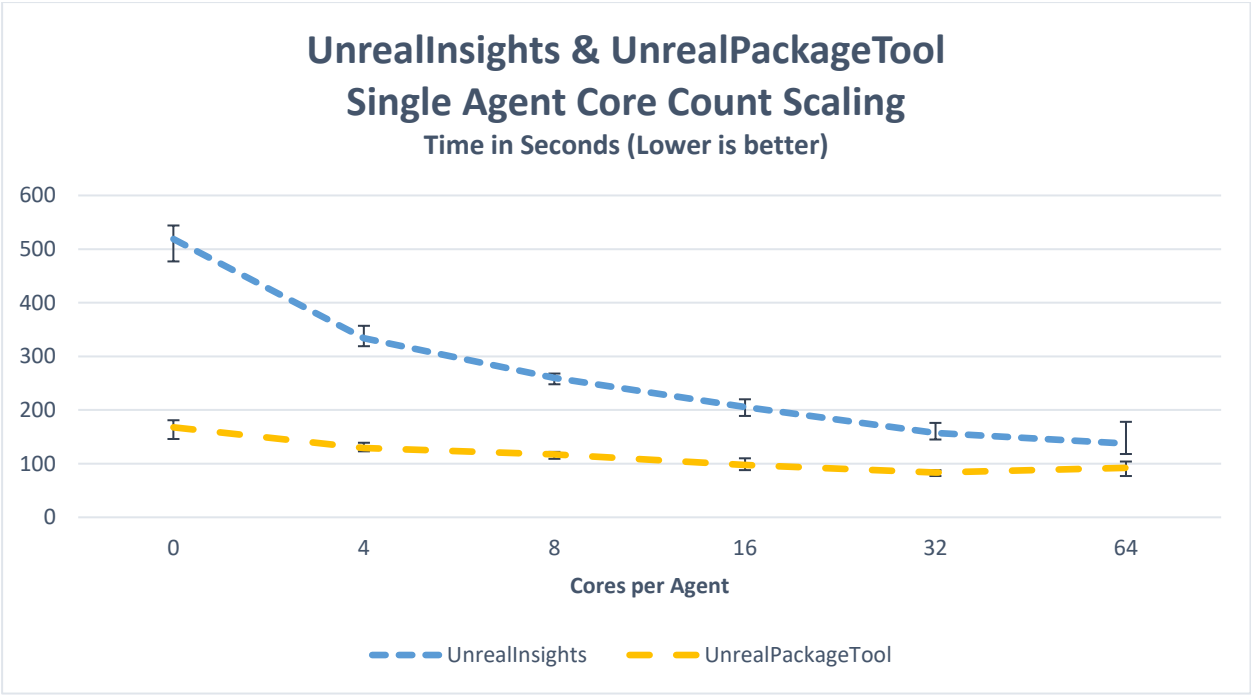


Figure 17. Compile time scaling of a single helper agent with varying amounts of CPU cores for the UnrealInsights & UnrealPackageTool targets.

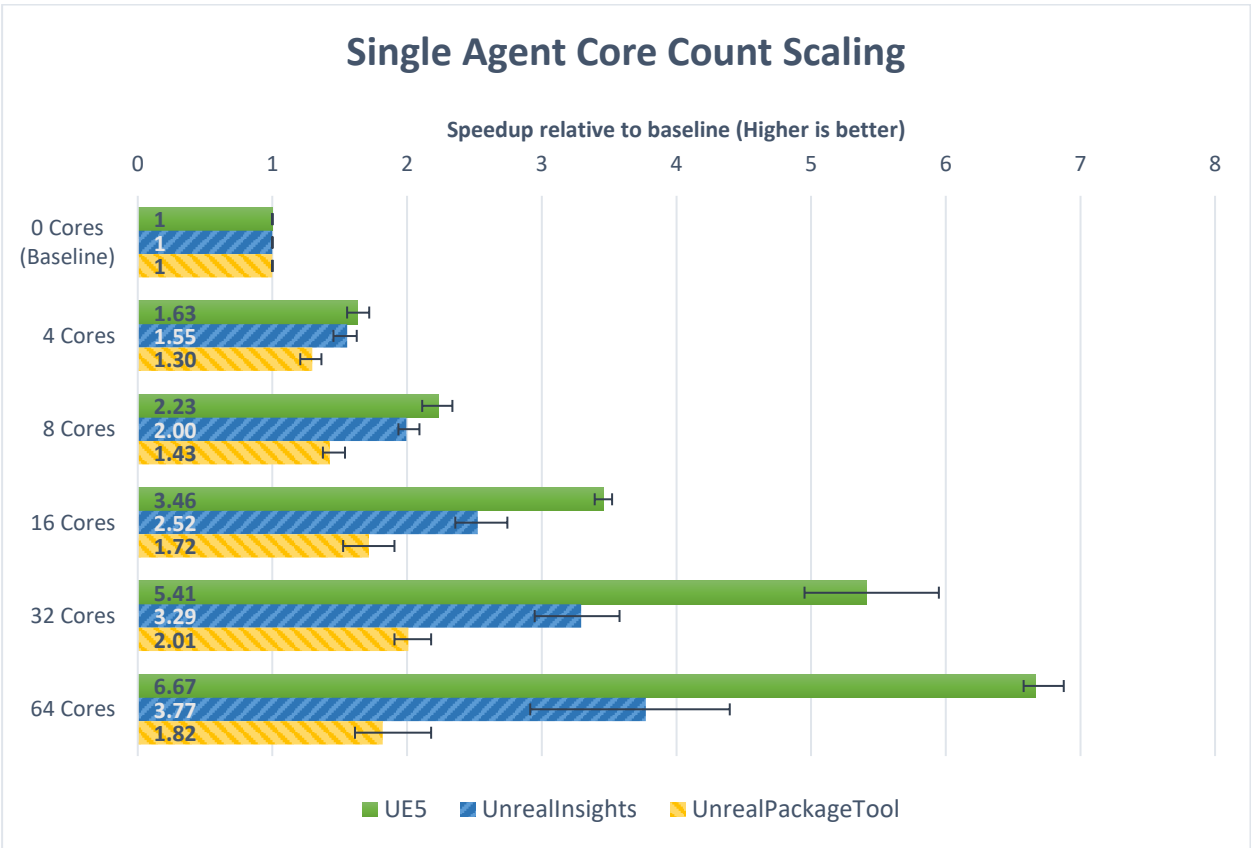


Figure 18. Speedup scaling of a single helper agent with varying amounts of CPU cores.

As can be seen in Figure 17, the diminishing returns appear to reach an asymptote beyond which further cores no longer contribute to accelerating the compile task. The graph that follows this scaling can be stated as the formula shown in Figure 19. Excel's GRG Nonlinear solver was set to minimize

error² between the averaged data points and the function by adjusting the values of A, B, and C. This solver was also constrained to not exceed any observed measurements, to not overestimate the potential for acceleration. Underestimation was allowed if it resulted in a cleaner fit of the data. The GRG algorithm (later superseded, see below) resulted in the values 858.1, 47334, and 4.81 for A, B, and C respectively, with a Root Mean Square Error (RMSE) of 167.7 seconds.

$$y = A + \frac{B}{x + C}$$

Figure 19. Multiplicative inverse function (reciprocal) adjusted by constants A, B, and C to match the data points gathered. x is the variable representing the amount of remote cores. The outcome y represents the predicted time of compilation.

During analysis of this scaling, one supervisor warned about the starting point for the values. The GRG Nonlinear algorithm finds local optimal points, but there is no guarantee that this will be a global optimum. And since the original starting values had not been noted, the algorithm returned different local optima. For this reason, a switch was made to the Evolutionary algorithm. This algorithm is based on natural selection, optimizing towards an optimum, whilst still exploring for solutions that are globally better than its current local optimum. This algorithm is substantially slower, but yields more reliable results by exploring a larger range of possible results. Due to this, Excel was dropped in favor of Python, as Python can run these evolutionary algorithms without the overhead of recalculating formulae in a worksheet. The evolutionary algorithms had one drawback, which is their tendency to violate the set constraints if it results in a meaningfully better fit. To counter this, constraint violations were multiplied by a factor of one million and added to the objective as a penalty term. Since the algorithm attempts to minimize the objective, this penalty assures that constraint violations are never seen as a better fit. The exact Python script recreation will be attached in appendix F. This resulted in the values shown in Figure 20. These values have an RMSE of 126.8 seconds, representing an average deviation of 2.7% from the mean of all measured values for this target. UnrealInsights and UnrealPackageTool had an RMSE of 2.4 seconds (0.8% deviation) and 4.6 seconds (4.0% deviation) respectively.

$$y = 795.6 + \frac{51980}{x + 5.27}$$

Figure 20. Multiplicative inverse function with constants for UE5 target.

This suggests that 795.6 (A) is the asymptote of the dataset, where with infinite remote cores, it would still take ~796 seconds for the compile of UE5 to complete, representing the amount of work that cannot be distributed to other agents. The constant 51980 (B) represents the amount of work that can be distributed to other agents. The third constant 5.27 (C) represents the amount of compute that the initiator is contributing to work that could be distributed. Due to the initiator having a different CPU, this value can be seen as the equivalent of ~5.27 AMD Epyc cores; as x represents the amount of remote AMD Epyc cores that are allocated to the compile session. Furthermore, by dividing the

1314 baseline by the found asymptote, we can also estimate the theoretical maximum speedup to be roughly
 1315 13.4x faster than baseline. The values found for UnrealInsights and UnrealPackageTool are shown in
 1316 Figure 21 and Figure 22, with a maximum speedup of 4.9x and 2.1x respectively.

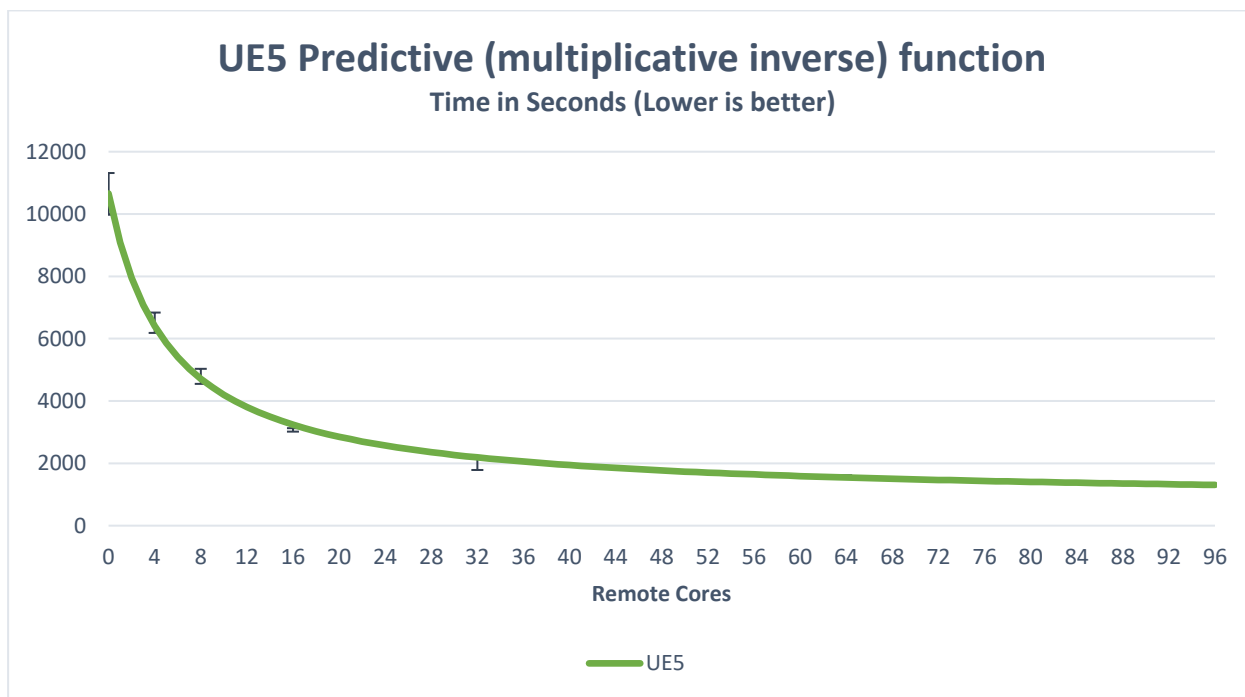
1317
$$y = 106.7 + \frac{1999}{x + 4.85}$$

1318 *Figure 21. Multiplicative inverse constants for UnrealInsights target.*

1319
$$y = 78.7 + \frac{462}{x + 5.17}$$

1320 *Figure 22. Multiplicative inverse constants for UnrealPackageTool target.*

1321 Figure 23 and Figure 24 show these multiplicative inverse functions plotted out, with predictions up to
 1322 96 cores. The error bars represent the actual measured data points (min and max).



1323 *Figure 23. Multiplicative inverse function plotted on graph, predicting scaling up to 96 cores for the UE5 target. Error bars*
 1324 *represent minimum and maximum observed values.*
 1325

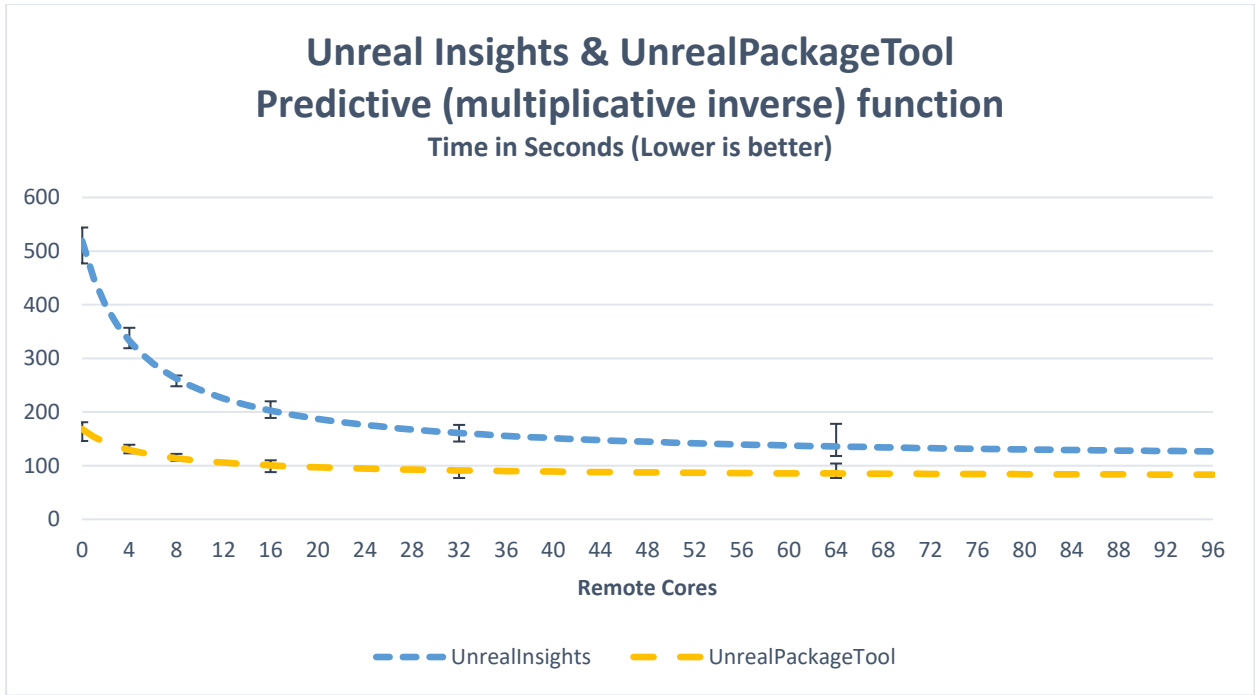


Figure 24. Multiplicative inverse functions plotted on graph, predicting scaling up to 96 cores for the UnrealInsights and UnrealPackageTool targets. Error bars represent minimum and maximum observed values.

If these values are applied back to Amdahl's law, we can calculate the percentage of parallel work by entering the same constants into the formula in Figure 25. This yields 92.5% for the UE5 target, 79.4% for the UnrealInsights target, and 53.2% for the UnrealPackageTool target.

$$p = \frac{B/C}{A + B/C}$$

Figure 25. Formula to get the percentage of parallel work (p).

4.5.1 Aitken Extrapolation (Delta Squared)

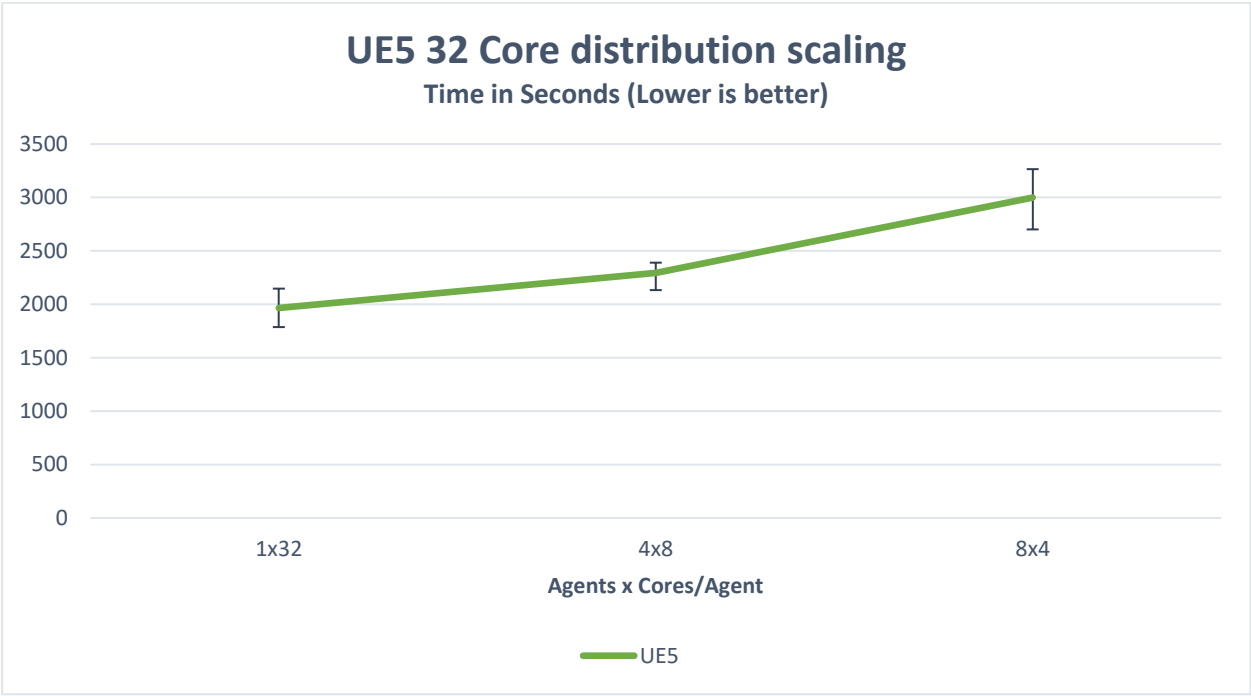
At first, Aitken extrapolation was considered to predict the asymptote of the data points. Aitken works on three points rather than minimizing error across a larger range. This means that slight deviations in the data points such as run-to-run variance can influence it too much, yielding wildly varying values between 1740 and -1760 seconds at its extremes. For this reason, nonlinear regression was selected over Aitken extrapolation, as it can benefit from the accuracy provided by more than 3 data points.

4.5.2 Lost Trace Files

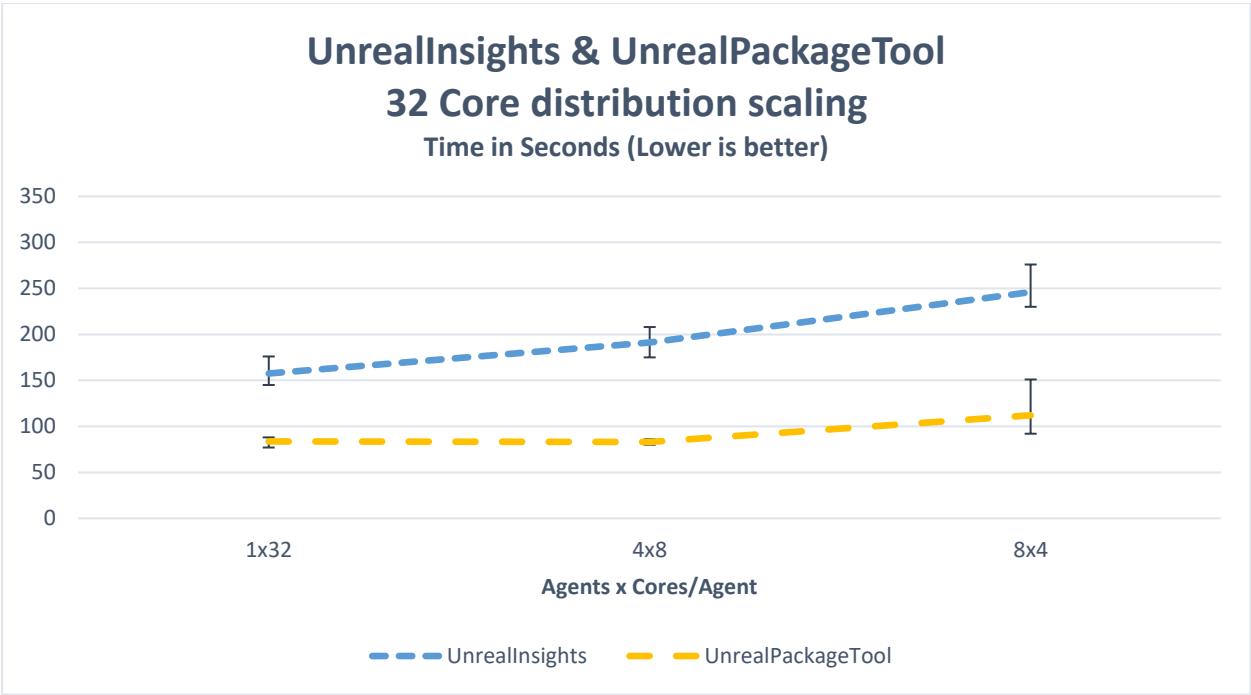
During data collection, a few trace files were lost due to an accidental overwrite of the file. This occurred 4 times, but on a scale of 216 runs, these lost trace files did not contain any meaningful findings that could not be found in other trace files, as every configuration has at least 1 trace file present. These missing trace files have been marked as "Lost" in the raw data overview of the data repository.

1347 4.6 Agent Scaling Tests

1348 Tests also showed that there is definite overhead of having multiple agents assisting a compile task,
1349 however the design of the test cannot determine how much of this overhead came from the VPN's
1350 connections or from UBA's scheduling. Figure 26 and Figure 27 show the differences between different
1351 distributions of 32 cores. This overhead of multiple agents does not overpower the benefit of adding
1352 additional agents, as shown in Figure 28.



1353
1354 Figure 26. Compile time scaling of 32 cores distributed across a varying amount of agents for the UE5 target.



1355
1356 Figure 27. Compile time scaling of 32 cores distributed across a varying amount of agents for the UnrealInsights and
1357 UnrealPackageTool targets..

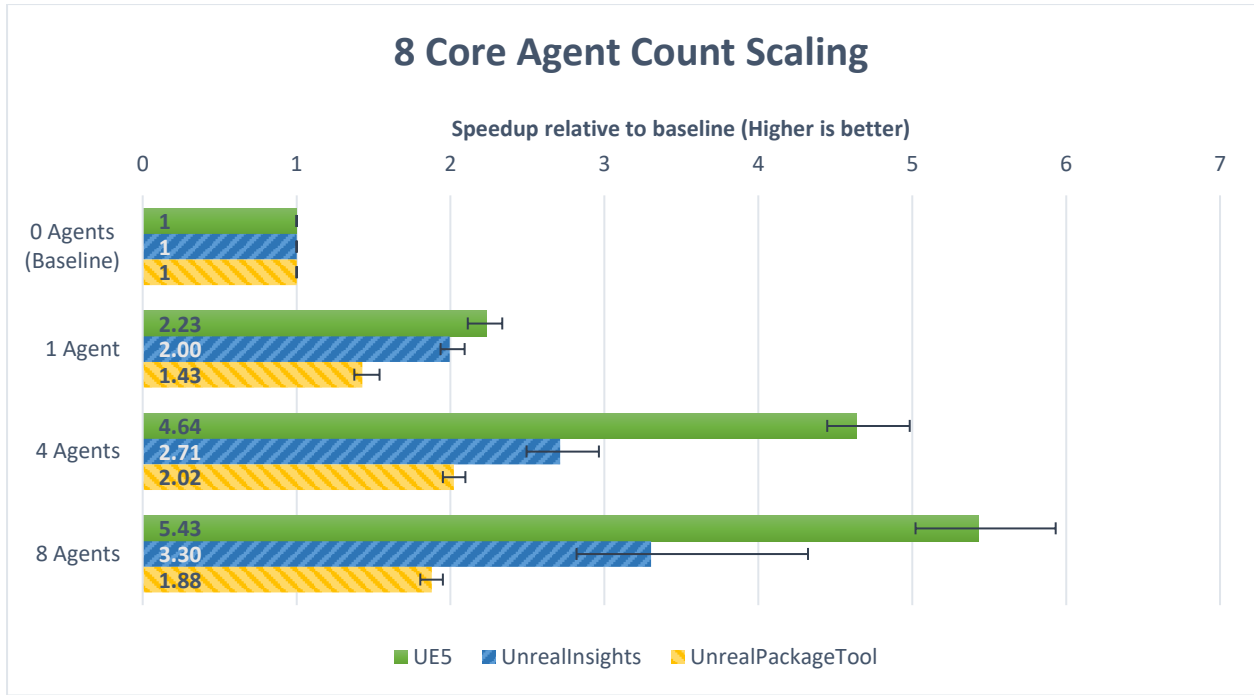


Figure 28. Speedup scaling of varying amount of agents with 8 cores assigned to them.

If the Python evolutionary algorithm is set to find the curve for the four agents instead of the single agents, it results in the values in Figure 29. Doing the same with the eight agent setup results in the values shown in Figure 30. Whilst they follow the data points, they suggest that more agents (and more scheduling overhead) results in a lower asymptote. This is physically implausible as the serial floor (A) is a property of the compile task rather than agent count. Combined with the constants B and C increasing with agent count, it is likely that the model is struggling to account for scheduling overhead, and that the other values are absorbing this inaccuracy. It could also be an indication that there are too few data points when compared to the single-agent tests (4 compared to 6), or the data points had too much variance to allow clear predictions for further scaling.

$$y = 1029.3 + \frac{48834}{4x + 5.08}$$

Figure 29. Multiplicative inverse constants for UE5 target with 4 agents. Since x is measured as cores/agent, it is multiplied by 4 to get the total cores assisting in the compile.

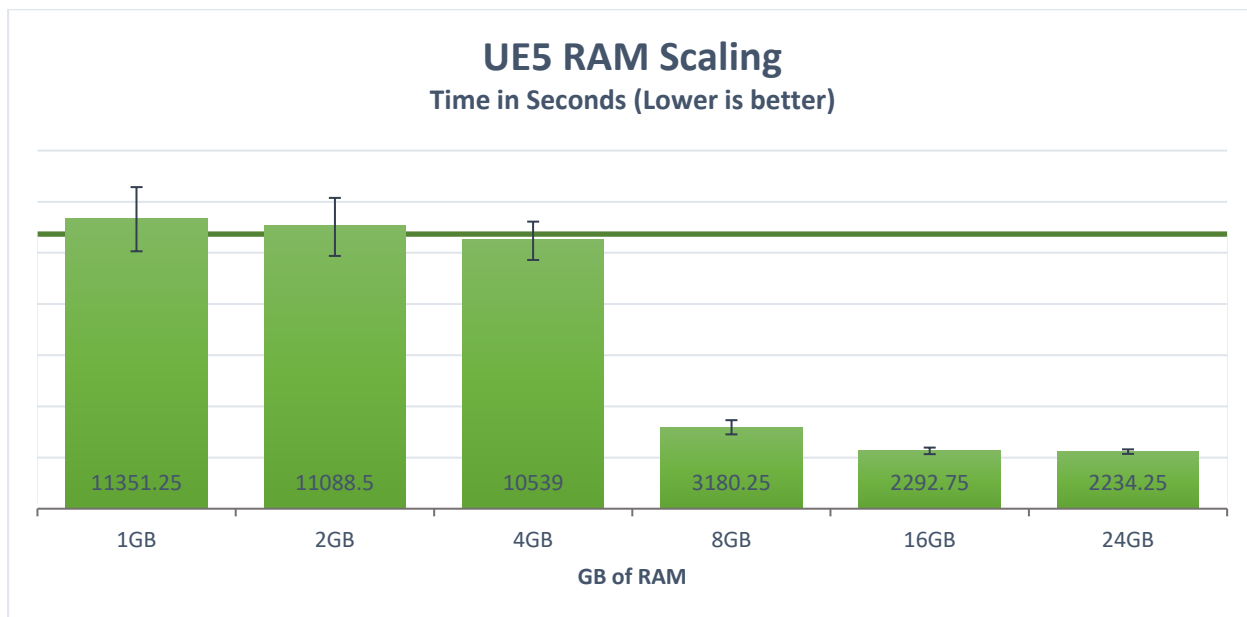
$$y = 843.6 + \frac{87123}{8x + 8.88}$$

Figure 30. Multiplicative inverse constants for UE5 target with 8 agents.

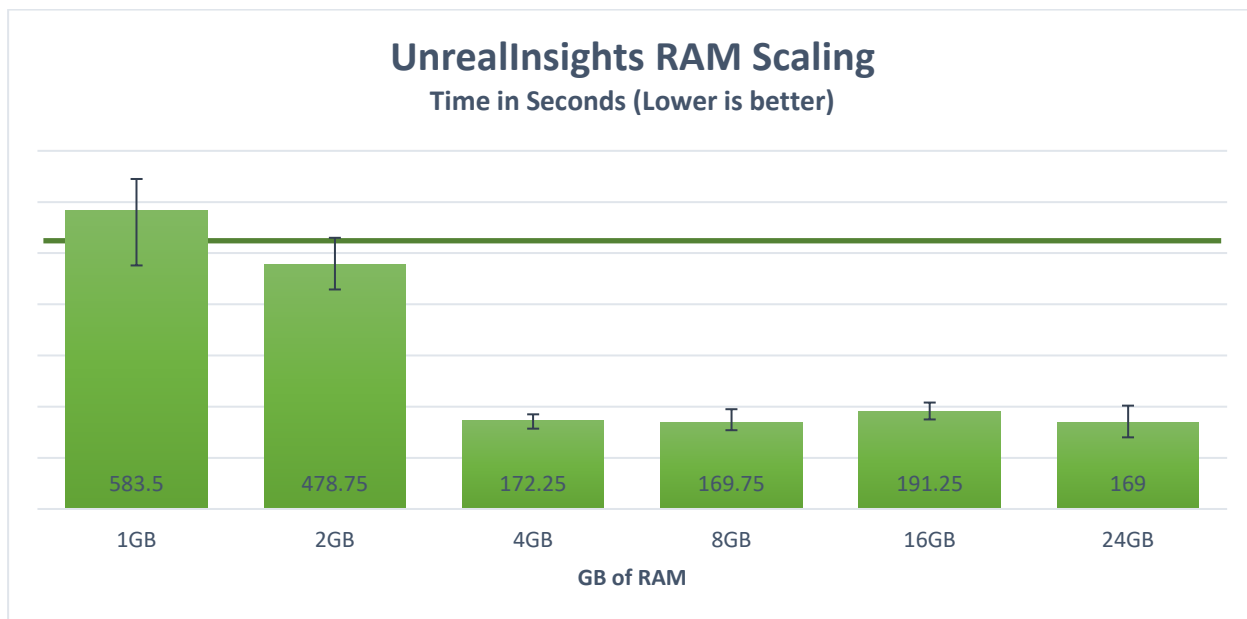
4.7 RAM Scaling Tests

RAM scaling also showed that lowering the RAM allocated to agents has a clear cutoff point where performance diminishes all the way to baseline, occasionally surpassing it. Interestingly, the point at which this reduction occurs differed based on the size of the compile task. UE5 crashed down to

1379 baseline around 4 GB of RAM (Figure 31), where UnrealInsights and UnrealPackageTool still benefitted
1380 from the acceleration, but crashed down to baseline around 2 GB (Figure 32 & Figure 33).



1381
1382 *Figure 31. UE5 Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.*



1383
1384 *Figure 32. UnrealInsights Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.*

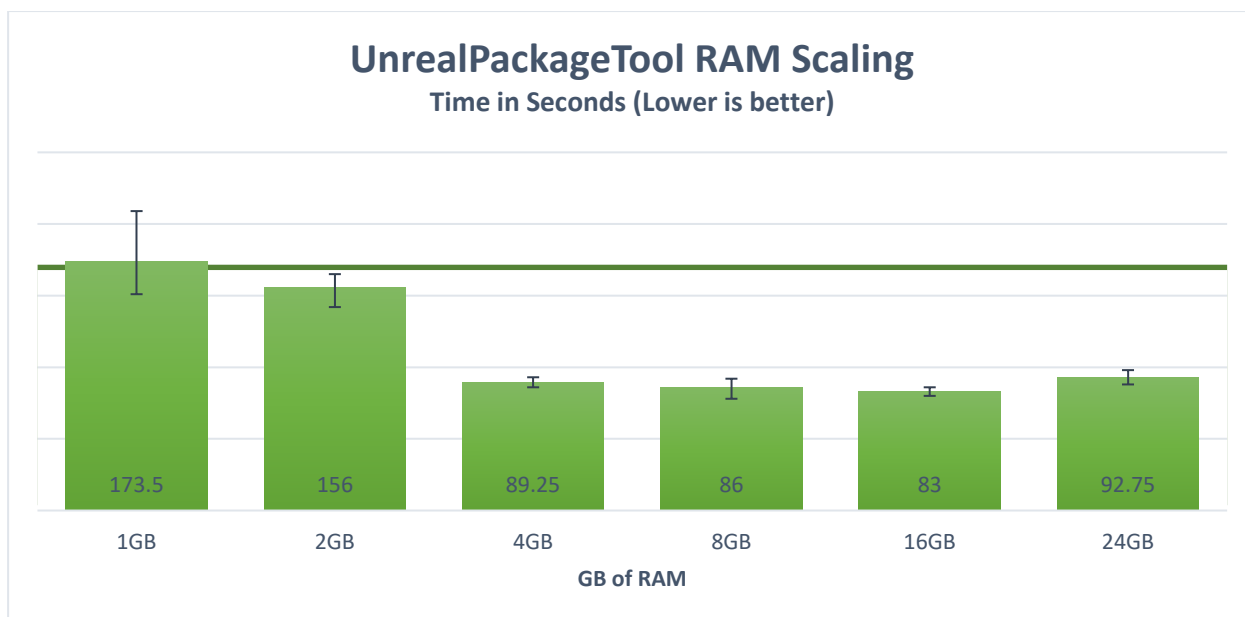


Figure 33. UnrealPackageTool Scaling with varying amounts of RAM. Green horizontal line represents no acceleration baseline.

The way that this lack of RAM manifests itself is by agents picking up tasks – maybe finishing a few – but failing to complete most before disconnecting. After a period of time the agent clears its memory pressure and reconnects to pick up tasks again. The worst consequences of this happen when an agent picks up a compile task towards the end of a session, as the initiator will not retry running this frozen task until the agent disconnects. A great example of this happened during one of the UnrealInsights 1GB RAM tests (shown in Figure 34).

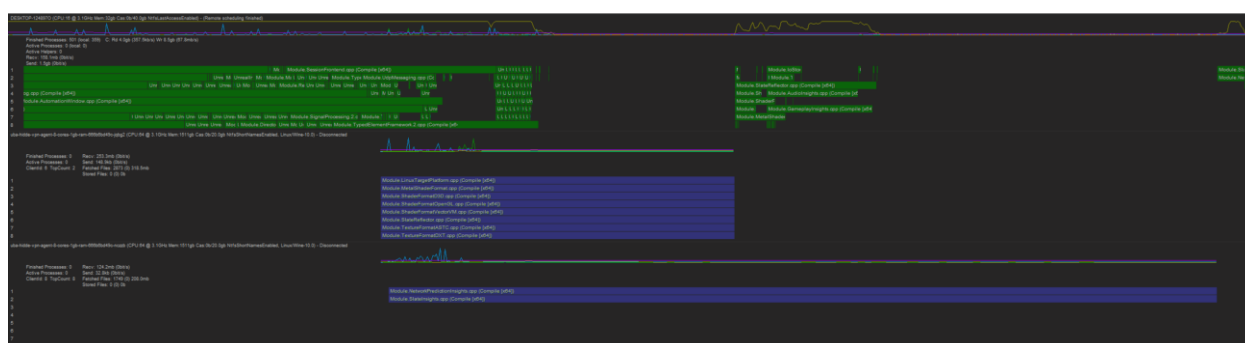


Figure 34. Cropped UbaVisualizer trace showing inactivity on the initiator (empty space between top green row) due to agents (blue rows on bottom) picking up tasks and failing to finish them in a timely manner. Only after the agents disconnect (end of blue bar) can the initiator pick up those final few tasks to complete the session. Full trace file can be found as: Group-1/RAM-Test-1GB-UnrealInsights-Target-1.uba.

4.8 VPN Scaling Tests

In the methodology, there was also the plan to evaluate VPN test cases against non-VPN test cases. Due to an inability to set up a non-VPN test case, this comparison could not be performed. Each containerized agent would have needed its own IP address that initiators could route to. This would mean that routing rules had to be made to have the initiators route to IP addresses on the internal Calico overlay network of Kubernetes. Not only would this require another VPN to create this routing

1405 tunnel, it would also provide great risk to the functionality and security of the Kubernetes cluster. For
1406 these reasons, the VPN comparison was dropped, and all agents exclusively ran over the VPN. The
1407 known limitations of this VPN are both latency and bandwidth constraints. Minimum client-to-client
1408 latency observed through the VPN relay with ICMP ping requests is 18ms, but this latency occasionally
1409 spiked during compile sessions, reaching latencies upwards of 10000ms. Bandwidth capacity to and
1410 from the VPN server was measured at ~200Mbps. Trace logs only show maximum transfer peaks of
1411 ~120Mbps. Bandwidth to and from the server without VPN overhead was measured at ~260Mbps,
1412 with a latency of 12ms using a speedtest.
1413

Chapter 5: Discussion of Data and Results

Using the data from Chapter 4, the earlier hypotheses to the research (sub)questions have been evaluated. After evaluating hypotheses to the research sub-questions, additional findings that did not match any of the hypotheses have been listed separately.

5.1 Sub-Question 1: What services need to be set up for teams to benefit from distributed caching and compilation?

The first sub-question tackled the question of what services need to be set up for teams to benefit from distributed caching and compilation. The Horde Server is clearly needed to coordinate the tasks between agents, and the Horde Agents are required to accept tasks. Without either of these, distributed compilation cannot happen. These services need to be able to communicate across the network, but the ports for this communication can be minimized. Agents need to be able to reach the Server over an HTTP(S) and GRPC port (Dashboard & API), which can be combined into a single port if HTTPS is configured. Agents also need to be able to reach each other's IP over port 7000, potentially 7001-7010 if ports 7000-7009 are occupied by other software. Should security reasons prevent ports from being opened up, or from IPs being reachable (such as machines behind a Network Address Translation (NAT) layer), a VPN can be utilized to make these connections. In those cases, the agent's VPN IP needs to be explicitly specified as the 'Horde__ComputeIp' environment variable.

In case that the machines that run as Horde Agents are also used as workstations for developers, it would also be recommended to install Unreal Toolbox for monitoring activity, along with giving more control over when the Horde Agent should be assisting compiles or not. That being said, it is not a requirement, and distributed compilation can be run without as well.

This relatively short list of requirements tackles distributed compilation, and the setup stage of the TAR cycles revealed that the difficulty lies less in the number of services than in their configuration. Getting the services online is as easy as following a step-by-step guide, but getting these services configured and integrated with existing pipelines is where the challenge lies. This challenge can be overcome by someone familiar with the software stack (existing and/or incoming), or by someone who has sufficient knowledge of similar applications. Should authentication be desired, the main integration challenge would lie in authentication, specifically the automated provisioning of users in Horde's native user management or OpenID Connect authentication. Other integration challenges can appear when trying to configure more advanced features of Horde such as Perforce integration or project automation, but for distributed compilation these additional features are not required. User setup is

1448 the easiest part of deployment, as it is an installation like any other software, and pool allocation can be
1449 (re)configured server-side.

1450
1451 It should be noted that these requirements are specific to the current state of Epic Games' Horde and
1452 UBA toolchain. Alternatives such as Incredibuild or distcc will impose different infrastructure
1453 requirements, but remain out of scope for this research.

1454
1455 When it comes to distributed caching, measuring has been more challenging. Native caching kicks in
1456 transparently, usually only notifying about the remaining tasks that missed the cache. SCM servers and
1457 SCM clients were already in place before Horde was deployed to the teams. As these services are
1458 required for collaboration within teams, using them for caching provides no additional overhead.
1459 However, there is the issue of race conditions when merging multiple separate branches into the
1460 repository. To tackle these race conditions, further services might be required for proper handling of
1461 compile results. For example, Ari Arnbjörnsson mentions tools such as Unreal Game Sync in his article
1462 [3]. These tools tend to integrate with automation servers that prepare and perform the merging of
1463 branches or shelves into the main repository. Due to time and resource constraints, these full
1464 deployments could not be tested.

1465
1466 In summary, distributed compilation requires the combination of a Horde Server and Horde Agent(s),
1467 along with the potential for a VPN should communication between these services require it.
1468 Distributed caching largely utilizes existing SCM infrastructure, though for teams with multiple
1469 developers touching source code, the coordination complexity would quickly require further tools that
1470 manage compile results on the developers' behalf.

1471

1472 5.2 Sub-Question 2: What build time reductions can be expected from 1473 implementing distributed caching and compilation in specific configuration 1474 contexts?

1475 Hypothesis for Sub-Question 2 was about the scaling that could be expected from implementing
1476 distributed compilation and caching. It was hypothesized that the shape of the graph would be
1477 hyperbolic. The exact function that was found from the scaling tests turned out to be a reciprocal,
1478 confirming the hypothesized hyperbolic shape. The exact functions that were found to match the
1479 dataset are shown in Figure 20, Figure 21, and Figure 22. The hypothesis for large compiles (UE5
1480 target) was a $\lesssim 8x$ speedup with 8 agents when compared to no acceleration. This hypothesis did not
1481 state the strength of an agent, but if we were to select the 8-core agent, an average speedup of $5.4x$
1482 (± 0.45) was observed. When single agent accelerated runs are compared to baseline, a speedup of

roughly $6.7x (\pm 0.15)$ has been found on the strongest single agent (64 cores). The inconsistency with the hypothesized speedup was not surprising, as the impact of scheduling and network overhead was unknown at the time this hypothesis was made, and did not take into account the amount of cores per agent. For small compile tasks, the hypothesis stated that there would be a near $1x$ speedup, not benefitting from distribution of the compile. However, compiling the smallest UnrealPackageTool target yielded a maximum speedup of roughly $1.8x (\pm 0.29)$ compared to baseline. This speedup exceeded expectations. Though whether this directly equates to real-world speedups is debatable, as the UnrealPackageTool is a target with 65 compile actions. Small or incremental compiles can have even lower amounts of compile actions, negatively affecting the amount of parallelization / acceleration that developers could benefit from.

The scaling models used to find the functions in Figure 20, Figure 21, and Figure 22 have been accurate for single agents, but scaling up to multiple agents has yielded implausible results, indicating that there are likely inaccuracies that are being absorbed by the existing variables. Adding a variable to distinguish scheduling overhead from compilation tasks could solve this, but it would require being able to distinguish between time spent on scheduling and time spent on compilation. When expanding the model with another parameter, the current multi-agent datasets consisting of 4 data points would be insufficient to reliably fit a four-parameter model.

The constants found for the reciprocal functions in Figure 20, Figure 21, and Figure 22 have also been used to calculate the percentage of the compile which can be parallelized (Formula in Figure 25). These percentages are not exact, and can vary when initiators have stronger CPU cores, or when there is more scheduling overhead due to a larger amount of agents. This is because the true percentages are a property of the compile task being executed, but using this methodology, they have been derived from measurements performed on specific hardware. Meaning that the percentages presented for each compile task are approximations of their true parallelizability. Despite these potential deviations in the percentages, they can still provide an estimation of acceleration on different systems, so long as that system's baseline (no acceleration) time is known.

For RAM scaling, the effect of lower amounts of RAM depended more on the size of the compile task than expected. The hypothesis stated that less than 1GB of RAM per compilation core would start degrading compilation performance. For the large UE5 target, this appears to have been close to the mark, where 1 GB per core started slowing down the degree of acceleration. For both the UnrealInsights and UnrealPackageTool, the slowdowns did not appear until 0.5GB of RAM per core. For all these cases, dropping below these amounts of RAM effectively prevented agents from assisting in the compile at all, as though there were zero agents assigned to the compile in the first place. In some

cases agents assisted a small amount when below their noted RAM thresholds, managing to compile a small number of actions, but these successful completions did not accelerate the compilation in a way that could be distinguished from regular run-to-run variance.

To minimize the potential for run-to-run variance, the initiators were fresh installs of windows. Any installations of software were done on both to keep the testing conditions as tight as possible. However, since these are laptops, connected over WiFi, and running Windows. External factors such as WiFi speeds, background processes, or simply timing form an unavoidable variation between runs. Measures such as fresh installs, identical hardware, and identical setup have been used to avoid this as much as possible, but can never negate it.

Finally, the VPN scaling unfortunately had to be dropped. Due to firewall problems and the agents being hosted within a Kubernetes cluster (resulting in them being behind a NAT of the Kubernetes node's IP), the non-VPN case could not be set up properly. This meant that there was no comparison between a VPN case and a non-VPN baseline. All the results of these tests were therefore performed via a VPN layer, adding a network latency of 18ms to each request, and limiting bandwidth to ~200Mbps up or down. Without VPN overhead, the connection to the VPN server has a bandwidth of ~260Mbps, with a latency of 12ms. Trace files did not show transfer speeds above ~120Mbps, due to both initiator and agent sharing network connections, this can be close to the VPNs bandwidth limit, especially when combining upload and download. Trace files however also showed large spans without major network bandwidth utilization, which makes it unlikely that bandwidth limitations impacted the compile times, especially since agents have multiple actions running simultaneously. The lower latency of a LAN compile could yield even better results than the numbers found in this thesis, but to know the full effect of network latency and bandwidth, further testing would be required. For this reason, the parallelization percentages mentioned previously are a conservative estimate.

5.2.1 *NetBird Relay Mode*

By design, NetBird is a Peer-To-Peer (P2P) VPN, usually this means that connections are made between clients directly. This reduces the latency overhead and increases the bandwidth, as it no longer needs to traverse via the VPN server. However, due to firewall restrictions, NetBird was forced to fallback onto its Relay protocol. This meant that traffic was relayed through the VPN server. For this reason, NetBird was not able to benefit from these more optimal P2P routes.

5.2.2 *Aitken's Extrapolation*

At first, Aitken's extrapolation was evaluated as a method of series acceleration to estimate the asymptote (theoretical maximum for acceleration), but results were inconclusive and inconsistent.

Aitken's method works over 3 successive points in a sequence, with the assumption that there is a consistent change between points. This led to the middle of those 3 points being extremely influential on the eventual outcome. Due to this, applying a method that assumes geometric convergence to real-world, inherently imperfect data, led to answers that carried little to no meaning. This manifested itself as asymptotes above a measured data point (suggesting that tests outperformed a theoretical limit), or a negative asymptote. For these reasons, regression algorithms that optimize across all data points were preferred, as these models account for variance in the data whilst trying to minimize it.

5.3 Sub-Question 3: How should distributed caching and compilation be configured for developers to obtain ideal workflows?

Hypothesis 3 is about how distributed compilation and caching should be configured for developers to obtain ideal workflows. This hypothesis was hard to formulate at the start, as no data existed about the form of problems that the TAR cycle would run into, and how tweaks to configuration could potentially solve these problems. As stated in the Methodology, resource pooling had already been configured based on pre-emptive findings, ensuring a more fair distribution of compute resources within teams. This way, a team will only ever be affected by their own compile tasks, never by another team.

One of the expectations was that survey question 15 would yield negative results (indicating unclarity of when / what the system was running), resulting in negative results for question 16 (Indicating that the system was not running when it was expected to). Question 15 did indeed yield negative results, which can be explained by the activity indicator no longer being shipped with the agent by default, having been integrated into Unreal Toolbox instead. However, Question 16 yielded positive results, the opposite of what was expected. But due to the negative score on Question 15, it is also more challenging to gauge whether Question 16's results are reliable. If participants could not distinguish whether the system was running, then it would inherently also become challenging to realize that it was not running when it should have been. It was hypothesized that this would negatively impact the scoring, but it appears to have done the opposite. Due to this it is not unlikely that the positive scores on question 16 could be attributed to acquiescence bias.

After the second iteration, the Unreal Toolbox was deployed as part of the configuration. This improved the visibility of the system, giving clearer indication of when it was or was not running. Initial evaluations of the tool appeared to disable the Horde Agent more than it should have, but this was a misconfiguration on the researcher's part. Though this thesis aimed to keep all configurations consistent across machines, a production deployment should also tweak these values based on the machine and its capabilities. Configuring the Horde Agent to not use 100% of the CPU improves the

impact on performance noticed by developers, but unfortunately the size of tasks picked up can only be tweaked by CPU count, not by RAM availability. In cases of low RAM availability, memory will be swapped to a pagefile, which is stored on persistent storage, incurring heavy latency and bandwidth restrictions. This swapping to and from persistent storage causes heavy stuttering to user applications, often being more noticeable than high CPU utilization, because the operating system prioritizes user applications over compilation processes launched by the Horde Agent. Unreal Toolbox can also monitor RAM availability, but this only gives control over fully enabling/disabling the agent, rather than scaling for the amount of RAM that is available.

RAM scaling tests confirmed that under high memory pressure, the Horde Agent still accepts the same amount of compile tasks. This thin spread of available memory results in compile actions taking substantially longer, or fully timing out, which prompts redistribution of that compile task to another available agent, taking up additional compute resources for zero acceleration benefit. If the amount of accepted compile tasks could scale with memory availability, both CPU and RAM could be utilized more efficiently. This is particularly relevant given the ongoing global DRAM shortage, which has seen memory prices increase substantially since late 2025 [52]. Utilizing existing RAM more efficiently reduces the need for RAM expansions to account for peak load.

In summary, the system should be configured to only compile on roughly 50-60% of its logical cores. This allows each core to be compiling, with potential of a small amount of threads that can run on cores that are stalled for I/O or other reasons. This can use a large part of the potential compute of the agent, without generating an excessively large memory footprint that needs to be shared with user applications. Unreal Toolbox is a good addition for developer workstations, both for visibility of what Horde is doing in the background; along with easier configuration of the Agent, as configuration would otherwise have to happen through a file in the program's data folder. In an educational setting, it would also be recommended to configure resource pools for teams to only utilize their own resources. In a studio environment, this can be done for automation purposes, but separating compile tasks between teams is less necessary in that context.

5.4 Sub-Question 4: What configuration trade-offs exist between cluster hardware variance, compile performance, and developer workflows?

When using developer's workstations as the core source of compute power, the impact on their usual workflow should be minimized. It was hypothesized that the default configuration would cause a more noticeable performance impact on low-end laptops based on the amount of CPU cores a laptop has. Instead, it appears that the performance impact is more dictated by the amount of RAM available per

CPU core. The laptop on which the largest performance hit was noticed had 24 CPUs and 32 GB of ram, yielding 1.3 GB per core, being one of the lower RAM-per-core ratios observed as shown in Table 9. This ratio is not a direct cause, but facilitates a larger amount of simultaneous compile tasks which have to share the relatively lower amount of RAM. This closely relates to one of the primary tradeoffs that can be made through configuration, which is the amount of compute that a developer's machine shares with Horde. Sharing a larger percentage allows distributed compiles to be run faster, but has a higher risk of incurring noticeable slowdowns. Sharing a lower percentage lowers the risk of noticeable slowdowns, but also reduces the speedup that can be achieved. This is because more devices would be required to reach the same amount of compute, incurring a larger scheduling overhead. Amongst the participants, some members from Team-1 have a laptop with only 1 GB of RAM per logical CPU thread. But this team also did not have many large-scale compile tasks. And as shown by the RAM scaling tests, smaller-scale compile tasks suffer less from memory pressure.

Despite not having been able to measure the impact of a VPN, setting up one (even if not required for firewall restrictions) can still be beneficial to allow developers access to distributed compilation when working outside of the office network, such as working from home. Trace files from the teams do show that the educational teams utilize acceleration less often, this could be because there are fewer cases where distributed compilation is beneficial. But could also point to the VPN authentication having expired, and not having been re-authenticated. The hypothesis for Question 16 of more negative responses from the educational team also does not hold true. This could be explained by the same reasoning as was mentioned in 5.3; of question 15 (visibility of the system) yielding negative results.

In summary, agents hosted on developer workstations should be configured in a way where their RAM footprint is manageable. A future improvement to the Horde Agent could allow it to dynamically scale based on RAM availability. VPNs can also be deployed for more benefits than just enabling communication across networks with restrictive firewalls.

5.5 Sub-Question 5: What practical trade-offs exist when increasing hardware variance of the cluster?

The final hypothesis is about trade-offs or implications that arise from practical sources. These are aspects that cannot be solved by tweaking configuration. It hypothesizes that a VPN service such as NetBird would be beneficial over a centralized VPN server due to connection resiliency via other peers. In practice, it is rarely one of the VPN tunnels that causes failure, more often it is the VPN having been disabled, or authentication having expired. When developers initiate a compile, it would be easier to notice something wrong, as compile logs would show failed connections to the Horde Agents. But for

1658 developers that do not compile and merely share their compute power with the cluster, it is hard to
1659 notice anything wrong. On the one hand, this is good news, as it does not require attention from regular
1660 developers, and keeps running in the background; on the other hand, this does also cause issues to go
1661 unnoticed, leading to a reduction in the compute available for acceleration. If Horde is to be deployed
1662 more permanently, and firewalls still mandate a VPN, it would be recommended to have a static VPN
1663 tunnel. This avoids users having to handle authentication client-side, whilst keeping services routable
1664 over the network. The only caveat with static VPN tunnels is that the access required to configure this
1665 is usually similar to the access required to configure the firewall. At which point configuring the
1666 firewall to allow this LAN traffic would be better due to the aforementioned overhead in latency and
1667 limitations in bandwidth.

1668
1669 Another hypothesized trade-off was in the discipline of participants. Due to programmers often being
1670 the ones that need to compile, it was hypothesized that they would experience the benefits of the
1671 system more positively than other participants. It might be a bit more specific, as not all programmer
1672 tasks are equally accelerated. Programmer tasks that include compiling large sections of code (e.g. the
1673 whole project for a release, or a new engine version that the team might switch to). Other
1674 programming tasks tend to have smaller compiles, consisting of smaller modules, or incremental
1675 compiles after an iteration. These smaller compiles are not enough to meaningfully benefit from
1676 distributed compilation.

1677
1678 Lastly, it was hypothesized that non-programming participants would disable the system more
1679 frequently than programmers. In practice, the opposite was true. Programmers are often the ones
1680 compiling with the system, and therefore also run into problems head-first. Non-programmers did not
1681 run into these issues, and therefore likely saw less reason to disable it. Of the problems that
1682 programmers ran into, the first observed problem was intermittent compile errors. Building a second
1683 time always fixed these errors, which might point to distribution of compile tasks occasionally failing.
1684 These intermittent errors could always be resolved by retrying these failed parts, but Horde does not
1685 detect or retry these compile actions natively. The second, and more problematic issue, was that having
1686 distributed compilation configured has a chance of breaking the cook stage of packaging. Due to Horde
1687 also serving as a shared DDC provider, the cook stage attempts to connect to Horde to check for cached
1688 items that can be re-used. The compile and cook stage each communicate with the Horde server in
1689 their own way, meaning that the compile stage can benefit from the distributed compile Horde
1690 facilitates, but the cook stage fails authentication and causes the build to fail. Within Unreal Engine this
1691 authentication failure should be a warning and continue locally as if no Horde is configured, but due to
1692 it being an error is treated as a failure of the cook stage. Unreal has no known method of re-
1693 authenticating the cook stage with Horde to resolve this authentication failure. This example is a

known issue with proposed pull-requests to fix it [51], but it serves as an example of the problems that a team might run into.

Summarizing this means that VPNs should still be carefully considered. VPNs add a point of failure that can act up in the background or appear as a misconfiguration. If using a VPN to bypass firewalls, it would be best to minimize the amount of (re-)authentication for it. A secondary VPN could be set up with more strict authentication requirements for developers connecting from outside of the office's LAN. It was also found that Programmers performing large-scale compilation tasks are the most likely to interact with and disable the system. They benefit most, but due to this higher usage also run into problems more regularly. Non-programmers, contrary to expectations, were the least likely to disable the system. Most of the problems observed thus far are likely to be fixed in future updates of the Horde system.

5.6 Additional Finding 1

One of the additional interesting findings that does not quite fit in any of the hypotheses is the usage of Kubernetes within this research. The Horde Server was deployed in a Kubernetes cluster for Team-3. They experienced frequent prompts for (re-)authentication. This might have had something to do with the fact that this Kubernetes node was configured to use NUMA-aware node scheduling. Non-Uniform Memory Access is a technology used if a system has multiple CPU sockets, usually in servers. Due to the structuring of L1, L2, and L3 cache on a CPU, it is preferred to have one application run on one CPU rather than running part on one and part on the other CPU (physical cache locality). This meant that if other applications request a large part of one of the NUMA nodes, the Kubernetes scheduler would attempt to shuffle workloads around on the CPUs to make space for that application to fit onto the NUMA node. This shuffling of workloads often happens by recreating a workload elsewhere, as Kubernetes assumes deployments to be stateless, therefore not caring about restarting. The Horde server is not a stateless container though. Its database is hosted internally, and any active sessions would be lost upon restart, prompting re-authentication. For this reason it is recommended that the Horde server not be hosted fully within a Kubernetes environment, or to at least ensure that the potential of workloads being shuffled will not break existing sessions.

That being said, the Scaling tests also used Kubernetes to host Horde agents. This worked without issue, as the agents hold no state. Agent disconnects also prompt redistribution of any jobs that the agent was executing at time of disconnect. So this recommendation of not fully hosting the Horde server within a Kubernetes environment does not apply to Horde agents. In hindsight, it is possible that the reconfiguring of Scaling Tests (e.g. moving from an 8-core test to a 16-core test) amplified the amount of prompts that caused Kubernetes to move the Horde Server container of Team-3, as the

Kubernetes cluster used to host Team-3's Horde Server is the same cluster as was used for the scaling test agents. Though this is still not recommended, it could be less prevalent in a stable production environment that rarely shifts the applications being run in Kubernetes.

5.7 Additional Finding 2

Another interesting finding from the attempts of setting up the non-VPN scaling tests was the behaviour that the Horde agent employs for port mapping. The agent can configure both its ComputeIP and ComputePort through settings or environment variables. These settings are then used by the Horde Server to make and maintain connections to issue job leases. However, whenever a compile is started by an initiator, connections are made using the compute IP, but not the port, which always defaults to the 7000-7010 range. The agent decides which port from this range to use based on availability in its environment. It will attempt to bind to 7000 first, if this fails, it will try 7001, 7002, etc. But because the agent is hosted within a container, it has a dedicated port mapping space and does not know about the ports available on the host machine, always binding to 7000. Combined with the fact that its internal port will be advertised to the Horde server (and thus used for connections by initiators), there will be multiple agents mapping to the same IP and port.

These constraints mean that there were very few methods that could work, none of which were viable:

- Using a VPN to "join" the IP subnet of the internal Kubernetes container network. This would give initiators a routable path to each agent's individual IP, but would solve the non-VPN scaling test by deploying a VPN, defeating the purpose. This is compounded by the security implications of a very dubious network routing solution.
- Forcefully occupying increasingly many ports within the container to force the fallback to choose the ports 7000-7010 in succession, and then mapping those externally to the same port range. However, besides being a mildly ridiculous solution, it would force an 11 agent maximum per IP. Finally, Kubernetes' nodePort prohibits binding to ports below 30000 by default.
- Mapping agents to Kubernetes node's host network. This would work, but is heavily discouraged for security reasons, and due to the vLAN layout of the Kubernetes nodes, initiators would not be able to route to that host node. Similarly to the previous point, this would also suffer from the 11 agent maximum.

5.8 Additional Finding 3

The way Technical Action Research was implemented within this research was less than optimal. Iterative changes to the configuration worked excellently. It is the SUS part that yielded less usable

results. Throughout the cycles and eventual analysis, it was noted that more emphasis was placed on the comments made in the survey, along with the additional questions that were asked about the system. They formed more actionable changes, raising issues through descriptions rather than numbers. Especially for a system that is supposed to work in the background with little to no user interaction, evaluating user interaction with the system is counterproductive if the point of the system is to work with as little interaction as possible. Instead of the SUS survey, a more qualitative approach might have been preferred. Formal thematic analysis would be excessive for what might as well be bug reports, but a conversation can often lead to a more direct diagnosis of a participant's experiences or issues with a system.

With SUS heavily focusing on interaction with the system, it is true that participants that utilized the compilation acceleration can fill in the SUS survey with meaningful data, but participants that had the Horde Agent running purely in the background would add data points with little experiences to inform those scores.

Another solution that could have worked is to keep quantitative metrics according to the SPACE framework, but to measure these metrics at a different point in the pipeline. The survey that was used measures the perception of the system by the developers, not the objective telemetry data of participating teams. This was partially done through the analysis of UbaVisualizer trace files, though this also depended upon participants sending these files to the researcher. In the Horde version that was used for this research, the telemetry backend is still incomplete, but for future research, implementing more objective telemetry analysis could be a viable augmentation of how developers perceive the system.

5.9 Main Research Question: How can short term Unreal Engine projects improve their development processes by creating and distributing compiled data by utilizing existing infrastructure in a more optimized way?

To bring all this back to the main research question, benefitting from this system optimally has a few requirements and recommendations. To guide distributed compilation deployments, the flowchart in Figure 35 has been prepared.

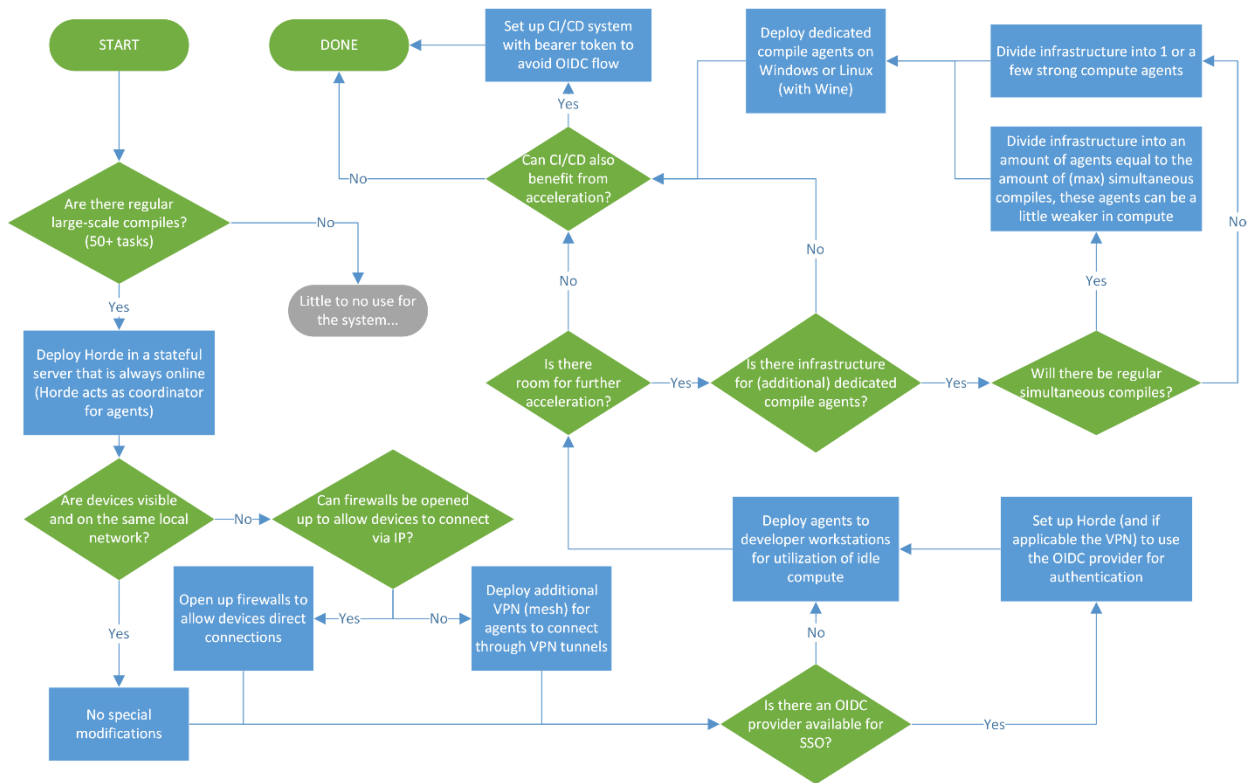


Figure 35. Flowchart steps to deploy Horde for distributed compilation

Before even starting, there is the evaluation whether the system will benefit anyone. If there is not much to compile in the first place, there will be little to no use for the system. 50 compile actions has been chosen as the threshold. This has been based on the observed 1.8x speedup at 65 compile actions, rounded to a more generalizable threshold.

To start off, the Horde Server will need to be deployed somewhere on the network, this does not necessarily need to be a strong machine, as it will not be doing any of the compilation, nor will any compile session data flow through this machine. Its purpose is being the central hub that connects and coordinates agent tasks. It should however, not be deployed as a Kubernetes Deployment (which assumes statelessness). Instead, a direct installation, or a Docker (Compose) would be preferred.

After the Horde Server has been deployed, the network will need to be configured such that future Horde Agents can connect and communicate with this Horde Server instance, for example through an optional reverse-proxy, or just the direct IP of the Horde Server. The network will also need to allow connections between devices that are going to be Horde Agents, at minimum, needing to allow TCP ports 7000-7010. If desired, this is also the opportune moment to configure any authentication settings such as OIDC.

With the Horde Server deployed and reachable, agents can now start connecting and downloading the agent software packages (Horde Agent & Unreal Toolbox). Idle compute from developer workstations should be leveraged first, as this comes at no additional cost. From this point, developers can start benefitting from the system by configuring the new Horde server as their UbaController in the Engine.ini file for the project, or for the engine as a whole (impacting every project to be opened by that engine). For developer machines, having a 50-60% allocation of CPU cores in 'Workstation' mode is a good starting point. Depending on tasks and available RAM, tweaking this a little further might be needed to minimize interference with user applications. If the background compilation of the agent starts interfering with the developer's workflow, it negatively impacts their workflow. This can lead to further issues and frustrations that outweigh the performance gain.

If using resource pools within Horde, there are different ways of structuring the pools to which agents (especially workstation agents) get assigned. A few practical solutions could be:

- Static Team assignments. If a team is always statically working on a single project, it can be configured according to those teams, and not need updating.
- Project name assignments. If projects switch regularly, the developer can configure which project(s) their agent should contribute to by adjusting the agent.json file on their machine.
- Developer name assignments. If projects switch regularly, the server can group developer names into a project pool by adjusting the globals.json file.

There are also some other considerations that could matter. First off, it's worth asking whether there is room left for further acceleration after developer workstations have been enrolled as agents. The parallelization percentages mentioned in Section 4.5 can give an estimate of acceleration headroom, but these percentages cannot be cleanly mapped onto other projects with different dependency structures. If it is found that there is further potential for acceleration, more classical dedicated compile agents can be considered. Depending on the amount of developers compiling simultaneously, it can be beneficial to split this compute in different ways. With the note that containerized deployments can only host a limited amount of agents per IP/machine.

Finally, automation systems such as Continuous Integration (CI) servers can also be configured to connect to this distributed compile to accelerate the automated builds. However, in the case of CI servers, it might be a cleaner solution to have it build on its own, as to prevent the distributed compilation system from inserting potential errors into the automated builds. This consideration will be left to whoever will be implementing this system, as the trade-off between speed and reliability will vary from project to project.

That takes care of distributed compilation, Distributed caching is another section that can benefit development workflows. SCM infrastructure can be used for this, but can cause mismatches in binaries when developers are pushing the binaries themselves. This means that the binaries that the engine attempts to execute are not aligned with the latest source code, as the engine expects those two to always be in sync. There are a few potential solutions for this problem, but this thesis did not have the time to evaluate full implementation of these methods:

- More rigorous submit standards can be one solution to this problem, requiring developers to get the latest version of source code, and compile everything before submission. This does introduce new race conditions between source code changes. Especially in the time between when the last version was retrieved, and the time the compile is ready to be submitted.
- Branching with automation on merging into a main branch. Where an automation server takes charge of performing the compile for the rest of the team, ensuring that the main branch always has binaries matching their source code. This would queue up any other merges into the main branch so that only one merge can be performed at once, solving the race condition that the previous option created.
- Software with the purpose of explicitly tracking which source code revisions match with which compiled binary revisions, sitting as a middle-man between the developer's local version and the SCM server. This can solve race conditions by not retrieving the latest source code unless binaries are also available for them. Usually, these systems are also combined with an automation server performing compilation as was mentioned in the second option. An example of software that is designed to fit this purpose is UnrealGameSync [53].

The reason for not exploring distributed caching further is that two of the participating teams did not have their SCM server configured to support branching. And they preferred to stick with each developer using distributed compilation instead of deploying distributed caching in the team. Due to these restrictions distributed caching has not been evaluated as thoroughly as distributed compilation. This leaves the possibility that smaller teams perceive distributed caching as less important than distributed compilation. There is a certain level of invisibility bias that can appear here, as the speedup of compilation is more tangible than the lack of needing to compile in the first place, but this research was not designed to evaluate this perception.

Chapter 6: Conclusion and Future Directions

In conclusion, this research has shown that deploying distributed compilation across a cluster consisting of developer workstations can be a viable alternative or supplement to dedicated compile hardware for Unreal Engine builds. So long as compile tasks are large enough to distribute across a cluster of workstations and do not noticeably slow down said workstations from the additional workload. These slowdowns can be prevented by limiting compute tasks statically, but due to the variability of development workflows, the potential for dynamically limiting the amount of compute tasks based on system load remains unexplored.

This method of distributing compilation is helpful as a cost-saving method when dedicated compile is too expensive. Especially in budget-constrained educational environments, or small studios that are running on low budget. Allowing developers to spend less time waiting on long compiles, without always requiring the acquisition of dedicated compile hardware. Studios with larger budgets can also utilize developer workstations for acceleration, but for them the higher performance of dedicated compile hardware might be valued more than the potential cost-savings.

This research focused specifically on Unreal Build Accelerator in combination with Unreal Horde, tools specifically made to integrate with the Unreal ecosystem. This means that these tools bear little use outside of said ecosystem, however, there are other tools available that are applicable for different compilation contexts, which can show similar improvements from distributing compilation efforts across a cluster of developer workstations; both in game development, and outside of it. Using the reciprocal scaling model as used in this thesis, estimations can also be made as to the parallelizability and potential speedup of other codebases.

Distributed caching was planned to also be a large part of this research paper. Though due to practical restrictions, it could not be fully evaluated. This is because the SCM servers used by the participating teams did not meet the requirements of the software required to prevent race conditions in code merges. This suggests that distributed caching – especially in the context of Unreal – may first require different SCM configurations than the (functional) default.

The System Usability Scale (SUS) that was used to evaluate the different iterations, yielded little useful results. The augmenting questions were more useful to gauge participant's experiences and inform iterative changes to the configuration. SUS is made for measuring interactions with a system, but for a system where the ideal case is minimal interaction, SUS might not be the correct tool for the job.

1917 There were some limitations throughout this research. Naturally, these lead to several points that
 1918 could be further researched in future explorations of this field:

- 1919 • VPN overhead compared to LAN
 1920 How much faster is compilation when VPN latency and bandwidth limitations are removed?
- 1921 • Distributed caching infrastructure
 1922 What additional services might be required for a proper distributed caching layer
 1923 complementary to the distributed compilation?
- 1924 • Parallelizability scaling
 1925 What factors influence the parallelizability percentages of compile tasks?
- 1926 • Scaling agent compute capacity dynamically
 1927 Can agents be made to dynamically scale their remote compute capacity based on user's
 1928 current CPU and/or RAM utilization?
- 1929 • Perception of caching vs compilation benefits
 1930 Is the perceived benefit from distributed compilation and distributed caching different from the
 1931 actual benefit?
- 1932 • Larger-scale deployments
 1933 How does the system's effectiveness change when deployed at larger scales?
- 1934 • Intermittent compile failures
 1935 How can the intermittent compile failures caused by distributed compilation be fixed?
- 1936 • Authentication for cook stage
 1937 Can the authentication failures observed during the cook stage be resolved in a reliable way?
- 1938 • Evaluation metrics for transparent systems
 1939 What methods are most effective for evaluating user experience with systems designed for
 1940 minimal user interaction?

1941

1942 Ultimately, this research has shown that the hardware to accelerate compilation may already be
 1943 present in a team's existing workstations. By configuring these workstations to share idle compute,
 1944 teams can reduce the time developers spend waiting on builds – and with it, the cognitive cost of going
 1945 off-task when compiles take too long.

1946

1947
1948
1949
1950
1951

1952
1953

1954
1955

1956
1957

1958
1959

1960
1961

1962
1963

1964
1965

Appendix A: Horde installation guides

A.1 Unreal Horde Server Deployment (Windows)

Most up-to-date version: <https://blog.hidde-derks.com/unreal-horde-server-deployment-windows>



Unreal Horde Server Deployment (Windows).pdf

A.2 Unreal Horde Server Deployment (Docker)

Most up-to-date version: <https://blog.hidde-derks.com/unreal-horde-server-deployment-docker>



Unreal Horde Server Deployment (Docker).pdf

A.3 Unreal Horde Agent Deployment (Windows)

Most up-to-date version: <https://blog.hidde-derks.com/unreal-horde-agent-deployment-windows>



Unreal Horde Agent Deployment (Windows).pdf

A.4 Unreal Horde Agent Deployment (Docker)

Most up-to-date version: <https://blog.hidde-derks.com/unreal-horde-agent-deployment-docker-linux>



Unreal Horde Agent Deployment (Docker).pdf

1966

Appendix B: Scaling Test agents' Kubernetes Manifests

1967

B.1 Main manifest

```

kind: Deployment
apiVersion: apps/v1
metadata:
  name: uba-vpn-agent-16-cores
  namespace: horde-testing
spec:
  replicas: 0
  selector:
    matchLabels:
      name: uba-vpn-agent-16-cores
  template:
    metadata:
      labels:
        name: uba-vpn-agent-16-cores
    spec:
      volumes:
        - name: netbird-data
          emptyDir: {}
        - name: agent-json
          secret:
            secretName: agent-configs
            items:
              - key: agent.json
                path: agent.json
            defaultMode: 420
        - name: appsettings-json
          secret:
            secretName: agent-configs
            items:
              - key: appsettings.json
                path: appsettings.json
            defaultMode: 420
        - name: production-appsettings-json
          secret:
            secretName: agent-configs
            items:
              - key: appsettings.Production.json
                path: appsettings.Production.json
            defaultMode: 420
        - name: entrypoint-sh
          secret:
            secretName: agent-configs
            items:
              - key: entrypoint.sh
                path: entrypoint.sh
            defaultMode: 493
      containers:
        - name: vpn-agent-16-cores
          image: >-
            gitea.sigma-erebus.com/sigma-erebus/netbird-horde-agent-
truenas:latest
          env:
            - name: NB_SETUP_KEY
              valueFrom:
                secretKeyRef:
                  name: agent-configs

```

```

        key: NB_SETUP_KEY
    - name: NB_MANAGEMENT_URL
      value: https://netbird.example.com:443
    - name: TZ
      value: cest
    - name: HORDE_SERVER_URL
      value: https://horde.example.com
    - name: HORDE_DIRECTORY
      value: /home/horde-docker/Horde/Agent
    - name: NETBIRD_TIMEOUT
      value: '600'
    - name: Horde__cpuCount
      value: '16'
  resources:
    limits:
      cpu: '32'
      memory: 32Gi
    requests:
      cpu: '32'
      memory: 32Gi
  volumeMounts:
    - name: netbird-data
      mountPath: /var/lib/netbird
    - name: agent-json
      readOnly: true
      mountPath: /home/horde-docker/Horde/Agent/Data/agent.json
      subPath: agent.json
    - name: appsettings-json
      readOnly: true
      mountPath: /home/horde-docker/Horde/Agent/appsettings.json
      subPath: appsettings.json
    - name: production-appsettings-json
      readOnly: true
      mountPath: /home/horde-
docker/Horde/Agent/appsettings.Production.json
      subPath: appsettings.Production.json
    - name: entrypoint-sh
      mountPath: /entrypoint.sh
      subPath: entrypoint.sh
  imagePullPolicy: IfNotPresent
  securityContext:
    capabilities:
      add:
        - NET_ADMIN
  restartPolicy: Always
  dnsPolicy: ClusterFirst
  nodeSelector:
    kubernetes.io/hostname: static-node-name
strategy:
  type: Recreate

```

1968

1969

B.2 agent.json

```

{
  "horde": {
    "wineExecutablePath": "/usr/bin/wine",
    "ephemeral": true,
    "workingDir": "/home/horde-docker/Horde/Workspace",
    "logsDir": "/home/horde-docker/Horde/Logs",
    "properties": {
      "EducationTeam": "Scaling-Test-1"
    }
  }
}

```

```

    }
  }
}

```

1970

1971

B.3 entripoint.sh

```

#!/bin/bash
set -e

NETBIRD_TIMEOUT="${NETBIRD_TIMEOUT:-60}"
HORDE_USER="horde-docker"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"
}

if [ -z "$NB_SETUP_KEY" ]; then
    log "ERROR: NB_SETUP_KEY environment variable is required"
    sleep 100
    exit 1
fi

log "Starting NetBird daemon..."
netbird service install 2>/dev/null || true
netbird service start
netbird up --setup-key "$NB_SETUP_KEY" --rosenpass-permissive

log "Waiting for NetBird connection..."
elapsed=0
while [ $elapsed -lt $NETBIRD_TIMEOUT ]; do
    if netbird status 2>/dev/null | grep -q "Management: Connected"; then
        if netbird status 2>/dev/null | grep -q "Signal: Connected"; then
            if ! netbird status 2>/dev/null | grep -q "NetBird IP: N/A"; then
                break
            fi
        fi
    fi
    sleep 2
    elapsed=$((elapsed + 2))
    log "Waiting for Netbird... (${elapsed}s)"
done

if [ $elapsed -ge $NETBIRD_TIMEOUT ]; then
    log "ERROR: NetBird failed to connect within ${NETBIRD_TIMEOUT} seconds"
    netbird status || true
    sleep 100
    exit 1
fi

log "NetBird connected, extracting IP address for Horde..."
NETBIRD_IP=""

NETBIRD_IP=$(netbird status --json 2>/dev/null | jq -r '.netbirdIp // empty' |
cut -d'/' -f1)

if [ -z "$NETBIRD_IP" ]; then
    log "ERROR: Could not determine NetBird IP address"
    netbird status || true
    sleep 100
    exit 1
fi

```



```
log "Netbird connected with IP: $NETBIRD_IP"

export Horde__ComputeIp="$NETBIRD_IP"

log "Environment configured:"
log "  Horde__ComputeIp=$Horde__ComputeIp"

mkdir -p /home/horde-docker/.wine
chown -R horde-docker:horde-docker /home/horde-docker/.wine
export WINEPREFIX=/home/horde-docker/.wine
export WINEARCH=win64

log "Starting Horde Agent"

cd Agent
exec sudo -E -u $HORDE_USER dotnet HordeAgent.dll
```

1972

1973

1974

1975

Appendix C: TAR Survey Data

C.1 Iteration 1

What is your given participant login name? (username used when logging into horde.sigma-erebus.com or hermes.sigma-erebus.com)	User-41	User-43	User-46	User-45	User-42	User-44
What is your discipline within the team?	Designer	Programmer	Artist	Designer	Producer	Artist
What is your specialization within the team?	Technical Designer	Build Engineer	Environment art, tech art	Game Design Level Design Audio	Producer and writer	Environment and UI Artist
1. I think that I would like to use this system frequently	4	5	3	2	4	3
2. I found the system unnecessarily complex	2	1	4	1	1	1
3. I thought the system was easy to use	5	4	2	5	4	3
4. I think that I would need the support of a technical person to be able to use this system	1	2	5	2	1	5
5. I found the various functions of this system were well integrated	3	5	3	5	4	3
6. I thought there was too much inconsistency in the system	2	4	3	1	1	3
7. I would imagine that most people would learn to use this system very quickly	4	4	1	5	5	4
8. I found the system very cumbersome to use	2	4	5	1	1	3

9. I felt very confident using the system	4	4	2	2	4	3
10. I needed to learn a lot of things before I could get going with this system	1	2	4	1	1	1
11. I think this system positively impacted my team development workflow	4	2	3	2	3	3
12. I noticed that the system affected my computer's responsiveness	3	5	5	5	2	1
13. I think this system positively impacted my personal development workflow	3	5	2	1	4	1
14. There were times when I found it necessary to disable the system (during work hours)	3	1	1	5	1	1
15. I could easily tell whether the system was active (either ready to receive tasks or actively running a task)	1	5	1	2	1	1
16. There were times where I expected the system to be running when it was not	3	1	3	3	1	1

C.1.1 Additional Notes

- I filled this in based mostly on experiences of other team members since I am currently using a Mac and therefore not using the UBA set-up, but I have interacted with it through other people's laptops and have gone through the initial set-up as a team.
- I do get exceptions like this where I have to build again for it to work.

Unhandled Exception: System.IO.FileNotFoundException: The specified module could not be found. (Exception from HRESULT: 0x8007007E)

ASSERT: Unhandled exception/crash. Suppress debugger startup and try to report issue instead.

This message is here to hopefully see callstack

Permissions do not work.

Working remote it does try to connect and then times out when I close the game. It wants to use telemetry. But it can't out of the office.

- With the initial set-up I had a very noticeable hit in performance, which made working in Unreal uncomfortable and stuttery.

Initially I did not know how to reduce the limit in the config file, but with help I then installed Unreal Toolbox where I reduced my core amount from max (24) to half (12).

After this I've not noticed any issues with performance (however I believe someone also mentioned that it seems my laptop is not being used for the swarm at all anymore, although I cannot confirm or deny this because I'm not sure how to check, but all I know is that I haven't had any issues since I changed the core amount. Hopefully you can check the stats so you can draw some valuable conclusions)

After installing Unreal Toolbox and working from home I got frequent Windows notifications from Unreal Toolbox of "connecting to server" which would always fail (which makes sense because I was on my own network), but this frequent pop-up was a bit annoying. I ended up just muting this in Windows settings which made it no problem again.

I have not made any builds myself so therefore I haven't noticed the benefits myself directly, but I know our programmer was happy with the faster build when it was using my laptop

- I was wondering how much of my laptop resources were being used, like a graph somewhere. I think this is possible through the dashboard but only for admins.
- Honestly I even forgot I had it, haven't noticed it messing with my laptop performance, so I guess that's good haha! :)

C.2 Iteration 2

What is your given participant login name? (username used when logging into horde.sigma-erebus.com or hermes.sigma-erebus.com)	User-07	User-03	User-01	User-43	User-06	User-05	User-41	User-46	User-42
What is your discipline within the team?	Designer	Programmer	Producer	Programmer	Programmer	Artist	Designer	Artist	Producer
What is your specialization within the team?	Level Designer	Ci cd	.	Build Engineer	AI	Character Art	UI Design Tech Design & System	Environment art, tech art	Writer/Concept designer

							Program ing		
1. I think that I would like to use this system frequently	5	4	4	4	3	3	5	2	3
2. I found the system unnecessarily complex	1	2	2	2	1	1	2	3	2
3. I thought the system was easy to use	5	4	3	2	4	4	3	4	4
4. I think that I would need the support of a technical person to be able to use the system	2	2	4	3	2	5	1	5	2
5. I found the various functions in this system were well integrated	3	3	3	4	3	3	3	5	4
6. I thought there was too much inconsistency in this system	2	3	3	4	2	1	1	4	1
7. I would imagine that most people would learn to use this system very quickly	5	4	3	3	5	4	3	4	4
8. I found the system very cumbersome to use	2	1	2	4	2	2	3	4	2
9. I felt very confident using the system	3	3	3	4	3	3	3	2	3
10. I needed to learn a lot of things before I could get going with this system	3	2	4	2	1	3	1	1	1

11. I think this system positively impacted my team's development workflow	4	3	4	4	3	3	4	2	2
12. I noticed that the system affected my computer's responsiveness	1	2	1	4	3	1	2	5	2
13. I think this system positively impacted my personal development workflow	3	3	2	4	2	3	4	2	3
14. There were times when I found it necessary to disable the system (during work hours)	1	1	1	1	2	3	1	3	1
15. I could easily tell whether the system was active (either ready to receive tasks or actively running a task)	5	1	2	3	2	1	2	3	2
16. There were times where I expected the system to be running when it was not	1	1	1	1	3	2	4	5	2

C.2.1 Additional Notes

- I'm not sure if we even used it yet, but otherwise I can connect and disconnect easily. I am aware when the system is active and it is quite user-friendly.
- I don't know if I'm running it correctly.
- as a va who does not yet have tasks in unreal or shader compiling yet the installation of NetBird VPN has not affected me at all, which in my opinion is good as per the explanation it means that my work is not hindered by the use of the tool while my device might be able to help others who are using it.

2024
2025

- Build sometimes doesn't succeed due to Horde. I need to build again for a 2nd time, then it works.

C.3 Iteration 3

What is your given participant login name? (username used when logging into horde.sigma-erebus.com or hermes.sigma-erebus.com)	User-0	User-2	User-2	User-0	User-0	User-0	User-2	User-4	User-0	User-4	User-4
What is your discipline with the team?	Producer	Programmer	Programmer	Artist	Designer	Programmer	Programmer	Designer	Programmer	Programmer	Designer
What is your specialization within the team?	Producer	Gameplay Programmer	CICD and Audio	Character Artist	Level Designer - Lead	PR lead Engineer & Tool CI\CD	Gameplay, Tool	Technical Designer	AI&Physics	Build Engineer	Writer
1. I think that I would like to use this system frequently	4	2	3	3	5	4	5	5	3	4	4
2. I found the system unnecessarily complex	2	3	3	1	1	2	1	2	2	2	1
3. I thought the system was easy to use	5	2	2	4	5	3	4	4	4	2	4
4. I think that I would need the support of a technical person to be able to use this system	2	5	5	4	3	4	1	2	2	5	3
5. I found the various functions in this system were well integrated	3	3	3	5	5	4	4	4	3	4	4
6. I thought there was too much inconsistency in this system	2	4	5	1	2	1	4	1	3	4	1
7. I would imagine that most people would learn to use this system very quickly	5	2	3	4	5	3	4	4	4	4	5

8. I found the system very cumbersome to use	1	4	5	1	1	2	2	1	2	4	1
9. I felt very confident using the system	4	1	3	3	5	3	4	4	3	4	4
10. I needed to learn a lot of things before I could get going with this system	2	2	3	2	1	2	2	1	1	4	1
11. I think this system positively impacted my team's development workflow	4	3	2	3	5	3	5	4	3	2	4
12. I noticed that the system affected my computer's responsiveness	1	4	3	1	1	2	1	2	4	4	2
13. I think this system positively impacted my personal development workflow	1	2	1	3	3	3	4	4	3	5	3
14. There were times when I found it necessary to disable the system (during work hours)	1	5	4	1	1	1	1	1	3	4	1
15. I could easily tell whether the system was active (either ready to receive tasks or actively running a task)	4	3	3	1	5	2	2	4	2	2	2
16. There were times where I expected the system to be running when it was not	2	1	3	1	1	3	2	2	3	2	1

C.3.1 Additional Notes

- The main issue for us was that the log in prompt kept popping up randomly. Sometimes it would not show up the entire day sometimes it would 4-5 times. It also kept causing the build server to not build (this was also not consistent).
- We had many automated builds give false failures because Horde was unable to log in. It was inconsistent with those builds; sometimes it would be fine, other times it would fail.

I noticed barely any performance gains when I had to rebuild the project on my laptop, as this would be the only instance where Horde would kick in. Other times, my builds were too small to be distributed.

- It would be nice to have some insight in how many horde nodes are actually working together on a build
- I am very positive about the build time from 1,5 hour custom engine to 30 min when I build for Windows.

Problems I've encountered:

- Jenkins build for Linux I have to to set bAllowUBALocalExecutor to false.

<https://forums.unrealengine.com/t/ue5-6-linux-wrong-cross-compile-toolchain/2540791/14>

Error that I got in the past:

Failed to produce item:

ASSERT: ERROR: Access violation reading at address: 0x00000010CAC00600C

CALLSTACK:

_mi_heap_realloc_zero (alloc.c:710)

ucrtbase.dll: +0x3d189

ucrtbase.dll: +0x14a0c

ucrtbase.dll: +0x1488b

ucrtbase.dll: +0x14844

ucrtbase.dll: +0x14e01

combase.dll: +0x178d3d

combase.dll: +0x178d5d

combase.dll: +0xcaf23

combase.dll: +0xcac8c

combase.dll: +0xcb64a

combase.dll: +0x1d252f

combase.dll: +0x164034

combase.dll: +0x1c361f

combase.dll: +0x1c160d

combase.dll: +0x1bc4a8

C:\Jenkins\Project\Game\Binaries\Linux\Game-Linux-Shipping.sym

- With only Win11 agents + server still sometimes get the error down below:

LogUbaHorde: Warning: Getting Horde server URL failed [Source: Not found]

LogUbaHorde: Error: Failed to create HttpClient for UbaAgent

During cooking/packaging. While building C++ has no issues. I feel like when I am logged in in advance that it will work. Because then it has the correct browser cookies, not sure.

- Issue that is solved:

Unhandled Exception: System.IO.FileNotFoundException: The specified module could not be found. (Exception from HRESULT: 0x8007007E)

ASSERT: Unhandled exception/crash. Suppress debugger startup and try to report issue instead.

This message is here to hopefully see callstack

CALLSTACK:

UbaDetours.dll: +0x2b875

KERNEL32.dll: +0x78182

KERNEL32.dll: +0x511f2

KERNEL32.dll: +0x787ed

KERNELBASE.dll: +0x1158bc

ntdll.dll: +0x16a4a3

ntdll.dll: +0x121853

ntdll.dll: +0x16618f

ntdll.dll: +0x12327

ntdll.dll: +0xa961

KERNELBASE.dll: +0xca80a

clr.dll: +0x181078

I made sure there are only Win11 agents + server.

2097
2098

Appendix D: Core / Agent Scaling Test Data

Agents	Cores/Agent	Group	Run	Target	Total Cores	UBAVisualizer (Session) Time
0	0	1	1	UE5	0	9999
0	0	2	1	UE5	0	11316
0	0	1	2	UE5	0	9977
0	0	2	2	UE5	0	11231
1	32	1	1	UE5	32	1863
1	32	2	1	UE5	32	2147
1	32	1	2	UE5	32	1787
1	32	2	2	UE5	32	2066
1	16	1	1	UE5	16	3018
1	16	2	1	UE5	16	3120
1	16	1	2	UE5	16	3034
1	16	2	2	UE5	16	3133
1	64	1	1	UE5	64	1546
1	64	2	1	UE5	64	1616
1	64	1	2	UE5	64	1615
1	64	2	2	UE5	64	1602
1	8	1	1	UE5	8	4550
1	8	2	1	UE5	8	4850
1	8	1	2	UE5	8	4599
1	8	2	2	UE5	8	5033
1	4	1	1	UE5	4	6282
1	4	2	1	UE5	4	6783
1	4	1	2	UE5	4	6185
1	4	2	2	UE5	4	6838
4	4	1	1	UE5	16	3294
4	4	2	1	UE5	16	3562
4	4	1	2	UE5	16	3286
4	4	2	2	UE5	16	3331
4	8	1	1	UE5	32	2338
4	8	2	1	UE5	32	2390
4	8	1	2	UE5	32	2133
4	8	2	2	UE5	32	2310
4	16	1	1	UE5	64	1772
4	16	2	1	UE5	64	1801
4	16	1	2	UE5	64	1653
4	16	2	2	UE5	64	1842
4	24	1	1	UE5	96	1458
4	24	2	1	UE5	96	1562
4	24	1	2	UE5	96	1434
4	24	2	2	UE5	96	1552

8	4	1	1	UE5	32	2949
8	4	2	1	UE5	32	3265
8	4	1	2	UE5	32	2701
8	4	2	2	UE5	32	3082
8	8	1	1	UE5	64	1792
8	8	2	1	UE5	64	1920
8	8	1	2	UE5	64	2000
8	8	2	2	UE5	64	2117
8	12	1	1	UE5	96	1620
8	12	2	1	UE5	96	1945
8	12	1	2	UE5	96	1517
8	12	2	2	UE5	96	1839
0	0	1	1	UnrealInsights	0	477
0	0	2	1	UnrealInsights	0	544
0	0	1	2	UnrealInsights	0	522
0	0	2	2	UnrealInsights	0	532
1	32	1	1	UnrealInsights	32	149
1	32	2	1	UnrealInsights	32	176
1	32	1	2	UnrealInsights	32	145
1	32	2	2	UnrealInsights	32	160
1	16	1	1	UnrealInsights	16	197
1	16	2	1	UnrealInsights	16	216
1	16	1	2	UnrealInsights	16	189
1	16	2	2	UnrealInsights	16	220
1	64	1	1	UnrealInsights	64	133
1	64	2	1	UnrealInsights	64	178
1	64	1	2	UnrealInsights	64	121
1	64	2	2	UnrealInsights	64	118
1	8	1	1	UnrealInsights	8	248
1	8	2	1	UnrealInsights	8	267
1	8	1	2	UnrealInsights	8	257
1	8	2	2	UnrealInsights	8	268
1	4	1	1	UnrealInsights	4	319
1	4	2	1	UnrealInsights	4	357
1	4	1	2	UnrealInsights	4	323
1	4	2	2	UnrealInsights	4	336
4	4	1	1	UnrealInsights	16	243
4	4	2	1	UnrealInsights	16	234
4	4	1	2	UnrealInsights	16	231
4	4	2	2	UnrealInsights	16	231
4	8	1	1	UnrealInsights	32	208
4	8	2	1	UnrealInsights	32	200
4	8	1	2	UnrealInsights	32	182
4	8	2	2	UnrealInsights	32	175

4	16	1	1	UnrealInsights	64	129
4	16	2	1	UnrealInsights	64	168
4	16	1	2	UnrealInsights	64	124
4	16	2	2	UnrealInsights	64	169
4	24	1	1	UnrealInsights	96	136
4	24	2	1	UnrealInsights	96	138
4	24	1	2	UnrealInsights	96	131
4	24	2	2	UnrealInsights	96	135
8	4	1	1	UnrealInsights	32	276
8	4	2	1	UnrealInsights	32	236
8	4	1	2	UnrealInsights	32	242
8	4	2	2	UnrealInsights	32	230
8	8	1	1	UnrealInsights	64	184
8	8	2	1	UnrealInsights	64	120
8	8	1	2	UnrealInsights	64	163
8	8	2	2	UnrealInsights	64	161
8	12	1	1	UnrealInsights	96	144
8	12	2	1	UnrealInsights	96	158
8	12	1	2	UnrealInsights	96	138
8	12	2	2	UnrealInsights	96	154
0	0	1	1	UnrealPackageTool	0	146
0	0	2	1	UnrealPackageTool	0	174
0	0	1	2	UnrealPackageTool	0	170
0	0	2	2	UnrealPackageTool	0	181
1	32	1	1	UnrealPackageTool	32	77
1	32	2	1	UnrealPackageTool	32	85
1	32	1	2	UnrealPackageTool	32	84
1	32	2	2	UnrealPackageTool	32	88
1	16	1	1	UnrealPackageTool	16	98
1	16	2	1	UnrealPackageTool	16	110
1	16	1	2	UnrealPackageTool	16	88
1	16	2	2	UnrealPackageTool	16	95
1	64	1	1	UnrealPackageTool	64	87
1	64	2	1	UnrealPackageTool	64	104
1	64	1	2	UnrealPackageTool	64	77
1	64	2	2	UnrealPackageTool	64	101
1	8	1	1	UnrealPackageTool	8	121
1	8	2	1	UnrealPackageTool	8	122
1	8	1	2	UnrealPackageTool	8	109
1	8	2	2	UnrealPackageTool	8	118
1	4	1	1	UnrealPackageTool	4	123
1	4	2	1	UnrealPackageTool	4	123
1	4	1	2	UnrealPackageTool	4	133
1	4	2	2	UnrealPackageTool	4	139

4	4	1	1	UnrealPackageTool	16	102
4	4	2	1	UnrealPackageTool	16	104
4	4	1	2	UnrealPackageTool	16	97
4	4	2	2	UnrealPackageTool	16	101
4	8	1	1	UnrealPackageTool	32	82
4	8	2	1	UnrealPackageTool	32	86
4	8	1	2	UnrealPackageTool	32	80
4	8	2	2	UnrealPackageTool	32	84
4	16	1	1	UnrealPackageTool	64	82
4	16	2	1	UnrealPackageTool	64	84
4	16	1	2	UnrealPackageTool	64	75
4	16	2	2	UnrealPackageTool	64	80
4	24	1	1	UnrealPackageTool	96	78
4	24	2	1	UnrealPackageTool	96	84
4	24	1	2	UnrealPackageTool	96	81
4	24	2	2	UnrealPackageTool	96	85
8	4	1	1	UnrealPackageTool	32	98
8	4	2	1	UnrealPackageTool	32	151
8	4	1	2	UnrealPackageTool	32	92
8	4	2	2	UnrealPackageTool	32	107
8	8	1	1	UnrealPackageTool	64	87
8	8	2	1	UnrealPackageTool	64	93
8	8	1	2	UnrealPackageTool	64	86
8	8	2	2	UnrealPackageTool	64	91
8	12	1	1	UnrealPackageTool	96	98
8	12	2	1	UnrealPackageTool	96	87
8	12	1	2	UnrealPackageTool	96	85
8	12	2	2	UnrealPackageTool	96	94

Appendix E: RAM Scaling Test Data

RAM	Group	Run	Target	UBAVisualizer (Session) Time
1	1	1	UE5	10426
1	2	1	UE5	12571
1	1	2	UE5	10062
1	2	2	UE5	12346
2	1	1	UE5	10536
2	2	1	UE5	12150
2	1	2	UE5	9881
2	2	2	UE5	11787
4	1	1	UE5	10228
4	2	1	UE5	11222
4	1	2	UE5	9723
4	2	2	UE5	10983
8	1	1	UE5	3242
8	2	1	UE5	3461
8	1	2	UE5	2906
8	2	2	UE5	3112
16	1	1	UE5	2338
16	2	1	UE5	2390
16	1	2	UE5	2133
16	2	2	UE5	2310
24	1	1	UE5	2169
24	2	1	UE5	2321
24	1	2	UE5	2143
24	2	2	UE5	2304
1	1	1	UnrealInsights	645
1	2	1	UnrealInsights	626
1	1	2	UnrealInsights	476
1	2	2	UnrealInsights	587
2	1	1	UnrealInsights	455
2	2	1	UnrealInsights	530
2	1	2	UnrealInsights	429
2	2	2	UnrealInsights	501
4	1	1	UnrealInsights	185
4	2	1	UnrealInsights	167
4	1	2	UnrealInsights	157
4	2	2	UnrealInsights	180
8	1	1	UnrealInsights	158
8	2	1	UnrealInsights	172
8	1	2	UnrealInsights	154
8	2	2	UnrealInsights	195
16	1	1	UnrealInsights	208

16	2	1	UnrealInsights	200
16	1	2	UnrealInsights	182
16	2	2	UnrealInsights	175
24	1	1	UnrealInsights	146
24	2	1	UnrealInsights	202
24	1	2	UnrealInsights	140
24	2	2	UnrealInsights	188
1	1	1	UnrealPackageTool	158
1	2	1	UnrealPackageTool	209
1	1	2	UnrealPackageTool	151
1	2	2	UnrealPackageTool	176
2	1	1	UnrealPackageTool	142
2	2	1	UnrealPackageTool	157
2	1	2	UnrealPackageTool	160
2	2	2	UnrealPackageTool	165
4	1	1	UnrealPackageTool	93
4	2	1	UnrealPackageTool	88
4	1	2	UnrealPackageTool	90
4	2	2	UnrealPackageTool	86
8	1	1	UnrealPackageTool	87
8	2	1	UnrealPackageTool	87
8	1	2	UnrealPackageTool	78
8	2	2	UnrealPackageTool	92
16	1	1	UnrealPackageTool	82
16	2	1	UnrealPackageTool	86
16	1	2	UnrealPackageTool	80
16	2	2	UnrealPackageTool	84
24	1	1	UnrealPackageTool	88
24	2	1	UnrealPackageTool	98
24	1	2	UnrealPackageTool	91
24	2	2	UnrealPackageTool	94

Appendix F: Evolutionary Python Script

```

from scipy.optimize import differential_evolution
import numpy as np
from io import StringIO

# raw data copied from the excel file
agent_count = 1
raw = """10630.75 653.75
6522      337
4758      208
#N/A      #N/A
3076.25 58.25
#N/A      #N/A
1965.75 178.75
1594.75 48.75

"""
raw = raw.replace("#N/A", "nan")
raw = raw.replace("#DIV/0!", "nan")
data = np.genfromtxt(StringIO(raw), delimiter=None, skip_header=0) # load
into numpy array, with #N/A and #DIV/0! treated as NaN

cores = np.array([0, 4, 8, 12, 16, 24, 32, 64])
agents = np.array([0, 1, 4, 8])
measured = data[:, 0]
deviation = data[:, 1]
mask = ~np.isnan(measured) # Load data and mask nan values

def core_objective(params):
    A, B, C = params
    predicted = A + B / (agent_count * cores[mask] + C)
    error = np.sum((measured[mask] - predicted)**2)
    lower_bound = measured[mask] - deviation[mask]
    penalty = np.sum(np.maximum(0, lower_bound - predicted)**2) * 1e6 #
constraint applied through extreme penalty
    return error + penalty

def agent_objective(params):
    A, B, C = params
    predicted = A + B / (agents[mask] + C)
    error = np.sum((measured[mask] - predicted)**2)
    lower_bound = measured[mask] - deviation[mask]
    penalty = np.sum(np.maximum(0, lower_bound - predicted)**2) * 1e6
    return error + penalty

result = differential_evolution(core_objective, bounds=[(0, 2000), (0,
100000), (0.01, 20)], seed=1, maxiter=100000, tol=1e-9, popsize=100) #
perform evolution with specified bounds and parameters
print(f"Optimal parameters: A={result.x[0]:.2f}, B={result.x[1]:.2f},
C={result.x[2]:.2f}, Error^2={result.fun:.2f}") # print results

```

Bibliography

- [1] Henri Aïdasso, Mohammed Sayagh, and Francis Bordeleau. 2025. Build Optimization: A Systematic Literature Review. *ACM Comput. Surv.* 58, 1 (January 2025), 1–38. <https://doi.org/10.1145/3757912>
- [2] Gene M. Amdahl. 1967. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference on - AFIPS '67 (Spring)*, 1967. ACM Press, Atlantic City, New Jersey, 483. <https://doi.org/10.1145/1465482.1465560>
- [3] Ari Arnbjörnsson. 2024. Setting up an Unreal Engine Studio the Epic Way. *Epic Games Developer Community*. Retrieved October 26, 2025 from <https://dev.epicgames.com/community/learning/tutorials/8JYW/epic-for-indies-setting-up-an-unreal-engine-studio-the-epic-way>
- [4] Aaron Bangor, Philip Kortum, and James Miller. 2009. Determining what individual SUS scores mean. *Journal of Usability Studies* 4, 3 (May 2009), 114–123. <https://doi.org/10.5555/2835587.2835589>
- [5] Richard L. Baskerville. 1999. Investigating Information Systems with Action Research. *CAIS* 2, (1999). <https://doi.org/10.17705/1CAIS.00219>
- [6] John Brooke. 1996. SUS: A “Quick and Dirty” Usability Scale. In *Usability Evaluation In Industry*. CRC Press.
- [7] Ccache. Ccache — Compiler cache. Retrieved October 7, 2025 from <https://ccache.dev/>
- [8] Eelco Dolstra and Eelco Visser. 2008. The Nix Build Farm: A Declarative Approach to Continuous Integration. In *1st International Workshop on Academic Software Development Tools and Techniques (WASDeTT-1)*, June 2008. Paphos, Cyprus. Retrieved September 5, 2025 from <https://scg.unibe.ch/download/wasdet/wasdet2008-paper11.pdf>
- [9] Epic Games. 2024. Unreal Engine - README. *GitHub*. Retrieved September 27, 2025 from <https://github.com/EpicGames/UnrealEngine/blob/release/Engine/Source/Programs/Horde/README.md>
- [10] Epic Games. Unreal Engine 5.4 Release Notes. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved October 6, 2025 from https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5.4-release-notes?application_version=5.4
- [11] Epic Games. Horde Unreal Build Accelerator and Remote Compilation Tutorial for Unreal Engine. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved September 27, 2025 from <https://dev.epicgames.com/documentation/en-us/unreal-engine/horde-unreal-build-accelerator-and-remote-compilation-tutorial-for-unreal-engine>
- [12] Epic Games. Horde Server for Unreal Engine. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved October 26, 2025 from <https://dev.epicgames.com/documentation/en-us/unreal-engine/horde-server-for-unreal-engine>
- [13] Epic Games. Build Configurations Reference for Unreal Engine. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved October 20, 2025 from <https://dev.epicgames.com/documentation/en-us/unreal-engine/build-configurations-reference-for-unreal-engine>
- [14] Epic Games. Unreal Engine 5.5 Release Notes. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved October 16, 2025 from <https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5-5-release-notes>
- [15] Epic Games. Horde Analytics for Unreal Engine. *Epic Games Developer Community | Official Unreal Engine Documentation*. Retrieved October 7, 2025 from

- <https://dev.epicgames.com/documentation/en-us/unreal-engine/horde-analytics-for-unreal-engine>
- [16] Sendy Ferdian, Tjatur Kandaga, Andreas Widjaja, Hapnes Toba, Ronaldo Joshua, and Julio Narabel. 2021. Continuous Integration and Continuous Delivery Platform Development of Software Engineering and Software Project Management in Higher Education. *Jurnal Teknik Informatika dan Sistem Informasi* 7, 1 (April 2021). <https://doi.org/10.28932/jutisi.v7i1.3254>
 - [17] Jasmin Fluri, Fabrizio Fornari, and Ela Pustulka. 2024. On the importance of CI/CD practices for database applications. *Journal of Software: Evolution and Process* 36, 12 (2024), e2720. <https://doi.org/10.1002/smr.2720>
 - [18] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There's more to it than you think. *Queue* 19, 1 (2021), 20–48. <https://doi.org/10.1145/3454122>
 - [19] Franta Fulin. FASTBuild Documentation - Distribution. Retrieved October 24, 2025 from <https://www.fastbuild.org/docs/features/distribution.html>
 - [20] Josef Gattermayer and Pavel Tvrdik. 2012. Different Approaches to Distributed Compilation. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*, May 2012. 1128–1134. <https://doi.org/10.1109/IPDPSW.2012.137>
 - [21] Google. Hermeticity | Bazel. Retrieved October 6, 2025 from <https://bazel.build/basics/hermeticity>
 - [22] Sayed Moeid Heidari and Alexey A. Paznikov. 2022. Multipurpose Cloud-Based Compiler Based on Microservice Architecture and Container Orchestration. *Symmetry* 14, 9 (September 2022), 1818. <https://doi.org/10.3390/sym14091818>
 - [23] Incredibuild. 2025. Plans - Incredibuild. *Internet Archive*. Retrieved October 25, 2025 from <https://web.archive.org/web/20250825095222/https://www.incredibuild.com/pricing#compare-plans>
 - [24] Incredibuild. Incredibuild: Accelerate Compilation, Tests & Dev Workloads. *incredibuild*. Retrieved October 25, 2025 from <https://www.incredibuild.com/>
 - [25] Incredibuild. Incredibuild: The Leader in Unreal Engine Acceleration. *incredibuild*. Retrieved September 27, 2025 from <https://www.incredibuild.com/solutions/unreal-engine>
 - [26] Intel. Intel® Xeon® w9-3595X Processor - Product Specifications. *Intel*. Retrieved October 13, 2025 from <https://www.intel.com/content/www/us/en/products/sku/240482/intel-xeon-w93595x-processor-112-5m-cache-2-00-ghz/specifications.html>
 - [27] Ciera Jaspan and Collin Green. 2023. Developer Productivity for Humans, Part 4: Build Latency, Predictability, and Developer Productivity. *IEEE Softw.* 40, 4 (July 2023), 25–29. <https://doi.org/10.1109/MS.2023.3275268>
 - [28] Stephen Kemmis and Robin McTaggart. 2005. Participatory Action Research: Communicative Action and the Public Sphere. In *The Sage handbook of qualitative research, 3rd ed.* Sage Publications Ltd, Thousand Oaks, CA, 559–603.
 - [29] James R. Lewis. 2018. The System Usability Scale: Past, Present, and Future. *International Journal of Human-Computer Interaction* 34, 7 (July 2018), 577–590. <https://doi.org/10.1080/10447318.2018.1455307>
 - [30] Geunsik Lim, Lee Minh, R.J.W.E. Lahaye, and Young Ik Eom. 2014. Distributed compilation system for high-speed software build processes. January 01, 2014. 116–120. <https://doi.org/10.1109/BIGCOMP.2014.6741419>
 - [31] Rosen Matev. 2020. Fast distributed compilation and testing of large C++ projects. *EPJ Web Conf.* 245, (2020), 05001. <https://doi.org/10.1051/epjconf/202024505001>
 - [32] Jean McNiff. 2017. *Action Research: all you need to know*. SAGE, London Los Angeles New Delhi.

- [33] Memcached. Memcached Overview. Retrieved October 13, 2025 from <https://docs.memcached.org/>
- [34] Microsoft. BuildXL Documentation - Distributed-Builds. Retrieved October 24, 2025 from <https://github.com/microsoft/BuildXL/blob/main/Documentation/Wiki/Distributed-Builds.md>
- [35] Mozilla. sccache - README. Retrieved October 13, 2025 from <https://github.com/mozilla/sccache/blob/main/README.md>
- [36] Steffano Pereira, Maria Eduarda Rebelo, and George Valença. 2026. Ensuring Ethics on EdTech Technology: How to Safeguard Children in Digital Environments. In *Human-Computer Interaction – INTERACT 2025*, 2026. Springer Nature Switzerland, Cham, 216–221. https://doi.org/10.1007/978-3-032-05008-3_41
- [37] Reproducible Builds. Reproducible Builds. Retrieved October 5, 2025 from <https://reproducible-builds.org/>
- [38] Pooya Rostami Mazrae, Tom Mens, Mehdi Golzadeh, and Alexandre Decan. 2023. On the usage, co-usage and migration of CI/CD tools: A qualitative analysis. *Empir Software Eng* 28, 2 (March 2023), 52. <https://doi.org/10.1007/s10664-022-10285-5>
- [39] Charanjot Singh, Nikita Seth Gaba, Manjot Kaur, and Bhavleen Kaur. 2019. Comparison of Different CI/CD Tools Integrated with Cloud Platform. In *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, January 2019. 7–12. <https://doi.org/10.1109/CONFLUENCE.2019.8776985>
- [40] Sarah Spall, Neil Mitchell, and Sam Tobin-Hochstadt. 2020. Build scripts with perfect dependencies. *Proc. ACM Program. Lang.* 4, OOPSLA (November 2020), 1–28. <https://doi.org/10.1145/3428237>
- [41] Andrew S. Tanenbaum. 2015. *Modern operating systems* (Fourth edition ed.). Pearson, Boston.
- [42] Daniel Tutino-Galletti and Julian Gamble. 2025. Developer Iteration and Efficiency in Unreal Engine (Presented by Epic Games). In *Game Developers Conference (GDC) 2025*. Retrieved October 27, 2025 from <https://schedule.gdconf.com/session/developer-iteration-and-efficiency-in-unreal-engine-presented-by-epic-games/911185/>
- [43] Vincent Ugwueze and Joseph Chukwunweike. 2025. Continuous Integration and Deployment Strategies for Streamlined DevOps in Software Engineering and Application Delivery. *International Journal of Computer Applications Technology and Research* (January 2025), 1–24. <https://doi.org/10.7753/IJCATR1401.1001>
- [44] Valve. Steam Hardware & Software Survey. Retrieved October 7, 2025 from <https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>
- [45] Kaiyuan Wang, Daniel Rall, Greg Tener, Vijay Gullapalli, Xin Huang, and Ahmed Gad. 2021. Smart Build Targets Batching Service at Google. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, May 2021. 160–169. <https://doi.org/10.1109/ICSE-SEIP52600.2021.00025>
- [46] Kaiyuan Wang, Greg Tener, Vijay Gullapalli, Xin Huang, Ahmed Gad, and Daniel Rall. 2020. Scalable build service system with smart scheduling service. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, July 18, 2020. ACM, Virtual Event USA, 452–462. <https://doi.org/10.1145/3395363.3397371>
- [47] Jane Webster and Richard T. Watson. 2002. Analyzing the past to prepare for the future: writing a literature review. *MIS Q.* 26, 2 (June 2002), xiii–xxiii. <https://doi.org/10.5555/2017160>
- [48] Roel Wieringa and Ayşe Morali. 2012. Technical Action Research as a Validation Method in Information Systems Design Science. In *Design Science Research in Information Systems. Advances in Theory and Practice*, 2012. Springer, Berlin, Heidelberg, 220–238. https://doi.org/10.1007/978-3-642-29863-9_17

- [49] Shuai Yang. 2025. The Impact of Continuous Integration and Continuous Delivery on Software Development Efficiency. *Journal of Computer, Signal, and System Research* 2, 3 (April 2025), 59–68. <https://doi.org/10.71222/pzvfqm21>
- [50] Ying Zhang, Yanyan Jiang, Chang Xu, Xiaoxing Ma, and Ping Yu. 2015. ABC: Accelerated Building of C/C++ Projects. In *2015 Asia-Pacific Software Engineering Conference (APSEC)*, December 2015. 182–189. <https://doi.org/10.1109/APSEC.2015.27>
- [51] 2026. Reduce the log level for UBA connection failures from Error to Warning so that cooks are not failed if you are offline or lose connection during a build. - Development / Pipeline & Plugins. *Epic Developer Community Forums*. Retrieved April 24, 2026 from <https://forums.unrealengine.com/t/reduce-the-log-level-for-uba-connection-failures-from-error-to-warning-so-that-cooks-are-not-failed-if-you-are-offline-or-lose-connection-during-a-build/2704028/2>
- [52] Memory Wall Bottleneck: AI Compute Sparks Memory Supercycle. *TrendForce*. Retrieved April 22, 2026 from <https://www.trendforce.com/insights/memory-wall>
- [53] Unreal Game Sync (UGS) for Unreal Engine | Unreal Engine 5.7 Documentation | Epic Developer Community. *Epic Games Developer*. Retrieved June 7, 2026 from <https://dev.epicgames.com/documentation/unreal-engine/unreal-game-sync-ugs-for-unreal-engine>